中国研究生创新实践系列大赛

“华为杯”第二十二届中国研究生数学建模竞赛

学　　校	浙江财经大学
参赛队号	25114820009
队员姓名	1.胡锐
	2.於豪祺
	3.王奇超

中国研究生创新实践系列大赛

“华为杯”第二十二届中国研究生
数学建模竞赛

题 目：高速列车轴承智能故障诊断的建模与研究

摘 要:

高速列车轴承智能故障诊断受限于真实故障数据稀少，难以训练出高性能的智能诊断模型。随着人工智能技术的发展，迁移学习使基于实验数据的模型能够迁移至真实工况并进行应用成为了可能。但是智能诊断模型的可解释性较差，使得工作人员对诊断结果缺乏信任以至于很少在真实情况使用智能诊断模型。本**文通过分析并挑选出时域、频域以及时频域的特征，构建多模态故障诊断模型，采用对抗域适应的方法来解决迁移问题，并对模型进行事前、事中、事后解释。**最终给出安全可靠的轴承故障诊断模型。

针对问题一，首先对数据进行筛选与处理，**根据轴承故障机理将基座数据剔除掉**，并只**保留与各个端口最近的传感器数据**。为了提高迁移模型的效率与泛化能力，本文通过**多相 FIR 重采样**将源域数据的采样频率**转化为 32kHz**，与目标域保持一致。接着为了使迁移模型具有更好的可解释性，本文分别从**时域、频域和时频域**三个角度构建了反映振动信号信息的特征。使用常见的**描述性统计量作为时域特征**。对振动信号序列**进行快速傅里叶变换**后，**采用均值、标准差、频谱重心、频带能量比和故障特征频率幅值**作为频域特征。并**采用 db4 小波基对原始振动信号进行 3 层分解，算出 8 个频带子节点的能量**作为时频域特征。最后通过**散点矩阵图、堆叠柱状图、条形图、热力图**等可视化工具从 **26 个预选特征**中筛选出 **21 个显著特征，**分别是时域中的**均方根值、峰值、峰值因子与峭度**，频域中的**均值、标准差、频谱重心、前三段频带能量比、三种故障频率特征能量和基频能量**，以及时频域中的**小波包分解子节点能量**。并对各个特征的作用进行了分析，最终构建能够全面反映轴承故障状态的特征集合。

针对问题二，本文根据**轴承工作状态以及故障尺寸**将样本数据划分为 12 类，将时域信号**按照 1024 点为窗口进行切片**，作为时域输入。基于快速傅里叶变换对时域信号进行频谱分析，取**前 513 维幅值谱作为频域输入**，并将问题一中的提取的特征集合作为特征输入。本文提出了一个**基于深度学习的源域故障诊断模型**，该模型首先**提取了三种输入的特征**，并**采用域不变/域特定特征提取器来分离域不变特征**，最后**对所有特征采用注意力加权融合并分类**。本文**采用交叉熵损失和条件对比损失**来训练模型的分类性能，**采用余弦相似度损失训练域不变/域特定特征提取器确保可以分离出域不变特征**。实验结果表明，该模型在测试集上的准确率达到 **99.61%**，显著优于**传统 CNN、ResNet 和 LSTM** 模型，验证了模型的强鲁棒性。

针对问题三，本文提出一种**基于对抗域适应的跨工况迁移学习方法**。即在原来的诊断模型的基础上，**引入域判别器**，将诊断模型中提取的特征输入到域判别器中，判断数据的来源。在训练过程中训练诊断模型的分类能力，同时确保提取的特征无法让域判别器正确识别。最终实验结果显示，目标域数据集被**主要预测为球故障和外圈故障两类**，t-SNE 可视化表明，**源域与目标域在嵌入空间中实现了混合**，证实了域对抗训练的有效性。

针对问题四，本文**从事前、迁移过程和事后三个方面**对诊断模型的可解释性进行了分析。首先借助**全连接层和注意力机制**将事先挑选的故障特征和原始信号的特征进行展示。发现故障特征在模型诊断的过程中起到了信息补充的作用以及对后期模型的稳定做出了贡献。接着在迁移过程可解释性方面，借助 **t-SNE 降维并进行特征分布对齐可视化并画出域分类器准确率曲线，**直观展示了源域与目标域特征逐渐对齐的过程。域分类器准确率**从起初的 0.8 下降到 0.5** 并持续保持在 0.5 左右，说明了迁移学习的有效性。最后使用与事前相同的方法，对迁移后的模型进行分析。结果表明迁移后的模型在**外圈故障特征频率能量和滚动体故障特征频率能量**上的关注度明显提升。从总体上来说减少了对事先提取的故障特征的依赖，但是实现了更快的收敛速度和更好的模型性能，进一步提高了模型在目标域的诊断精度与泛化性能。

关键词：高速列车轴承；时域特征；频域特征；时频域特征；智能故障诊断；多模态特征；迁移学习；对抗域适应；特征可视化；模型可解释性

目录

一、 问题重述.....5
1.1 问题背景.....5
1.2 问题提出.....5
二、 模型假设与符号说明.....5
2.1 模型假设.....5
2.2 符号说明.....6
三、 问题一模型建立与求解.....6
3.1 问题一分析及建模思路.....6
3.2 问题一模型的建立.....7
3.2.1 数据筛选思路.....7
3.2.2 多角度故障特征构造.....8
3.3 问题一模型的求解.....10
3.3.1 振动信号特征提取与分析.....10
3.3.2 时域特征提取与分析.....11
3.3.3 频域特征提取与分析.....12
3.3.4 时频域特征提取与分析.....14
3.4 问题一小结.....15
四、 问题二模型建立与求解.....17
4.1 问题分析和模型建立思路.....17
4.2 基于深度学习的源域故障诊断模型建立.....18
4.2.1 数据预处理与数据集构造.....18
4.2.2 轴承诊断模型设计动机.....19
4.2.3 BearingQTFFC 轴承诊断模型结构.....19
4.2.4 损失函数与优化目标.....20
4.2.5 模型评估.....21
4.3 问题二的求解.....21
4.4 问题二的小结.....23
五、 问题三模型建立与求解.....24
5.1 模型总体思路与架构设计.....24
5.2 问题三模型的构建.....24
5.2.1 目标域数据建模与域差异分析.....24
5.2.2 多源信号特征提取与表示构建.....25
5.2.3 故障分类器设计与源域监督训练.....25
5.2.4 域判别器与对抗训练机制.....26
5.2.5 联合优化目标函数的构建.....26
5.3 问题三结果与分析.....26
5.4 问题三的小结.....28
六、 问题四模型建立与求解.....29
6.1 问题分析及建模思路.....29
6.2 问题四模型建立.....29
6.2.1 事前、事后可解释性模型建立.....29
6.2.2 迁移过程可解释性模型建立.....30

6.3 模型求解.....30
6.4 问题四小结.....33
七、 模型评价.....34
7.1 模型的优点.....34
7.2 模型的不足.....34
八、 参考文献.....35
九、 附录.....36

一、 问题重述

1.1 问题背景

高速列车的轴承是决定高速列车走行系统是否正常运转的关键，而轴承的智能故障诊断又是对高速列车运行安全和人员安全的重要保障。一旦轴承在高速重载工况下发生故障，若未能被及时准确地诊断，容易引发严重的运行事故，造成零件故障，损坏部件本身，而且极易诱发高速列车整个走行系统的瘫痪，不仅影响列车的延误更容易引发人身安全事故，造成巨大的经济损失和不良的社会影响。为了高速列车的正常运营和乘客的人身安全。因此，发展智能、可靠的轴承故障诊断技术，实现对列车轴承健康状态的实时监测与早期预警，是保障轨道交通系统安全运行的重大需求。当前在实际情况中，由于对故障轴承的及时处理导致在途列车的真实故障数据非常少，用于训练模型的标记数据并不充足完备，导致训练数据分布严重失衡，限制了利用模型去研究并用于实践的发挥。

由于实际数据的条件限制，利用台架实验环境下采集的大量轴承数据，其故障演化机理与实际在途列车轴承数据相似，故通过解析大量试验台数据，提取表现显著的具有代表性的轴承故障特征，并建立相关模型，引入迁移学习技术旨在减少试验台数据与车载数据之间的分布差异，将诊断模型的诊断结论迁移至实际运营车辆，解决实际车辆轴承智能诊断问题。

如此可以有效缓解因故障样本稀缺导致的样本不平衡问题，提升模型在真实复杂工况下的泛化能力和诊断精度，为“实验室模型”迁移至现实实践问题，提供一条可行的技术路径，对推动轨道交通智能化运维具有重要的理论意义和工程价值。

1.2 问题提出

问题一：多角度的特征分析与提取

将 161 个轴承试验台架振动数据文件作为源域数据集，那根据什么特征可以对源域数据集分类？又根据什么可以筛选有显著故障的轴承数据？如何对整体数据集进行特征的提取？又如何对整体数据进行特征分析？

问题二：轴承故障分类模型

在问题一提取的故障特征基础上，如何划分源域训练集和测试集？如何设计合适的诊断模型实现源域诊断任务？如何评价诊断结果？

问题三：迁移学习

在问题二设计的故障诊断模型的基础上，源域数据和目标域数据的共性是什么？差异特征是什么？合适的迁移学习方法如何设计？如何构建目标域诊断模型？如何对目标域未知标签的数据分类和鉴别？如何分析迁移的结果以及识别对应标签？

问题四：“黑箱“问题的可解释性

如何解决迁移学习模型在跨工况、跨设备故障诊断中的透明性问题？如何对迁移诊断的事前、迁移过程以及事后进行可解释性分析？

二、 模型假设与符号说明

2.1 模型假设

假设 1：采样频率统一性假设。假设将源域数据重采样至 32kHz 能保留故障关键成分，且与目标域采样频率一致，不会引入显著信息损失或失真。

假设 2：故障独立性假设。假设不同故障类型在特征空间中线性可分或可通过非线性映射分离，且故障尺寸变化对特征分布的影响是连续的

假设 3：目标域故障单一性假设。假设目标域只有一种故障情况，不存在多种故障状态。

假设 4：注意力机制有效性假设。假设模型中的注意力权重能准确标识信号关键时段或频段，为故障定位提供可视化依据。

假设 5：工程部署环境假设。假设实际应用环境中计算资源可支持模型推理，且振动数据采集系统稳定可靠，满足实时诊断需求。

2.2 符号说明

符号	内容说明
$y(k)$	源域数据的振动信号
$A(f)$	源域样本的频谱
E_j	第 j 个频带的能量比
f_i	i 的特征频率
$c_l(n)$	小波包系数
$sim(\cdot)$	余弦相似度
τ	温度参数
$h(\cdot)$	FIR 滤波器冲击效应
x_{time}	经过 $1D-CNN$ 提取的时间域基特征
x_{freq}	FFT 后经 $1D-CNN$ 提取的频域特征
x_{hand}	人工挑选出的特征
Θ_F	各特征提取模块参数
Θ_C	分类器参数
Θ_D	域判别器参数
ℓ_i	i 的损失函数
W_i	i 的权重值

三、 问题一模型建立与求解

3.1 问题一分析及建模思路

对于高速列车轴承故障诊断任务，当前的做法主要还是以专家经验来对轴承故障进行诊断。上述做法相对于以大数据为驱动的人工智能方法来说受主观意识的影响较大。但使用人工智能的方法，需要使用大量数据进行训练，碍于列车安全运营的原则，真实故障数据较为稀缺。所以，目前的解决方法是使用故障演化机理与在途列车轴承相似的台架实验环境下的实验数据，作为诊断模型的训练数据，之后再将训练好的模型所具备的功能迁移到目标域的诊断当中。现实情况中，由于轴承运作时的工况不同，会导致将模型迁移到目标域后诊断效果不理想，会影响高速列车的安全运行。问题一是在源域数据中探寻一组可以分辨轴承是否故障的特征，并用于后续的诊断任务。

根据问题一的要求，需要从提供的 161 个源域数据中筛选出代表性数据，通过合适的方法与指标进行特征分析，并对特征进行提取用于后续的诊断任务。

在诊断任务中，特征的选取关乎着模型的诊断效果。选取的特征只有包含用于诊断任务的信息，才能将轴承是否故障和故障类型判断出来。在数据选取方面，尽可能地挑选与目标域工况相差不大的数据集，这样会使迁移过后的模型仍然具有强劲的分辨能力。为了使诊断模型在迁移后仍能具有较好的诊断效果，将数据在某一工况上进行对齐并保留其他工况的多样性以保证后续诊断模型的泛化能力。在特征提取过程中，将会从时域、频域、时频域三个方面进行，保证特征集包含诊断所用到的信息。问题一的模型建立流程图如图 3-1 所示。

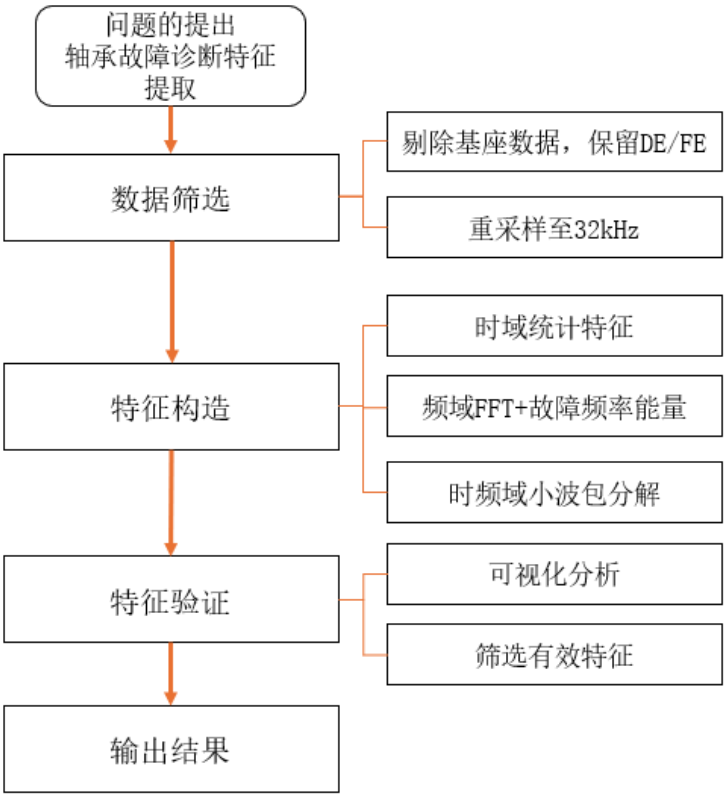


图 3-1 问题一建模流程图

3.2 问题一模型的建立

3.2.1 数据筛选思路

首先，根据轴承故障机理，本文将基座数据从后续的工作中剔除。题目提供的数据包括 161 个具有代表性的轴承试验台架振动数据作为源域数据集和真实情况中的 16 个轴承故障数据作为目标域数据集。在源域数据集中已知在一次轴承实验中会用三种不同位置的传感器采集振动信号，分别是驱动端数据（DE）、风扇端数据（FE）和基座数据（BA）。根据轴承故障机理可知，当轴承故障区域与其他部件接触后会产生异常振动，采集器会接收到正常与异常的振动信息，但是当采集器距离轴承较远时接收到的故障信号的幅值将会衰减。根据题目给出的采集器位置来看，驱动端与风扇端的采集器靠近轴承能够清晰地获取驱动端与风扇端轴承的振动信号，而基座处的采集器由于位置与采集器较远，振动信号在传递的过程中高度衰减，不能很清楚地记录轴承运作的相关信息，所以将基座数据剔除。并且每个传感器都收集这三个端点的数据，因为上述阐述的原因，本文只保留与传感器相邻最近的数据。

另外，为了后续使迁移后的模型适应目标域的数据形式，本文将源域数据中采样频率为 12kHz 和 48kHz 的记录通过多相 FIR 重采样转化为 32kHz 的振动信号数据。其转换公式如下：

$$x(t)=\sum_{k} y(k)h(nM-kL) \tag{3-1}$$

其中 $y(k)$ 表示源域数据的振动信号，$h(\cdot)$ 为 FIR 滤波器冲激响应，M 与 L 分别为下采样因子和上采样因子。

这样做有以下两点原因：

1、提高迁移学习的效率与泛化能力。将采样频率统一到 32kHz 后，可以消除采样频

率不同带来的分布偏移，使预训练模型在迁移学习的过程中不需要再去适应不同的采样频率。

2、保留完整的高频冲击分量。12kHz 的原始奈奎斯特频率是 6kHz，重采样到 32kHz 后可以将 6-12kHz 的信息完整保留下来。

最后，源域数据集中还给出了轴承的故障尺寸、轴承的载荷情况以及轴承转速。本文将不对这些数据进行处理，在之后预训练模型的过程中将考虑上述所有的工况以增加模型的泛化能力，保证预训练模型在通过迁移学习后还能保证故障诊断的精度。

3.2.2 多角度故障特征构造

根据轴承故障通过振动引发的周期性冲击的机理，从三个方面构建轴承故障特征集。分别是时域波形图表现的信号整体的强弱、波动与冲击性，频域谱线表现的信号在不同频率上的能量分布，以及时频域表现的频率随时间的变化情况。通过这三个不同角度的特征，表征故障轴承运行时表现出来的异常信号波动。在之后的模型建立过程中，使用 $x_i(t)$ 来表示第 i 个源域样本经过重采样后的振动信号序列，其序列的长度为 N，采样频率为 f_s。上述三个角度的故障特征定义如下：

（1）时域统计特征：分别计算振动信号的均值（μ）、方差（σ^2）、均方根值（RMS）、峰值（PK）、峰值因子（CF）、偏度（SK）、峭度（KU）。这些特征反映了振动信号的平均水平、能量强弱、冲击性和波形形态，综合来看可以对振动信号序列的状态进行全面的量化。具体公式如下所示：

$$\mu_i = \frac{1}{N}\sum_{n=1}^{N} x_i(n) \tag{3-2}$$

$$\sigma_{\mathrm{i}}^2 = \frac{1}{N}\sum_{n=1}^{N} (x_i(n)-\mu_i)^2 \tag{3-3}$$

$$RMS_i = \sqrt{\frac{1}{N}\sum_{n=1}^{N} x_i^2(n)} \tag{3-4}$$

$$PK_i = \max\left|x_i(t)\right| \tag{3-5}$$

$$CF_i = \frac{PK_i}{RMS_i} \tag{3-6}$$

$$SK_i = \frac{\frac{1}{N}\sum_{n=1}^{N}(x_i(n)-u_i)^3}{\sigma_i^3} \tag{3-7}$$

$$KU_i = \frac{\frac{1}{N}\sum_{n=1}^{N}(x_i(n)-u_i)^4}{\sigma_i^4} \tag{3-8}$$

上述公式中每个特征的下标为 i，表示源域中第 i 个样本的特征。例如 μ_i 表示第 i 个样本的均值。

（2）频域能量特征：本文对源域数据中的振动信号序列进行快速傅里叶变换（FFT），用得到的频谱来反映振动信号在频域里的能量分布与故障特征，用 $A_i(f)$ 来表示第 i 个源域样本的频谱，由于实数信号的频谱是共轭对称的，所以只取其中的正频率，这样频谱的长度为 $\frac{1}{2}N$。之后分别计算频谱幅值的均值（μ_A）与标准差（σ_A），反映频谱幅值的集中位置与波动的大小，初步掌握频谱的大致特征。其公式如下：

$$\mu_{A,i} = \frac{2}{N}\sum_{k=0}^{N/2} A_i(f_k) \tag{3-9}$$

$$\sigma_{A,i} = \sqrt{\frac{2}{N}\sum_{k=0}^{N/2}(A_i(f_k)-\mu_{\mathrm{A,i}})^2} \tag{3-10}$$

其中 $f_k = \frac{kf_s}{N}$ 表示正频率。

接着，本文选择使用频谱重心（C_{sc}）作为频域特征集中的一员。因为轴承在正常运作时振动信号的能量主要集中在低频区域，而当轴承故障后高频区域的能量将明显增强，所以可以用频谱重心来反映能量重心在轴承故障后上移。其计算公式如下：

$$C_{sc,i} = \frac{\sum_{k=0}^{N/2} f_k A_i(f_k)}{\sum_{k=0}^{N/2} A_i(f_k)} \tag{3-11}$$

为了更好地反映轴承在故障之后，反映出来的振动信号的异常，加入频带能量比来对其进行量化。当轴承出现故障后，高频的能量占比将会升高。本文将正频率区间划分为四段不重叠的频带，使得从低到高的能量占比清晰地展现出来。计算公式如下：

$$E_j = \frac{\sum_{f\in F_j}|A(f)|^2}{\sum_{f\in F}|A(f)|^2} \tag{3-12}$$

其中 E_j 表示第 j 个频带的能量比，F 包含全部频带，F_j 表示第 j 个频带。

最后，根据题目中已给出的轴承尺寸参数和题目中表 2 给出的公式，算出源域数据集中的轴承故障特征频率。分别为外圈故障特征频率（f_{BPFO}）、内圈故障特征频率（f_{BPFI}）和滚动体故障特征频率（f_{BSF}），另外还计算出转频（f_r）用来区分纯旋转激励和轴承缺陷激励。由于上述特征值是用公式计算出来的理论值，在真实的工况中会因为转速、缺陷的大小、机器的抖动等情况会使故障特征频率发生轻微的偏移。为了解决这个问题，本文在故障特征频率附近 10Hz 的窄带内计算振动信号的能量，用来描述故障频率的特征。其计算公式如下：

$$E_{fault} = \sum_{|f_k - f_{fault}|<\Delta f}|A(f_k)|^2 \tag{3-13}$$

其中 $fault \in \{BPFO, BPFI, BSF\}$，$f_{fault}$ 表示故障类型的特征频率，Δf 为窄带的范围 10Hz。

（3）时频域能量特征：在上文中本文已经提取了振动信号的时域和频域特征，为了更加精细地表征信号在不同时间和频率范围内的局部能量分布并使之后的诊断模型更加稳健，本文引入小波包分解（Wavelet Packet Decomposition,WPD）能量特征作为时频域特征。本文采用 db4 小波基对原始振动信号进行 3 层小波包分解，得到 8 个频带子节点。对

每个节点计算能量，接着进行归一化转化为能量占比。

对每个节点 l 的小波包系数 $c_l(n)$ 计算能量，计算公式如下：

$$E_l = \sum_{n=1}^{N_l} \left| c_l(n) \right|^2 \tag{3-14}$$

接着为了消除采样长度和信号幅值的影响，分别算出每个节点的能量占比。计算公式如下所示：

$$P_k = \frac{E_l}{\sum_{j=1}^{8} E_j} \tag{3-15}$$

其中 E_j 为第 j 个节点的能量。

上述三个角度的特征共同构成了高维的特征向量 $v \in R^{26}$，这些特征可以从不同角度充分地表现出轴承故障信号。具体的符号表示和作用会在特征提取后用表格展示。

3.3 问题一模型的求解

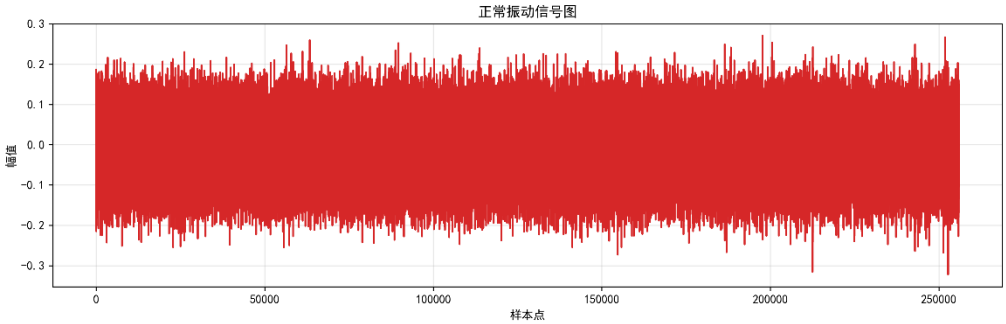
3.3.1 振动信号特征提取与分析

通过上述的数据筛选思路，本文将数据中的基座数据进行了剔除，并且只选取靠近采集器最近的轴承信号信息，最后将筛选出来的数据通过多相 FIR 重采样得到采样频率一致的数据，方便后续的故障诊断与模型的迁移学习。本文将在之后的篇幅中使用可视化来说明提到的特征是否能够判断轴承是否出现故障。但考虑到数据的工况组合较多，为避免臃肿，所以本文选择有代表性的工况进行比较。选取的工况详请如表 3-1 所示。

表 3-1 代表性工况故障情况

故障类型	故障尺寸	载荷
外圈故障	0.021	3
内圈故障	0.021	3
滚动体故障	0.021	3
无故障	无	3

故障尺寸选取 0.021 是根据轴承故障机理启发，因大尺寸的缺陷特征变化非常明显，更容易区分不同的故障，这使得对比起来更加的直观。选择载荷高的工况与上述的原因一致，也是为了让对比图更加的直观。将上述选取的代表数据的振动信号绘制折线图，结果如图 3-2 所示。



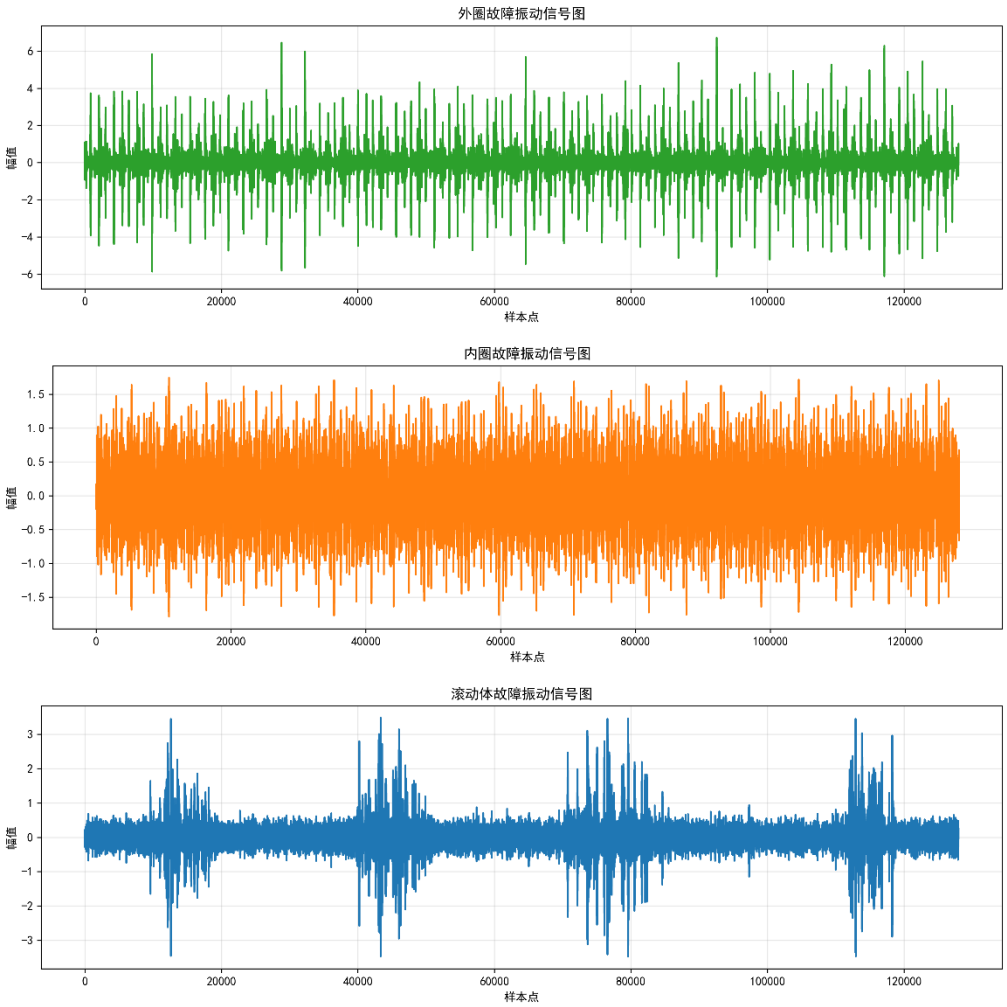


图 3-2 不同情况振动信号图

从图 3-2 可以看出正常信号波动较为平稳，数值大概在-0.2 到 0.2 之间。外圈故障振动信号图可以明显地看出当外圈出现缺陷时，时域波形上会出现间隔固定的尖峰，从整体上来看尖峰不是很密集，而且其峰值可以超过 6。从内圈故障振动信号图可以看出，时域波形的峰值在 1.5 左右，整体上来看波形幅值差距较小所以看起来非常的密集。最后从滚动体故障振动信号图可以看出，时域波形峰值在 3 附近，而且呈现出峰值密集区域和低幅值区域交替出现的现象。从上述的四幅时域波形图可以直观地看出轴承在受到不同的损坏时会有不同的波形特点，所以原始数据波形图已经可以作为诊断模型的训练数据。但为了获取更加全面的信号特征，本文还对时域、频域和时频域特征进行了分析。

3.3.2 时域特征提取与分析

对于时域特征，本文采用散点矩阵图来可视化时域特征。具体结果如图 3-3 所示

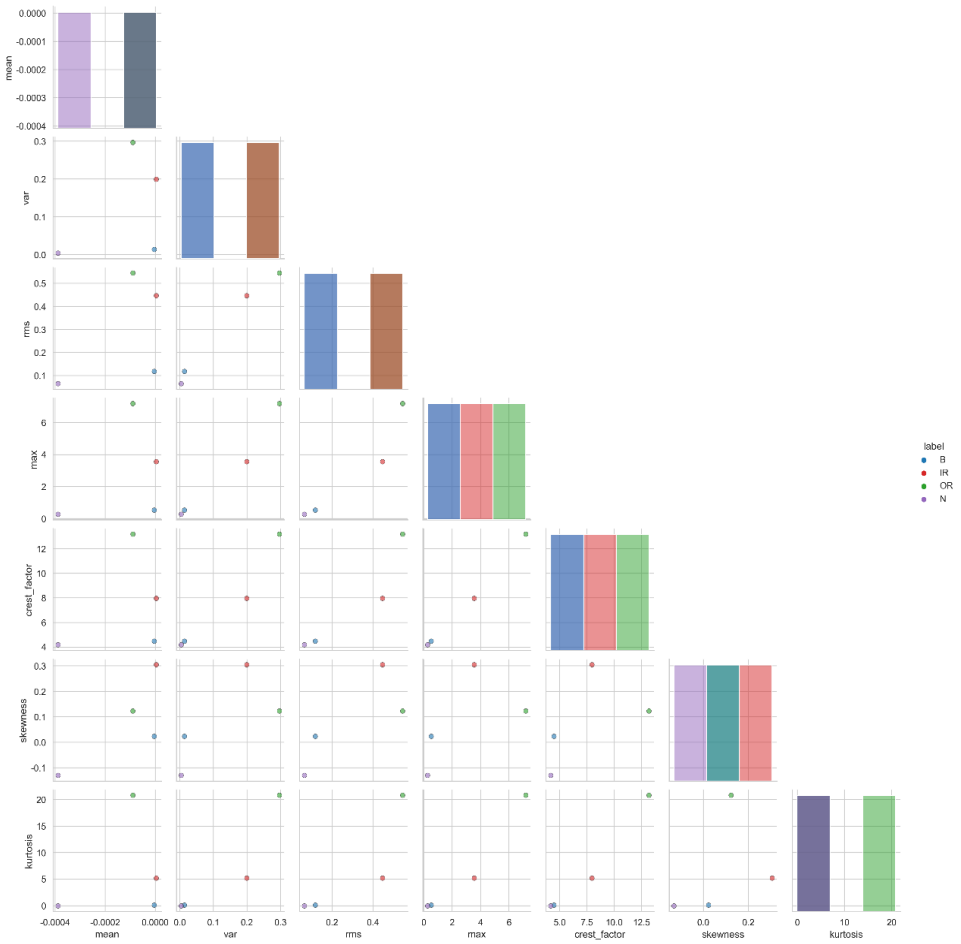


图 3-3 散点矩阵图

根据图中的直方图的颜色重叠程度来看，均值与偏度基本都接近于 0 区分度较差。其余特征在不同故障类别之间有明显的差异，对于后续的诊断作用较大。从散点图可以看出，方差与均方根值交叉的散点图中趋近于直线，说明两个特征高度相关。其余特征的散点图不存在高度相关的情况。最终对于时域特征来说，本文认为均方根值、峰值、峰值因子和峭度能够表现出轴承是否有故障。

3.3.3 频域特征提取与分析

对于频域特征，首先将频谱图的均值、标准差和频谱重心进行了可视化，结果如图 3-4 所示。

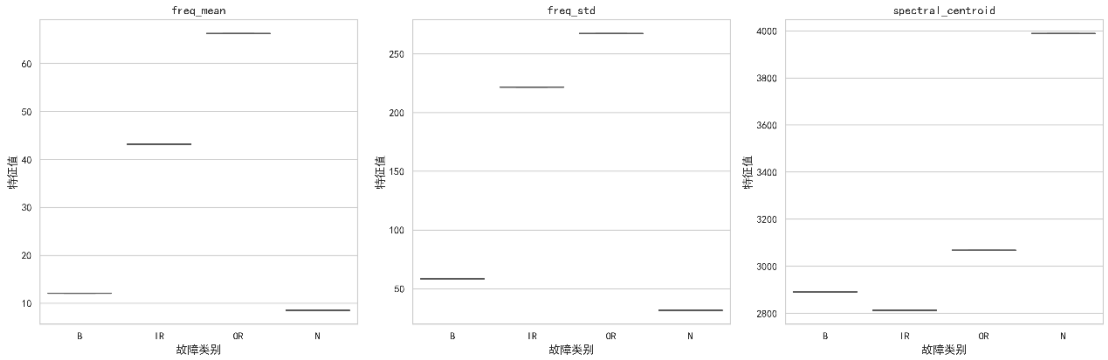


图 3-4 故障类别与特征值

根据图中所展示的分布信息，直观地看出不同故障的均值与标准差相差很大，说明轴承有不同的缺陷后，其频谱幅度值的分布也会改变。对于频谱重心来说，虽然三种故障的

值相近，但对于正常的轴承来说，如果出现故障频谱重心会显著下降，进而借助频谱重心能够分辨出轴承是否存在故障。接着本文使用堆叠柱状图来展示频带能量比，结果如图 3-5 所示。

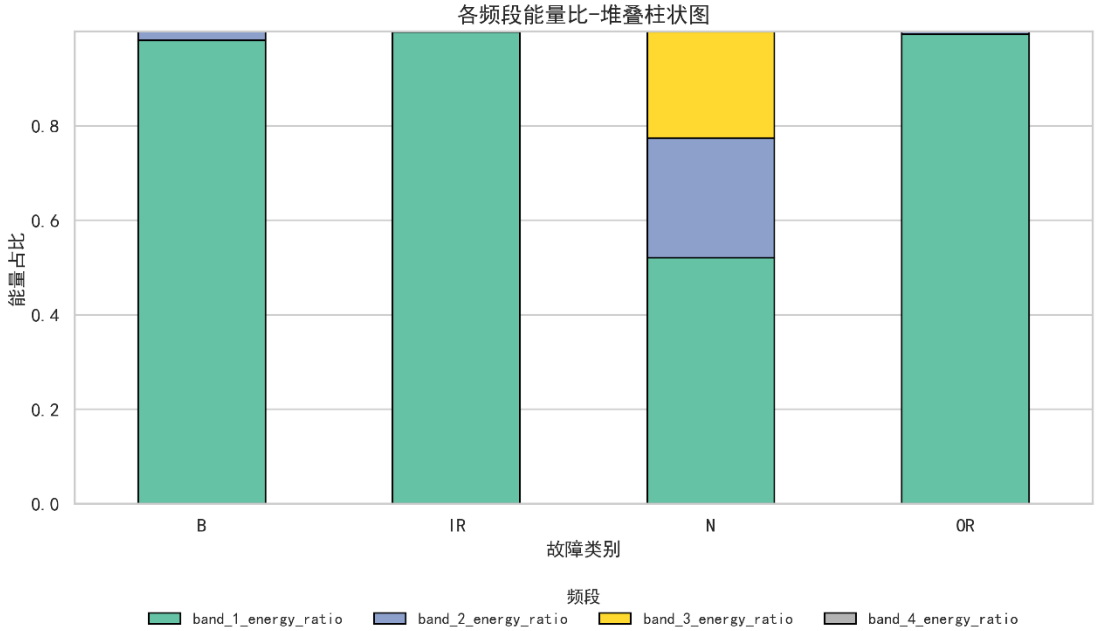


图 3-5 不同故障类别的能量占比

图中每个颜色的柱状图高度代表对应频段的能量比，可以看出三种故障的能量几乎都集中在第一段，正常类的能量分布比较分散第一段的能量占比约为 50%，第二段和第三段能量占比约为 25%。总的来说根据第一、二、三段的能量比可以分辨出轴承是否故障，但几乎分辨不出故障时那种类型的。所以说前三段的能量比可以为诊断工作提供更多的分辨轴承是否故障的信息。因为第四段的能量比无法发挥分辨能力，所以将其从特征集合中剔除。

最后本文将三种故障特征频率和基频频率的能量幅值用柱状图进行展示，结果如图 3-6 所示。

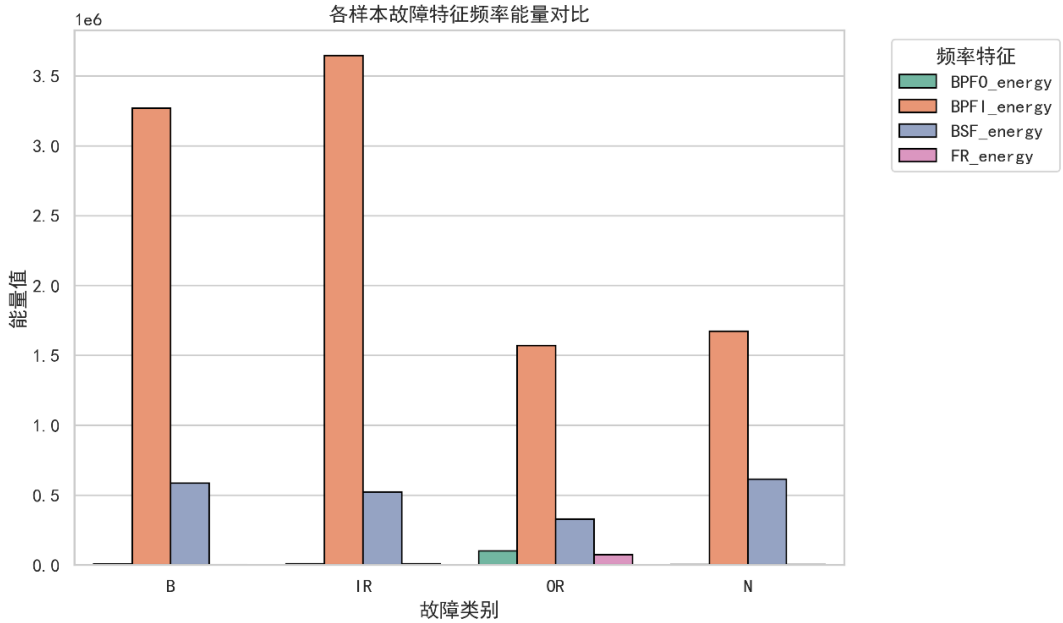


图 3-6 各样本故障特征频率能量对比

从上图可以看出外圈故障轴承的外圈故障特征频率的能量要高于其他故障的能量，所以用来作为诊断外圈故障的信息。对于内圈故障频率的能量来说，内圈故障的轴承要高于其他类型的故障轴承，所以可以用来作为诊断内圈故障的信息。但对于滚动体故障特征频率的能量来说，只能区分故障类型是哪一种，而不能很好地提供分辨滚动体故障和正常轴承的信息，所以只用来在已知轴承出现故障进而区分故障类型的条件中丰富诊断信息。对于基频的能量来说，外圈故障的能量值要大于其他故障的能量值，能够提供诊断外圈故障的信息。

3.3.4 时频域特征提取与分析

对于时频域特征来说，本文使用小波包分解得出 8 个子节点，在每个子节点算出能量，之后进行归一化算出能量占比。为了更好地展示计算的结果，本文将其中的数据各乘 7，最终画出小波包能量热力图，结果如图 3-7 所示。

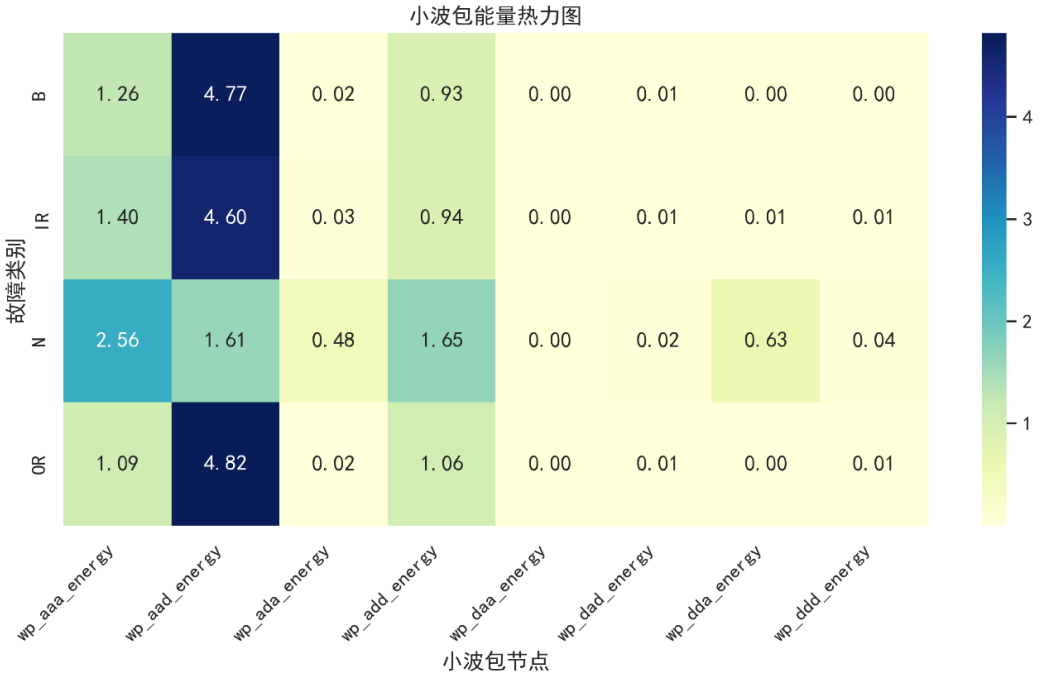


图 3-7 小波包能量热力图

从热力图中可以明显地看出正常轴承在 aaa、aad、ada、add、dda 这五个节点的能量值与其他三种故障的能量值有着明显的差异，所以根据小波包分解节点的能量数据可以诊断轴承是否出现故障。但是对于区分故障是那种类型，节点数据还不能很清晰地分辨出来。所以在后文使用小波包分解节点的能量数据提供诊断轴承是否故障的信息。

3.4 问题一小结

对于问题一，本文从时域、频域和时频域三个角度对诊断特征进行分析，并根据各自蕴含的辨别信息提取出有用的特征。提取出来的特征如表 3-2 所示。

表 3-2 诊断特征分析

特征名称	挑选角度	符号	作用
均方根值	时域	RMS	诊断故障类型
峰值	时域	PK	诊断故障类型
峰值因子	时域	CF	诊断故障类型
峭度	时域	KU	诊断故障类型
均值	频域	freq_mean	诊断故障类型
标准差	频域	freq_std	诊断故障类型
频谱重心	频域	spectral centroid	诊断是否故障
频带能量比（3 段）	频域	band_i_energy_ratio	诊断是否故障
外圈故障特征频率能量	频域	BPFO_energy	诊断外圈故障
内圈故障特征频率能量	频域	BPFI_energy	诊断内圈故障
滚动体故障特征频率能量	频域	BSF_energy	确定故障后诊断滚动体故障
基频能量	频域	FR_energy	诊断外圈故障
小波包子节点能量(7 子节点)	时频域	wp_energy	诊断是否故障

上表已展示各个特征的属于的空间与在诊断时发挥的作用，可以从多角度诊断出故障的类型。虽然在目标域的数据集中没有提供故障特征频率，但本文会在第二问中使用故障特征频率能量进行训练，以保证训练好的模型拥有故障特征频率的知识。本文也将振动信

号图作为模型输入，因为通过上述的分析可以得出，信号源数据已经可以初步判断是否故障和故障类型。

四、 问题二模型建立与求解

4.1 问题分析和模型建立思路

在源域故障诊断这一关键领域中，核心目标在于从复杂且高维的振动信号中精准提取出能够有效表征设备故障状态的特征信息，进而实现多类故障状态的精确分类与识别。在实际工业场景中，机械设备运行过程中产生的振动信号蕴含着丰富的故障信息，但这些信号往往具有高维度、非线性、非平稳等复杂特性，给故障特征的提取带来了巨大挑战。

传统的故障特征提取方法，通常依赖于专家基于特定领域知识所积累的丰富经验。专家们会依据信号的时域、频域特性，设计出诸如均值、方差、峰值、频谱能量等特征指标，试图从不同角度刻画信号的特征。然而，这种方法存在诸多局限性。一方面，手工设计的特征往往只能捕捉到信号的表层信息，难以充分挖掘隐藏在信号深层次的、更为复杂的故障特征模式。随着设备复杂度的不断提高以及故障类型的日益多样化，这些浅层特征难以全面、准确地反映设备的真实故障状态。另一方面，故障特征提取方法的泛化能力较差。由于不同设备、不同工况下的振动信号特性存在显著差异，基于特定设备或工况设计的特征提取方法，在应用到其他设备或工况时，可能无法取得理想的诊断效果，如果需要重新进行特征设计和调整，这无疑增加了故障诊断的难度和成本。

相比之下，基于深度学习的源域故障诊断方法展现出了巨大的优势。深度学习模型具有强大的自动特征学习能力，能够通过多层非线性变换，自动从原始振动信号中学习到不同层次的特征表示。从浅层的边缘、纹理等特征，到深层的语义、抽象特征，深度学习模型可以逐步挖掘出信号中蕴含的深层次故障信息，无需人工手动设计特征，大大减少了对专家经验的依赖。同时，深度学习模型具有良好的泛化能力，通过在大规模数据上进行训练，能够学习到通用的特征表示模式，从而在不同设备、不同工况下都能实现较为准确的故障诊断，为解决复杂工业环境下的故障诊断问题提供了新的思路和方法。

因此，本题采用多模态特征融合与对比学习约束的建模思路。一方面，利用深度卷积网络从原始信号中自动提取时域与频域特征；另一方面，结合问题一中提取的特征作为补充，提高模型对细微故障的敏感性。此外，本文在模型中引入域不变/域特定特征分解模块，为后续的迁移学习任务奠定基础。在优化目标上，本文不仅使用交叉熵损失进行分类，还结合条件对比损失增强同类样本的特征聚合性，从而提升模型的判别能力与泛化能力。问题二的模型建立流程图如图 4-1 所示。

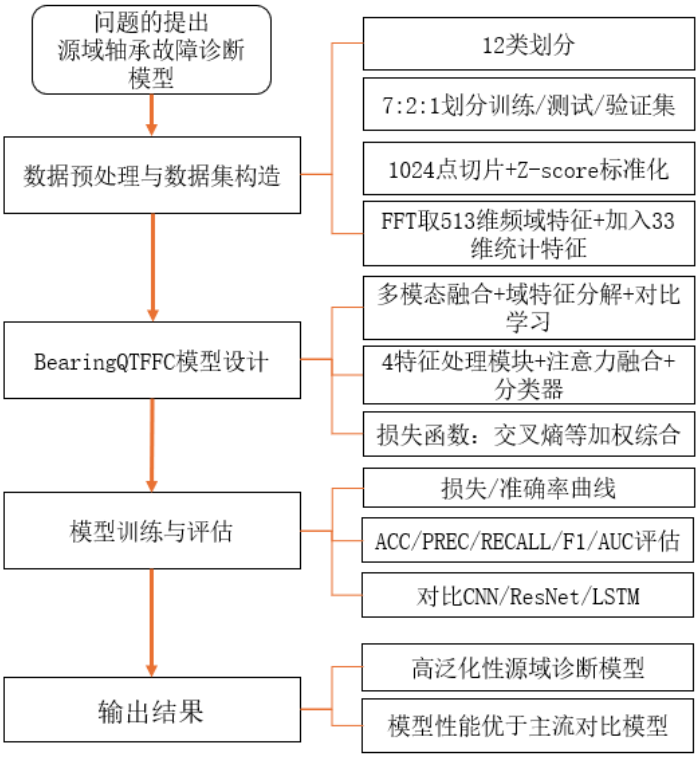


图 4-1 问题二模型建立流程图

4.2 基于深度学习的源域故障诊断模型建立

4.2.1 数据预处理与数据集构造

本文按照轴承工作状态以及故障尺寸将原有数据分成 12 类，并且按照 7:2:1 的比例来划分训练集、验证集以及测试集。其中训练集用于训练网络，测试集用于在训练的过程中保存最好的结果，验证集用于检验最终的分类结果。具体划分情况如表 4-1。

表 4-1 轴承工作状态以及故障尺寸划分

工作状态	故障直径	样本数			工作状态	故障直径
		训练集	验证集	测试集		
B	7	1750	500	250	B	7
B	14	1750	500	250	B	14
B	21	1750	500	250	B	21
B	28	700	200	100	B	28
IR	7	1750	500	250	IR	7
IR	14	1691	483	242	IR	14
IR	21	1750	500	250	IR	21
IR	28	700	200	100	IR	28
OR	7	5191	1483	741	OR	7
OR	14	1971	563	281	OR	14
OR	21	3791	1084	542	OR	21
N	0	635	182	91	N	0
总计		23429	6695	3347		

由于原始振动信号较长，直接输入模型不利于计算与学习。本文将其按 1024 点为窗口进行切片，每个窗口视为一个独立样本。为了避免幅值差异导致训练不稳定，对所有样本采用 Z-score 标准化：

$$x' = \frac{x-\mu}{\sigma} \tag{4-1}$$

其中，μ和σ分别为该样本的均值与标准差。

基于快速傅里叶变换对时域信号进行频谱分析，并取前 513 维幅值谱作为频域输入。此处理方式能够突出故障特征频率包括 BPFO、BPFI、BSF，增强模型对周期性冲击成分的识别能力。

问题一中结合轴承故障机理，本文提取了 21 维统计学特征，将其一同加入到输入集中。最终形成了时域输入、频域输入与特征输入集成的数据集，为深度学习模型提供了更为丰富、全面的信息，有助于提升模型的故障诊断性能。

4.2.2 轴承诊断模型设计动机

为了实现对源域数据的高精度故障诊断，本文设计了一种 BearingQTFFC 模型。该模型在传统卷积神经网络的基础上，创新性地融合了多模态特征融合、域不变与域特定表示分解、注意力机制以及对比学习约束等关键技术，显著提升了模型在复杂噪声环境和多工况下的鲁棒性与泛化能力。其设计思路与主要创新点如下：

1、模型的设计基于对轴承振动信号多模态特性的深入分析。时域信号能够有效反映故障引发的瞬态冲击特征，频域信号则擅长揭示 BPFO、BPFI 和 BSF 等故障特征频率，而统计学特征能够补充能量分布和高阶统计量等重要信息。由于单一模态难以完整刻画复杂故障模式，因此需要构建多模态融合机制来充分利用各类特征的互补优势。

2、实验室数据与列车实际运行数据在噪声强度和信号分布方面存在显著差异，模型特别注重区分域共享特征与域特定特征的能力。这一设计为后续的迁移学习奠定了坚实基础，确保模型能够适应不同工况下的诊断需求。

3、在诊断结果的可分性方面，传统交叉熵损失函数仅能保证基本的类别分界，而难以实现类内特征的紧凑表示。为此，模型引入对比学习损失机制，通过增强类内特征的聚合性和类间特征的可分性，进一步提升故障诊断的准确性和可靠性。

4.2.3 BearingQTFFC 轴承诊断模型结构

本文的诊断模型由四个特征处理模块，一个融合模块以及一个分类器组成，特征处理模块分别是时域特征提取器、频域特征提取器、特征处理模块、域不变与域特定特征提取器。

（1）时域特征提取器

时域特征提取器采用三层一维卷积神经网络，其中每层包含卷积、批量归一化、ReLU 激活以及最大池化。卷积核在时域信号上滑动，能够有效捕捉由轴承缺陷引起的冲击脉冲序列及其局部模式。在保证运算效率的同时，逐层扩大感受野，提取从短时冲击到长时趋势的多尺度特征。

（2）频域特征提取器

输入为 FFT 后截取的前 513 维频谱。采用两层卷积以及池化网络，并在末端加入全局平均池化（GAP）。故障特征在频域上往往表现为特征频率处能量突增，通过卷积操作可提取能量分布模式，GAP 则起到降维与抑制过拟合的作用。

（3）故障特征处理模块

输入为 21 维事先提取的统计学特征，经过两层全连接，并加批量归一化和 ReLU 激活函数。

（4）域不变与域特定特征提取器

在时域与频域特征拼接后，通过两条并行子网络进行分解。域不变分支：压缩并抽取不同工况共享特征，减少分布差异影响；域特定分支：保留源域的差异性特征。在后续融合模块以及分类模块中只采用域不变分支提取的共性特征。

（5）特征融合模块与注意力机制

将域不变特征、时域频域特征、事先提取特征三类特征输入注意力层，分别计算权重得到加权特征表示。拼接加权特征后，经全连接+BatchNorm+ReLU 降维至 128 维。注意力机制实现模态自适应选择，使融合后的特征表示具有更强的鲁棒性。

（6）分类器

两层全连接，中间加入 Dropout（p=0.5），最后 Softmax 输出。简洁的全连接层作为判别器，降低过拟合风险。

4.2.4 损失函数与优化目标

在诊断网络的训练过程中，本文采用了多损失联合优化策略，通过引入三类损失函数共同作用，保证模型既具备准确的分类能力，又具备良好的特征判别性与迁移能力。

1、分类损失函数

$$\ell_{CE} = -\frac{1}{N}\sum_{i=1}^{N}\sum_{c=1}^{C} y_{ic}\log(\hat{y}_{ic}) \tag{4-2}$$

其中，N 为批次样本数，C 为类别数，y_{ic} 表示样本 i 的真实标签，$\hat{y}_{ic}$ 表示模型预测的概率分布。交叉熵损失这是最常用的分类损失，确保网络输出的类别概率与真实类别一致。它是整体优化目标的核心，保证模型具有基本的分类判别能力。

2、条件对比损失函数

对比损失的思想是当同一类别的样本应该在特征空间中靠得更近，不同类别的样本应被推开。该损失函数形式为：

$$\ell_{CCL} = -\frac{1}{N}\sum_{i=1}^{N}\frac{1}{|P(i)|}\sum_{j\in P(i)}\log\frac{\exp(sim(f_i,f_j)/\tau)}{\sum_{k=1}^{N}1_{[k\neq i]}\exp(sim(f_i,f_k)/\tau)} \tag{4-3}$$

其中，f_i 为样本 i 的特征向量，$sim(\cdot)$ 表示余弦相似度，τ 为温度参数，$P_{(i)}$ 表示与样本 i 属于同一类别的样本集合。其中强化类内聚合性，让同一类别的特征在嵌入空间里更加紧密。如此一方面增强类间可分性使不同类别的特征相互远离，另一方面提升鲁棒性，使得模型在面对工况变化和噪声时，仍能保持特征判别能力。其中，温度参数 τ 不是固定值，而是通过余弦退火调度器动态调整，逐步细化特征聚合的强度。

3、余弦相似度损失函数

该损失计算域不变特征与域特定特征之间的相似性：

$$\ell_{\cos} = \frac{1}{N}\sum_{i=1}^{N} sim(f_{inv}^{(i)}, f_{spec}^{(i)}) \tag{4-4}$$

其中，$sim(\cdot)$ 为余弦相似度，f_{inv} 表示域不变特征，f_{spec} 表示域特定特征。该损失的引入是为了约束域不变特征和域特定特征的表示空间。理想情况下，域不变特征应该捕捉跨工况共享的模式，域特定特征应该补充源域独有的信息。通过余弦相似度约束，两者保持一定的相关性，从而避免特征分解过程中完全“分离”或过度“重叠”。

4、综合损失函数

最终的训练目标为三者加权组合：

$$\ell = \ell_{CE} + \lambda_1\ell_{CCL} + \lambda_2\ell_{\cos} \tag{4-5}$$

在代码实现中，超参数设置为：

（1）$\lambda_1 = 0.1$（对比损失权重）

（2）$\lambda_2 = 0.1$（余弦相似度损失权重）

这样设计以分类损失为主导，利用对比损失增强特征判别力，利用余弦相似度约束保

证域特征分解的合理性。

4.2.5　模型评估

（1）准确率：

$$Accuracy = \frac{TP + TN}{TP + FP + TN + FN} \tag{4-6}$$

其中 TP（TruePositive）、TN（TrueNegative）、FP（FalsePositive）、FN（False Negative）分别为四种分类结果的数量。

（2）精确率：

$$\Pr ecision = \frac{TP}{TP + FP} \tag{4-7}$$

（3）召回率：

$$\mathrm{Re}\,call = \frac{TP}{TP + FP} \tag{4-8}$$

（4）F1 分数：

$$F1 = 2 \times \frac{\Pr ecision \times \mathrm{Re}\,call}{\Pr ecision + \mathrm{Re}\,call} \tag{4-9}$$

4.3　问题二的求解

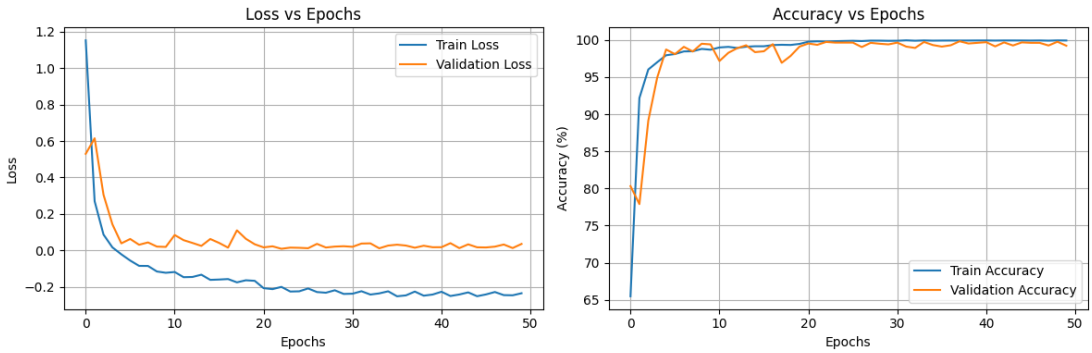


图 4-2　损失和准确率变化曲线

图 4-2 展示了模型在训练过程中的损失和准确率变化曲线。从图中可以看出，训练损失和验证损失随着训练轮次的增加逐渐下降，并在大约 20 轮后趋于平稳。这表明模型能够有效学习训练数据中的特征，并且没有出现明显的过拟合现象。训练准确率和验证准确率随着训练的进行逐步提升，最终验证准确率稳定在较高水平。这说明模型在训练集和验证集上都表现良好，具有较好的泛化能力。

表 5-2　模型测试集上的分类报告

类别	Precision	Recall	F1
0.00	1.00	0.95	0.97
1.00	1.00	1.00	1.00
2.00	1.00	1.00	1.00
3.00	1.00	1.00	1.00
4.00	1.00	1.00	1.00
5.00	1.00	1.00	1.00
6.00	1.00	1.00	1.00
7.00	1.00	1.00	1.00
8.00	1.00	1.00	1.00
9.00	0.96	1.00	0.98
10.00	1.00	1.00	1.00

11.00	1.00	1.00	1.00

最终模型的训练损失为-0.2262，训练准确率达到 99.92%，验证损失为 0.0144，验证准确率达到 99.82%，测试损失 0.0209，测试准确率达到 99.61%。这一结果充分证明了模型在未见数据上的强大泛化能力。表 5-2 展示了模型在测试集上的分类报告。从报告中可以看出，模型在各个类别上的精确率、召回率和 F1 值均达到较高水平，均超过 0.98。这表明模型对各类故障的识别能力都非常出色。

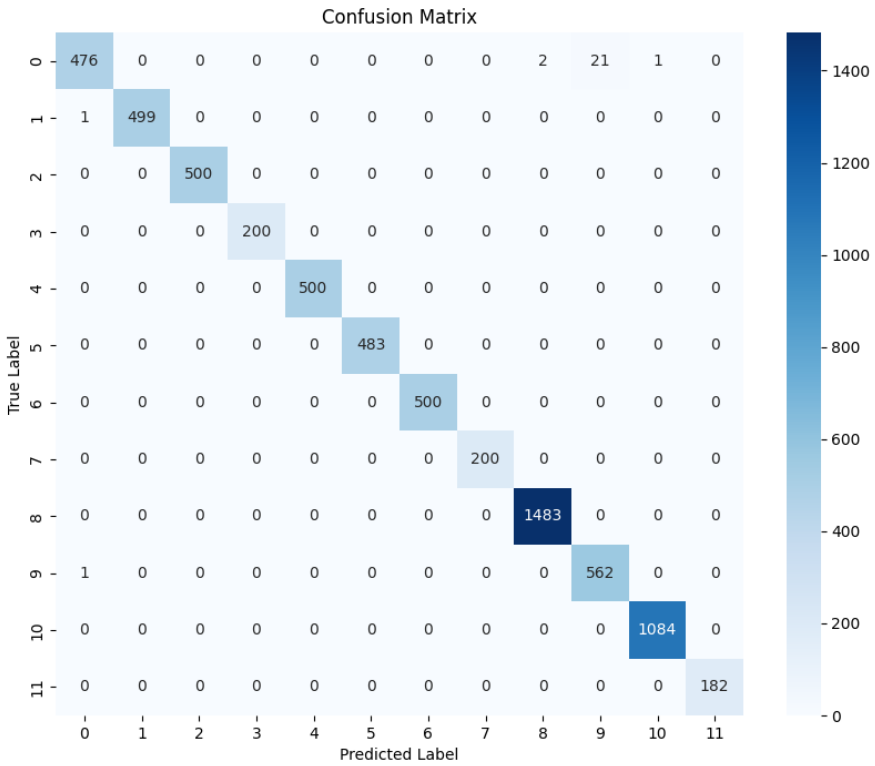


图 4-3 测试集上混淆矩阵

图 4-3 展示了模型在测试集上的混淆矩阵。从矩阵中可以看出，大部分样本都被正确分类，仅有极少数样本被误分类。Class8 和 Class10 之间有少量误分类，这可能是由于这两类故障的振动信号特征较为相似所致。

表 4-3 不同模型评估指标对比

模型	ACC	PREC	RECALL	F1
BearingQTFFC	0.9961	0.9962	0.9961	0.9961
CNN	0.922	0.9239	0.922	0.9215
ResNet	0.9818	0.9823	0.9818	0.9819
LSTM	0.9492	0.9496	0.9492	0.9488

为了全面评估本文所提出的模型的性能，本文进行了一系列对比试验，将本文的模型与当前几种主流的模型进行了比较，包括传统的卷积神经网络（CNN）、残差网络（ResNet）以及长短期记忆网络（LSTM）。在试验中，本文使用了相同的数据集和评估指标，以确保对比的公平性和准确性。试验结果表明，本文的模型在所有评估指标上均取得了显著优于其他对比模型的成绩。具体而言，本文的模型在准确率上达到了 0.9961，相比 CNN、ResNet 和 LSTM 分别提高了 0.0741、0.0143 和 0.0469。在精确率方面，本文的模型也以

0.9962 的成绩领先，比 CNN、ResNet 和 LSTM 分别高 0.0723、0.0139 和 0.0466。

在召回率上，本文的模型同样表现出色，达到了 0.9961，相较于其他模型有显著提升。F1 值作为精确率和召回率的调和平均数，本文的模型也以 0.9961 的高分展现了其在综合性能上的优势。

这些对比试验结果充分证明，本文的模型在处理相关任务时，相较于传统的 CNN、ResNet 和 LSTM 模型，具有更高的准确性和稳定性。这主要得益于本文模型在结构设计上的创新和优化，使其能够更好地捕捉数据中的关键特征，从而提高整体性能。未来，本文将继续探索模型在不同应用场景下的表现，并进一步优化其性能。

4.4 问题二小结

对于问题二，本文构建了一个基于深度学习的源域故障诊断模型 BearingQTFFC。该模型创新性地融合了多模态特征（包括时域信号、频域频谱和手工统计特征），并引入了域不变/域特定特征分解机制、注意力加权融合模块以及对比学习约束。通过多损失联合优化（交叉熵损失、条件对比损失和余弦相似度损失），模型在源域数据上实现了高精度的故障分类。实验结果表明，该模型在测试集上的准确率达到 99.61%，精确率、召回率和 F1 值均超过 0.996，显著优于传统 CNN、ResNet 和 LSTM 模型。这不仅验证了模型在复杂噪声和多工况下的鲁棒性，也为后续迁移学习任务提供了高质量的特征表示基础。

五、 问题三模型建立与求解

5.1 模型总体思路与架构设计

在实际工程应用中，高速列车轴承故障诊断面临两个关键挑战：一方面，实际运营列车轴承的故障样本极其稀缺且缺乏准确标签；另一方面，试验台架数据虽然丰富但与真实运行环境存在显著分布差异。这种域偏移问题导致直接在试验台数据上训练的模型在实际运营环境中性能严重下降。

为解决这一难题，本研究提出一种基于对抗域适应的跨工况故障诊断方法，实现在无监督条件下的知识迁移。该方法的核心思想是通过引入域判别器，构建特征提取器与域判别器之间的对抗博弈机制，迫使特征提取器学习具有域不变性的深度特征表示。通过这种方式，模型能够有效对齐源域数据和目标域数据的特征分布，增强模型在未知目标数据上的泛化能力，最终显著提升对无标签运营轴承数据的诊断准确率。问题三的模型建立流程图如图 5-1 所示。

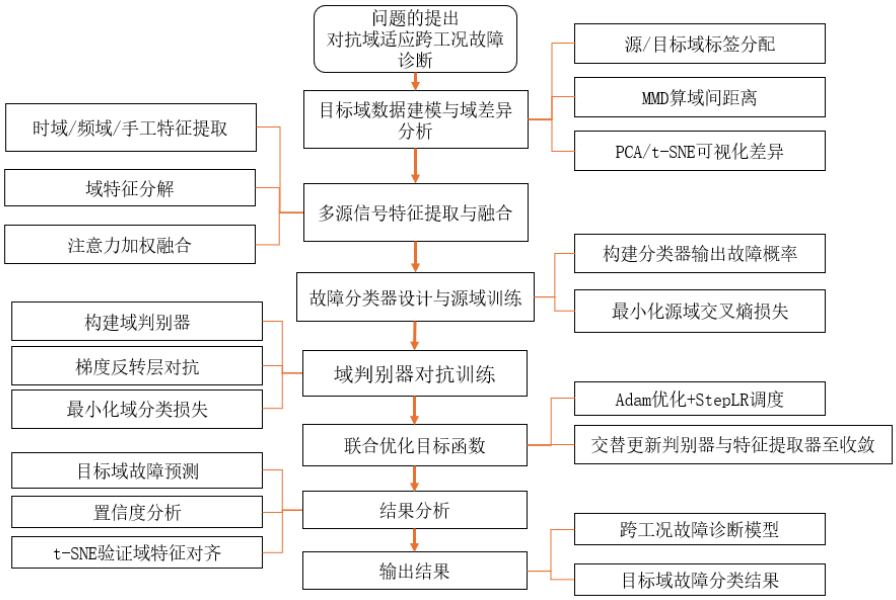


图 5-1 问题三的模型建立流程图

5.2 问题三模型的构建

5.2.1 目标域数据建模与域差异分析

在迁移学习框架中，目标域数据建模与域差异分析是确保模型有效泛化的关键步骤。本系统采用先进的域对抗学习方法，专门设计用于处理源与目标域之间的分布差异问题。目标域数据中每个样本包含原始时间序列振动信号、对应的频域表示以及统计特征。对于域标签分配，本文将源域数据标记为 0，目标域数据标记为 1。

设源域与目标域的特征分布分别为 $Ps(x)$ 与 $Pt(x)$。为了衡量两者差异，采用最大均值差异（Maximum Mean Discrepancy,MMD）指标，其定义如下：

$$MMD^2(D_s,D_t)=\left\|\frac{1}{n_s}\sum_{i=1}^{n_s}\phi(x_i^s)-\frac{1}{n_t}\sum_{j=1}^{n_t}\phi(x_j^t)\right\|_H^2 \tag{5-1}$$

其中，$\phi(\cdot)$ 表示在再生核希尔伯特空间（RKHS）中的特征映射，n_s ,n_t 分别为源域和目标域的样本数。实验结果表明，本研究中源域与目标域的域间距离为 5.39，表明两者存在显著分布差异，但尚未达到完全分离的程度。

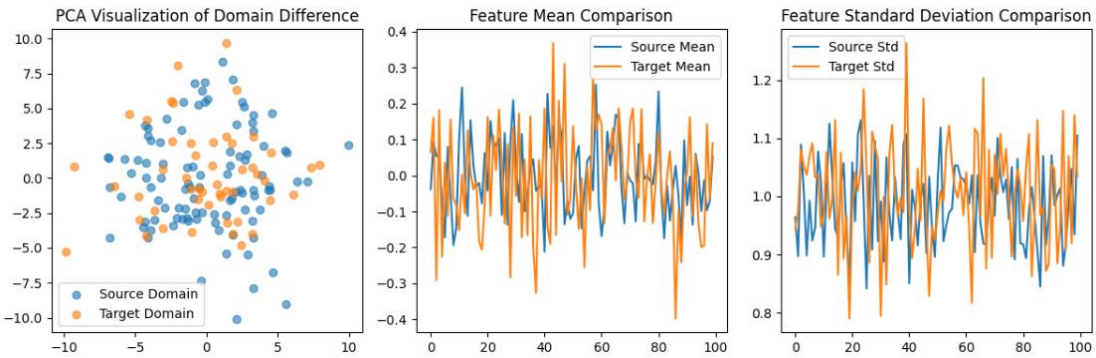


图 5-2 两类数据可视化

从图 5-2 看出 PCA 可视化将源域与目标域样本投影到二维主成分空间，可以看到两类数据在空间上存在部分重叠，但仍呈现一定程度的分布差异。这说明目标域数据在特征空间中并未完全偏离源域，但存在潜在的域偏移。对比源域与目标域的前 100 个特征均值曲线发现，两者走势基本一致，但在部分特征维度上仍存在明显偏差。这种偏差可能导致模型在目标域数据上分类时精度下降。源域与目标域的特征标准差整体接近，说明两域数据的整体波动性和分布宽度类似。但局部差异依然存在，提示在部分特征维度上目标域样本分布更为分散或收敛。

5.2.2 多源信号特征提取与表示构建

对每条时域振动信号 $x_i^{(s)} \in R^L$ (源域)或 $x_j^{(t)} \in R^L$ (目标域)，分别提取时域、频域与故障特征。

设 x_{time} 为经过 $1D-CNN$ 提取的时间域基特征，x_{freq} 为 FFT 后经 $1D-CNN$ 提取的频域特征，$x_{hand} \in R^{d_h}$ 为标准化后的 g 故障特征向量。

时间域基特征提取器可表示为 $h_{time} = f_{time}\left(x^{(\cdot)}\right)$，其中 f_{time} 为由三层卷积-池化单元组成的映射。频域特征提取器为

$$h_{freq} = f_{freq}(FFT(x^{(\cdot)})) \tag{5-2}$$

故障特征处理模块为

$$h_{hand} = f_{hand}(x_{hand}) \tag{5-3}$$

其中 h_{hand} 为两层全连接 $-BN-\mathrm{Re}LU$ 结构。

然后将时间基特征送入域不变特征提取器 g_{inv} 与域特定特征提取器 g_{spec}：

$$z_{inv} = g_{inv}(h_{time}), z_{spec} = g_{spec}\left(h_{time}\right) \tag{5-4}$$

三个分支特征通过注意力加权的融合模块 f_{fusion} 合并：

$$z = f_{fusion}(z_{inv}, h_{freq}, h_{hand}) \in R^{128} \tag{5-5}$$

其中注意力权重 α_{time}，α_{freq}，α_{hand} 由线性层 $+sigmoid$ 得出，实现各模态信息自适应加权。该步骤在保留故障判别性信息的同时剔除与工况相关的噪声成分。

5.2.3 故障分类器设计与源域监督训练

在共享特征空间 Z 中，构建故障分类器 $C(\cdot)$，输出各故障类别的概率：

$$P = C(z) = Soft\max(W_2\sigma(W_1 + b_1) + b_2) \tag{5-6}$$

其中 $\sigma(\cdot)$ 为 $\mathrm{Re}LU$，W_1, W_2 和 b_1, b_2 为可学习参数。对于源域有标签样本 $\left(x_i^{(s)}, y_i^{(s)}\right)$，最小化交叉熵损失：

$$\ell_{cls} = -\frac{1}{N_s}\sum_{i=1}^{N_s}\sum_{k=1}^{K} 1_{[y_i^{(s)}=k]} \log p_{ik} \tag{5-7}$$

其中 K 为故障类别数，$1_{[\cdot]}$ 为指示函数。

该步骤确保模型在源域上具备强判别能力，为后续知识迁移提供可靠的初始参数基础。

5.2.4 域判别器与对抗训练机制

为减少频域与目标域的特征分布差异，引入域判别器 $D(\cdot)$ 区分输入特征来自源域还是目标域。

$$\hat{q} = D(z) = Soft\max(V_2\sigma(V_1 z + c_1) + c_2) \tag{5-8}$$

域标签 $d \in \{0,1\}$,其中 0 表示源域，1表示目标域。

域分类损失为

$$\ell_{dom} = -\frac{1}{N_{(s)} + N_t}\sum_{i}\sum_{j=0}^{1} 1_{[d_i=j]} \log \hat{q}_{ij} \tag{5-9}$$

通过梯度反转层实现对抗，在反向传播时将 ℓ_{dom} 的梯度乘以 $-\alpha$，从而使特征提取器最大化域混淆程度，即最小化 ℓ_{cls} 同时最大化判别器损失。

通过对抗学习，迫使源域和目标域的特征分布趋于一致，增强模型对目标域样本的适应能力。

5.2.5 联合优化目标函数的构建

综合考虑分类准确性与域不变性，整个网络的优化目标为

$$\min_{\Theta_C,\Theta_F}\max_{\Theta_D} \ell_{total} = \ell_{cls} - \alpha \ell_{dom} \tag{5-10}$$

其中 Θ_F 表示各特征提取模块参数，Θ_C 表示分类器参数，Θ_D 表示域判别器参数，α 为控制域适应强度的超参数。本研究在实现中采用 Adam 优化器域 StepLR 学习率调度，并按“先更新判别器后更新特征提取器”的交替策略直至收敛，实现从台架数据向真实列车数据的有效迁移。

5.3 问题三结果与分析

表 6-1 预测目标域数据

文件	投票预测	最终预测	置信度	故障类型
A.mat	2	2	0.248	B
B.mat	2	2	0.284	B
C.mat	2	2	0.256	B
D.mat	2	2	0.252	B
E.mat	10	10	0.268	OR
F.mat	2	2	0.28	B
G.mat	2	2	0.236	B
H.mat	10	10	0.268	OR
I.mat	0	0	0.264	B
J.mat	2	2	0.328	B
K.mat	2	2	0.288	B
L.mat	2	2	0.304	B
M.mat	2	2	0.352	B
N.mat	0	0	0.28	B
O.mat	0	0	0.272	B

P.mat	2	2	0.296	B

通过对目标域数据的批量预测，本文得到了每个数据文件的最终故障类型分类及对应的置信度。故障类型分布：模型将目标域数据主要诊断为两种故障类型：B（球故障）和 OR（外圈故障）。其中，14 个文件被预测为 B 故障，2 个文件被预测为 OR 故障。这表明目标域设备的潜在故障模式集中于轴承的球体和外圈。

模型对本次预测的整体置信度一般，其值分布在 0.2360 至 0.3520 之间，平均置信度约为 0.280。置信度没有出现极高（接近 1.0）或极低（接近 0）的情况，说明模型对预测结果有一定把握，但同时承认了源域与目标域差异所带来的不确定性。

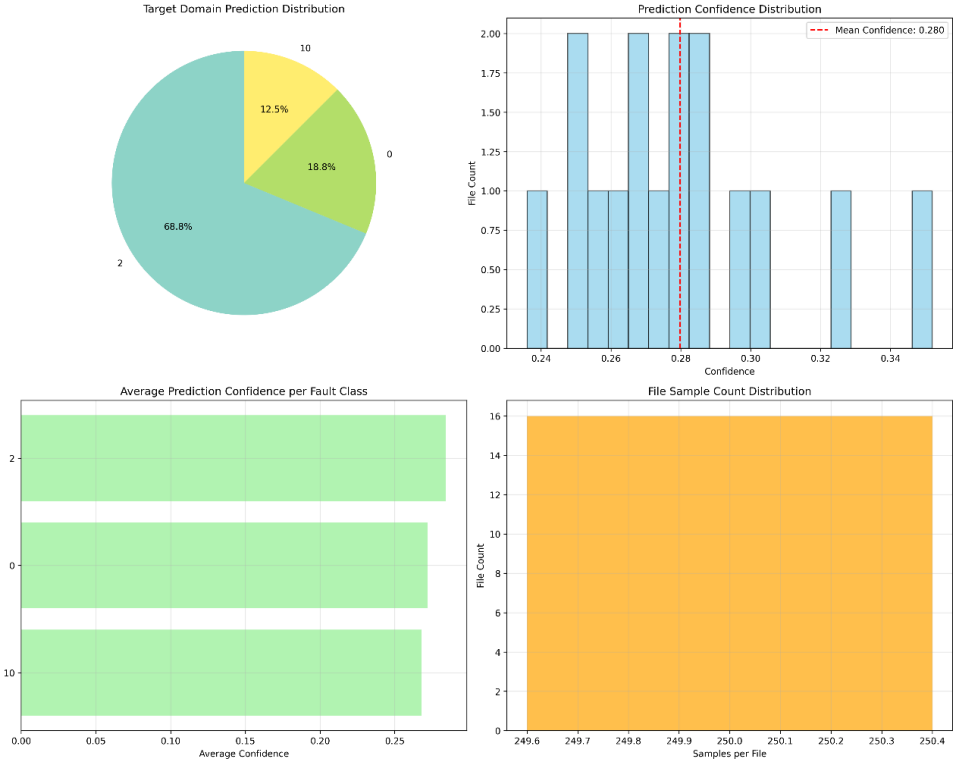


图 5-3 目标域预测结果总体分布情况

目标域预测结果总体分布情况如图 5-3 所示。故障类型分布如饼图所示，目标域数据的预测结果主要集中在 B（球故障）和 OR（外圈故障）两个类别，其中 B 类故障占比最高。

置信度直方图显示，模型对目标域样本的预测置信度呈近似正态分布，集中区间为 0.26 至 0.30，平均置信度为 0.280。整体置信度不高，这表明目标域与源域之间存在一定的分布差异，给模型预测带来了一定的不确定性，但模型仍然给出了相对明确的分类决策。大部分数据文件包含的样本数量分布较为集中，表明数据预处理阶段成功地将不同长度的时序数据规范到了相近的规模，为模型提供了统一的输入格式。

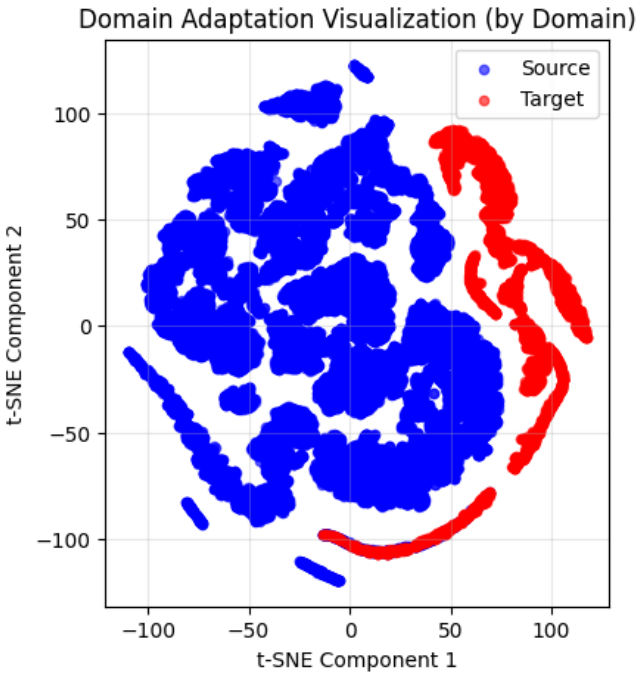


图 5-4 模型特征降维可视化

为验证迁移学习的有效性，本文通过 t-SNE 技术将模型提取的高维特征降维至二维空间进行可视化，图 5-4 按领域着色（蓝色为源域，红色为目标域）。可以看到，两个域的数据点形成了混合交织而非完全分离的分布，这表明域对抗训练成功地将源域和目标域的数据映射到了一个共同的特征空间，减少了域间差异，为迁移奠定了基础。

5.4 问题三小结

对于问题三，本文提出了一种基于对抗域适应的跨工况故障诊断方法，旨在解决源域与目标域之间的分布差异问题。通过引入域判别器与梯度反转层，模型在训练过程中实现了特征分布的对齐，从而将源域知识有效迁移至无标签的目标域数据。实验结果显示，目标域数据被主要预测为 B 和 OR 两类，平均置信度约为 0.280。t-SNE 可视化表明，源域与目标域特征在嵌入空间中实现了有效混合，证实了域对抗训练的有效性。该方法在缺乏真实故障标签的条件下，显著提升了模型在实际运营环境中的故障诊断能力，为工程应用提供了可靠的技术支持。

六、 问题四模型建立与求解

6.1 问题分析及建模思路

问题四要求在以上三个问题的基础上，从事前可解释性、迁移过程可解释性、事后可解释性三个方面对轴承故障诊断模型建立与迁移的过程进行解释。以降低工作人员对深度学习自身存在的“黑箱”问题和“迁移学习”的透明性问题的顾虑。

针对问题四的要求，首先从事前可解释性，对问题一选出的明确特征和模型预测过程中特征重要性的变化情况进行描述，以解决工作人员对模型“黑箱”问题的顾虑。接着在迁移学习的过程中，记录数据分布对齐的过程，将其直观地展示出来。最后使用迁移学习后的模型对特征的作用进行可视化处理，从可视化结果看出迁移过后那些特征的影响更大，也进一步解释了迁移后的模型的运作机理。问题四的模型建立流程图如图 6-1 所示。

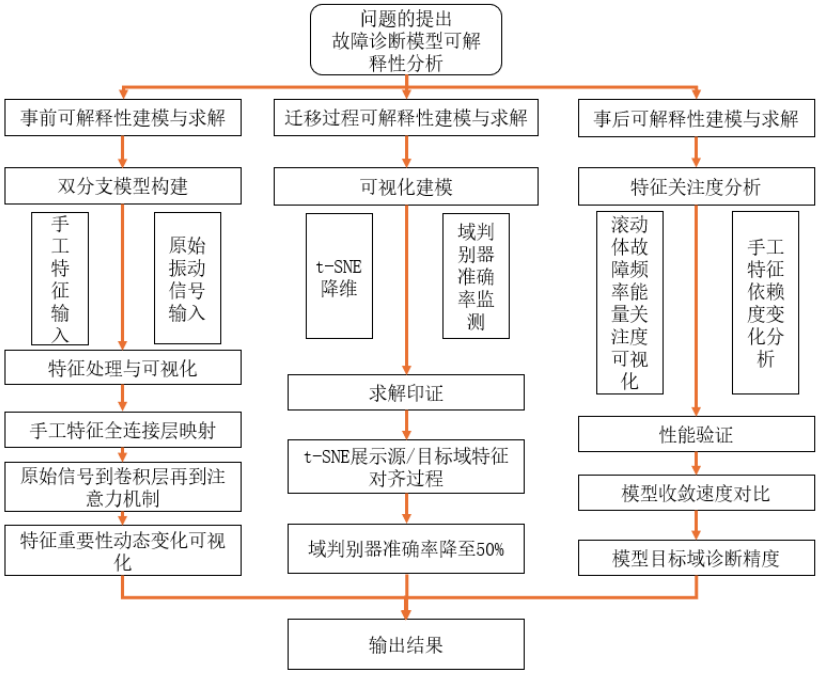


图 6-1 问题四的模型建立流程图

6.2 问题四模型建立

6.2.1 事前、事后可解释性模型建立

为了在迁移学习诊断的过程使模型有很好的性能与可解释性，本文建立了双分支结构的深度学习模型。在输入端分别将问题一提取出来的时域、频域、时频域三个角度的故障特征和振动信号的原始特征作为模型的输入特征。对于故障特征来说，其具有明确的物理意义，这使得模型具有语义透明的特征。故障特征向量如下所示：

$$X_{hand} = \left[x_1, x_2, \cdots, x_{13}\right] \tag{6-1}$$

其中 x_i 表示问题一中提取出来的故障特征。

将故障特征输入到特有的特征处理模块，会通过全连接层将其映射到潜在空间：

$$h_{hand} = \sigma\left(W_{hand} x_{hand} + b_{hand}\right) \tag{6-2}$$

其中 σ 为激活函数，W_{hand} 的向量值对应着每个故障特征的权重，可以直接用来解释特征的重要性。

对于原始信号，将其直接输入到卷积层中，自动提取原始信号的特征，为了弥补故障特征由于其局限性损失的有用信息。为了了解卷积层最终更倾向于那部分特征，本文将经

过卷积层处理后的深度特征 h_{auto} 带入注意力机制中：

$$e_t = v' \tanh(W_{at} h_t) \tag{6-3}$$

$$\alpha_t = \frac{\exp(e_t)}{\sum_k \exp(e_k)} \tag{6-4}$$

α_t 为模型在时间段中关注的强度，可以直观看到模型更加关注原始信号的那一部分。

6.2.2 迁移过程可解释性模型建立

问题二中训练结束的故障诊断模型，要应用在目标域中需要进行迁移学习，也就是说需要将诊断模型所拥有的知识从源域迁移到目标域中。想要知道迁移学习是否有效就必须将这个过程进行可视化，随着训练的进行查看特征对齐的过程。为了展示在迁移过程中源域与目标域特征的对齐效果，本文使用 t-SNE 对高维特征进行降维，将源域与目标域的样本点进行可视化，进而观察对齐的过程。公式如下：

$$\ell_{t-SNE} = \sum_{i \neq j} p_{ij} \log \frac{p_{ij}}{q_{ij}} \tag{6-5}$$

其中 p_{ij} 是高维空间中样本 x_i 与 x_j 的相似度，q_{ij} 是低维空间中映射点 y_i 与 y_j 的相似度。用这个公式进行梯度下降就可以得到高维特征的低维表示。另外在迁移学习的过程中，也可以借助域分类器准确率来判别是否正在进行有效的迁移学习，其工作原理是区分样本是来自源域还是目标域，当接近于 50%说明域分类器分不清样本来自于源域还是目标域，这时特征提取器已经学到了域不变性，这部分模型在问题三已建立，所以不过多赘述。

6.3 模型求解

对于事前可解释性来说，问题一中提取的特征将会作为诊断模型的特征输入，而这些特征具有明确的意义，所以可以增强诊断模型在输入端的可解释性。为了说明智能诊断模型内部的可解释性，本文将模型中的特征进行可视化，结果如图 6-1 所示。

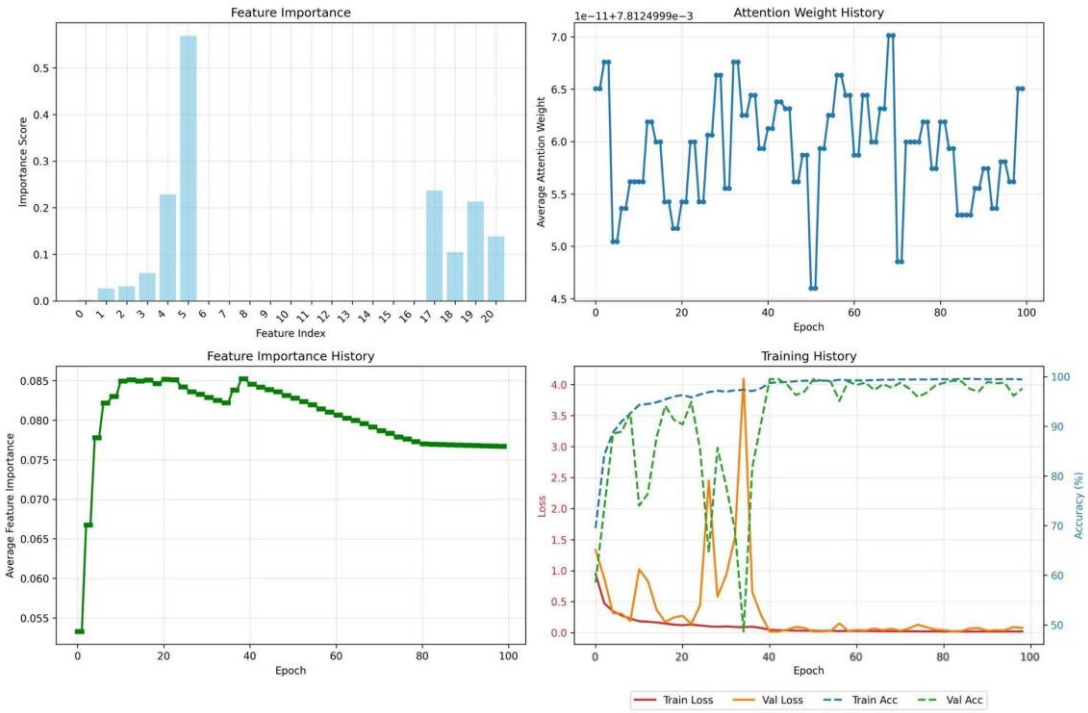


图 6-1 事前模型内部特征图

从左上方的柱状图来看诊断模型用到了 9 个手工提取的特征，其余特征的贡献接近于 0。这通常意味着这些特征对当前任务贡献较弱，或其信息已被其他高重要性特征替代。如果看右上方的折线图，本文可以看出原始数据振动信号特征在诊断模型中的作用较大，而且原始信号蕴含着频谱重心、频段能量比、均方根值等特征信息，所以这说明诊断模型是围绕着原始信号数据进行故障的判断，同时将故障特征作为补充信息，以弥补原始信号中未能显式表达的特征内容，从而提高诊断的完整性与准确性。从左下的图像中可以看出，随着训练的进行故障特征的重要性迅速提升，随后缓慢回落趋于稳定。这从另一个方面说明，在训练初期故障特征对模型贡献有限，随着训练往后推移模型逐渐识别这些特征中的信息，并在后期将其作为补充，从而提升模型的判别能力与整体性能。

对于迁移过程可解释性来说，通过模型建立提到的方法将模型迁移的过程进行可视化展示，如图 6-2 所示。

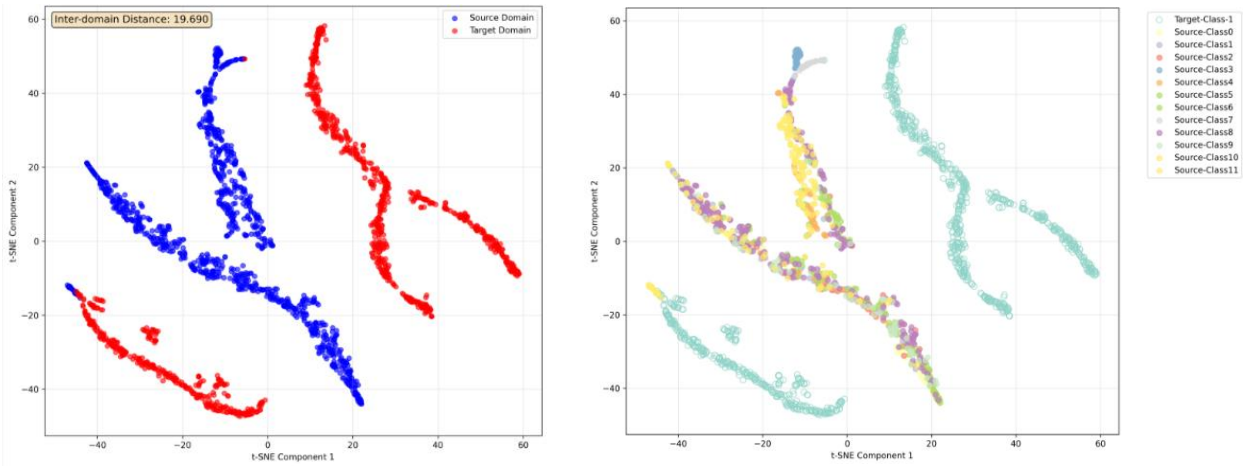


图 6-2 迁移过程可视化图

左上方为源域特征与目标域特征在迁移学习前在 t-SNE 空间中的分布情况，可以看出域间距离存在一定的距离。右上方为训练进行 50 个 epoch 后的分布情况，可以看出目标域样本逐渐靠近对应的源域簇。但是不同类别之间分界仍然存在，但模型已经开始把目标域数据映射到源域的相同类别区域，说明迁移正在进行。另外本文还将域判别器分类准确率随着训练进行的时间序列图画了出来，如图 6-3 所示。

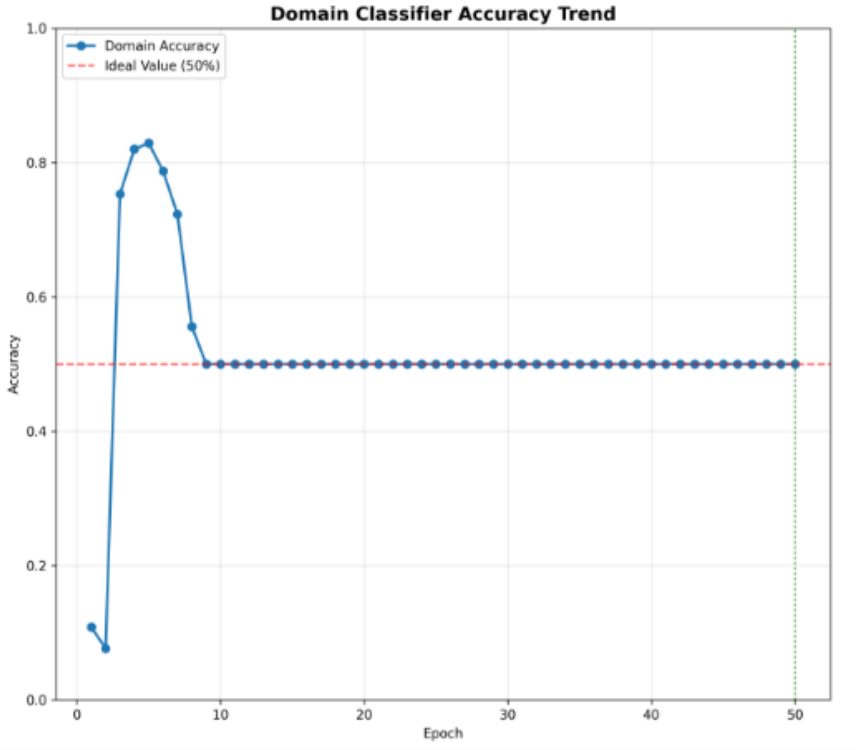


图 6-3 域判别器分类准确率图

从图中可以看出，在训练的前期准确率大于 0.8，说明两域差异明显，随着训练的进行迅速下降到 0.5 左右稳定，表明特征提取器已学到域不变特征域判别器失效。说明对抗训练成功，也代表着诊断模型的迁移学习是有效的。

对于事后可解释性，本文使用同事前可解释性相同的方法对其进行说明。结果如图 6-4 所示。

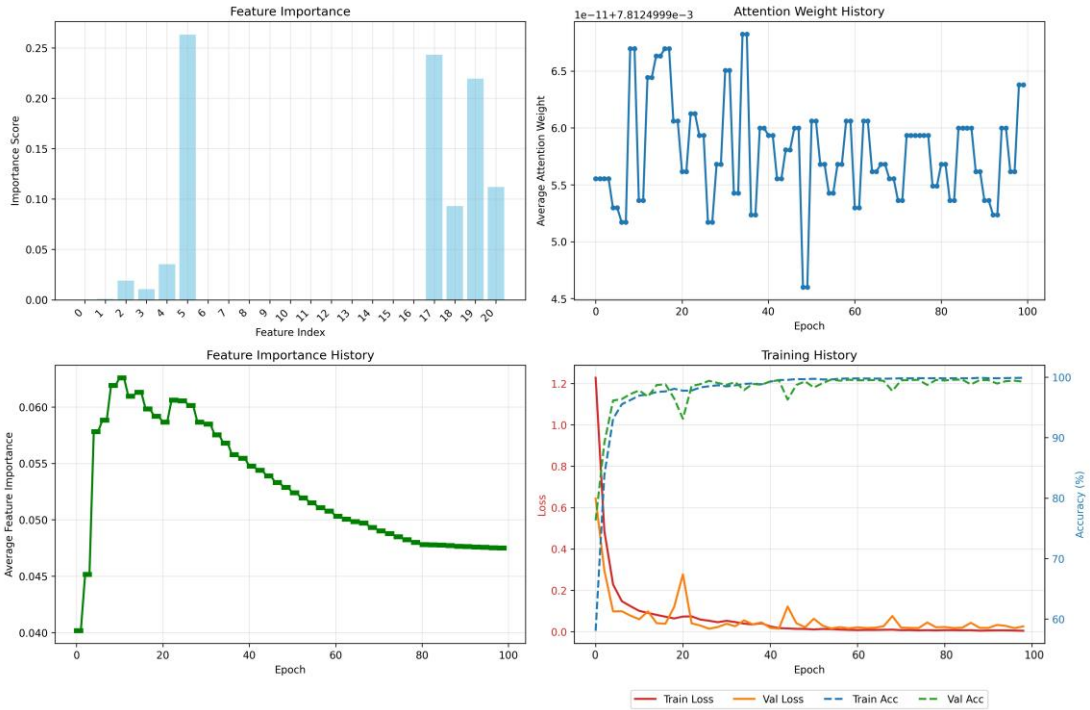


图 6-4 事后模型内部特征图

相对于事前模型内部特征图来说，迁移完成后的模型对外圈故障特征频率能量及滚动体故障特征频率能量的关注度显著提升。但从整体来看，迁移后的模型对手工提取特征的关注度有所下降，但是能够更快地收敛，表明其已在源域中充分学习了关键模式并有效迁移至目标域，从而在减少对故障特征依赖的同时提升了对故障本征特征的自动提取能力与诊断效率。这也可以直观地向工作人员进行展示，增加其使用迁移学习模型的信心。

6.4 问题四小结

对于问题四，本文首先在事前可解释性方面，通过对输入端故障特征与原始信号特征的可视化分析，展现出模型在故障识别中对不同特征的依赖程度及动态变化过程，并发现故障特征在模型诊断的过程中起到了信息补充的作用以及对后期模型的稳定做出了贡献。其次在迁移过程可解释性方面，借助 t-SNE 降维并进行特征分布对齐可视化以及域分类器准确率曲线，直观展示了源域与目标域特征逐渐对齐的过程，说明了迁移学习的有效性。最后在事后可解释性方面，可视化图像显示迁移后的模型在外圈故障特征频率能量和滚动体故障特征频率能量上的关注度明显提升，虽然减少了对故障特征依赖，但实现了更快的收敛速度和更好的模型性能，从而进一步提高了模型在目标域的诊断精度与泛化性能。整体研究结果不仅验证了迁移学习在智能故障诊断中的有效性，也为工程人员提供了直观的可视化证据，增强了迁移学习诊断模型在实际场景中的可解释性与可用性。

七、 模型评价

本文围绕高速列车轴承故障诊断问题，构建了一套从特征提取、源域诊断到跨域迁移的完整建模体系，并结合可解释性分析对模型机理进行了深入阐释。下面对所建模型的优缺点进行系统评价：

7.1 模型的优点

1.特征提取全面，物理意义明确

问题一从时域、频域和时频域三个角度系统提取了 21 维故障特征，包括均方根值、峰值因子、频谱重心、故障特征频率能量、小波包能量等。这些特征具有明确的物理意义，能够从不同维度刻画轴承故障状态，为后续诊断模型提供了丰富且可解释的输入信息。

2.多模态融合与深度学习结合，诊断精度高

问题二提出的 BearingQTFFC 模型创新性地融合了原始振动信号、频谱信息和手工提取特征，通过多分支网络结构与注意力机制实现自适应特征加权。模型在源域数据上取得了 99.61%的测试准确率，显著优于传统 CNN、ResNet 和 LSTM 模型，展现出强大的故障识别能力。

3.迁移学习机制有效，适应性强

问题三引入对抗域适应方法，通过域判别器与梯度反转层实现源域与目标域特征分布对齐。t-SNE 可视化表明两域特征在嵌入空间中有效混合，模型在无标签目标域数据上仍能给出合理的故障分类结果，验证了其在实际工况下的泛化能力。

4.可解释性强，增强工程可信度

问题四从事前、事中、事后三个层面系统构建了模型可解释性分析框架。通过特征重要性可视化、迁移过程动态展示和注意力权重分析，有效揭示了模型决策依据与迁移机制，降低了“黑箱”疑虑，提升了模型在工程应用中的可信度。

7.2 模型的不足

1.对目标域数据的诊断置信度偏低

尽管模型在目标域上给出了分类结果，但平均置信度仅为 0.280 左右，表明模型对目标域样本的判别仍存在一定不确定性，反映出源域与目标域之间的分布差异尚未完全消除。

2.对极端工况或未知故障类型的适应性有限

模型训练依赖于已知故障类型和尺寸的数据，若目标域中出现训练集中未包含的新型故障或极端工况，模型可能无法有效识别，存在一定的局限性。

3.计算复杂度较高，实时性有待提升

BearingQTFFC 模型结构较为复杂，包含多分支网络、注意力机制和对抗训练模块，训练和推理时间较长，在实时性要求较高的工业场景中可能面临挑战。

4.依赖故障特征，自动化程度可进一步提升

尽管模型融合了自动提取的深度特征，但仍部分依赖问题一提取的故障特征。在未来工作中可进一步探索端到端的全自动特征学习机制，减少对先验知识的依赖。

八、 参考文献

[1]陈硕日,韩险峰.基于卷积神经网络的机床轴承故障诊断应用研究[J].机械管理开发,2025,40(09):87-89.
[2]杨向兰,王国强,石念峰,等.SAGAN 结合改进 ResNet 的轴承故障诊断方法[J/OL].噪声与振动控制,1-8[2025-09-25].
[3]邵鲁川,赵兵,康旭涛.基于 DE 优化 VMD 与 CNN-BiGRU-Attention 结合的滚动轴承故障诊断[J/OL].电子测量技术,1-13[2025-09-25].
[4]李士哲,范嘉鸣,刘帅.未知工况下基于时频特征耦合与条件对比学习的轴承跨域故障诊断[J/OL].电力科学与工程,1-13[2025-09-25].
[5]赵小强,李森.不平衡数据下的轻量化轴承故障诊断方法[J].电子测量与仪器学报,2024,38(10):244-254.
[6]金怀平,刘志泳,王彬,等.融合时频特征的多源无监督域自适应轴承故障诊断方法[J].振动与冲击,2024,43(13):12-24.
[7]曹洁,尹浩楠,王进花.MACDCGAN 的发电机轴承故障诊断方法[J].振动与冲击,2024,43(11):227-235.
[8]任德珍.基于半监督深度神经网络的滚动轴承故障诊断方法[D].吉林化工学院,2024.
[9]罗煜坤.基于深度学习的滚动轴承故障诊断方法研究[D].盐城工学院,2024.
[10]徐鹏,皋军,邵星.基于 AMCNN-BiGRU 的滚动轴承故障诊断方法研究[J].振动与冲击,2023,42(18):71-80.
[11]钱思宇,秦东晨,陈江义,等.基于卷积神经网络的领域适配模型的多工况迁移的轴承故障诊断[J].振动与冲击,2022,41(24):192-200.
[12]吕静.基于实例的迁移学习算法研究[D].山西财经大学,2013.

九、 附录

```
#问题 1
import scipy.io
import numpy as np
import pandas as pd
import os
import re
import h5py
import matplotlib.pyplot as plt
import seaborn as sns
from scipy import signal
from scipy.stats import kurtosis, skew
from sklearn.preprocessing import StandardScaler
from sklearn.decomposition import PCA
import pywt
import warnings
warnings.filterwarnings('ignore')

class DataFormatConverter:
    @staticmethod
    def load_mat_data(file_path):
        try:
            return scipy.io.loadmat(file_path)
        except:
            try:
                with h5py.File(file_path, 'r') as f:
                    data = {}
                    for key in f.keys():
                        data[key] = np.array(f[key])
                    return data
            except Exception as e:
                print(f"无法加载文件 {file_path}: {e}")
                return None

    @staticmethod
    def convert_mat_to_single_sheet_xlsx(mat_file_path, output_dir):
        try:
            mat_data = DataFormatConverter.load_mat_data(mat_file_path)
            if mat_data is None:
                print(f"无法加载文件: {mat_file_path}")
                return False

            file_name = os.path.basename(mat_file_path)
```

```
        base_name = os.path.splitext(file_name)[0]
        output_file = os.path.join(output_dir, f"{base_name}.xlsx")

        all_data = {}

        for key in mat_data.keys():
            if not key.startswith('__'):
                data = mat_data[key]

                if hasattr(data, 'shape'):
                    if len(data.shape) == 1:
                        all_data[key] = data
                    elif len(data.shape) == 2:
                        if data.shape[0] == 1 or data.shape[1] == 1:
                            all_data[key] = data.flatten()
                        else:
                            for col_idx in range(data.shape[1]):
                                col_name = f"{key}_col{col_idx+1}"
                                all_data[col_name] = data[:, col_idx]
                    else:
                        print(f"警告: {mat_file_path} 中的 {key} 是高维数据，将展平处理")
                        all_data[key] = data.flatten()
                else:
                    all_data[key] = [str(data)]

        max_length = max(len(v) if hasattr(v, '__len__') else 1 for v in all_data.values())

        for key in all_data:
            data = all_data[key]
            if hasattr(data, '__len__') and len(data) < max_length:
                padded_data = np.full(max_length, np.nan)
                padded_data[:len(data)] = data
                all_data[key] = padded_data
            elif not hasattr(data, '__len__'):
                all_data[key] = np.full(max_length, data)

        df = pd.DataFrame(all_data)
        df.to_excel(output_file, index=False, sheet_name='Data')
        print(f"已转换: {mat_file_path} -> {output_file}")
        return True

    except Exception as e:
        print(f"转换文件时出错 {mat_file_path}: {e}")
        return False
```

```
    @staticmethod
    def convert_all_mat_files(root_dir, output_base_dir):
        if not os.path.exists(output_base_dir):
            os.makedirs(output_base_dir)

        file_count = 0
        success_count = 0

        for root, dirs, files in os.walk(root_dir):
            for file in files:
                if file.endswith('.mat'):
                    file_path = os.path.join(root, file)
                    relative_path = os.path.relpath(root, root_dir)
                    output_dir = os.path.join(output_base_dir, relative_path)

                    if not os.path.exists(output_dir):
                        os.makedirs(output_dir)

                    if DataFormatConverter.convert_mat_to_single_sheet_xlsx(file_path, output_dir):
                        success_count += 1

                    file_count += 1

        print(f"\n 转换完成！总共处理了 {file_count} 个文件，成功转换了 {success_count} 个文件")

class DataResampler:
    @staticmethod
    def resample_data_poly(data, original_fs, target_fs=32000):
        if original_fs == target_fs or len(data) == 0:
            return data

        if original_fs > target_fs:
            resampled_data = signal.resample_poly(data, up=2, down=3)
        else:
            resampled_data = signal.resample_poly(data, up=8, down=3)

        return resampled_data

    @staticmethod
    def get_sampling_rate_from_path(file_path):
        if '12kHz' in file_path or '12khz' in file_path:
            return 12000
        elif '48kHz' in file_path or '48khz' in file_path:
```

```
            return 48000
        else:
            print(f"警告: 无法从路径 {file_path} 确定采样频率，使用默认值 12kHz")
            return 12000

    @staticmethod
    def process_xlsx_file(xlsx_file_path, output_dir):
        try:
            original_fs = DataResampler.get_sampling_rate_from_path(xlsx_file_path)
            target_fs = 32000

            df = pd.read_excel(xlsx_file_path)
            file_name = os.path.basename(xlsx_file_path)
            base_name = os.path.splitext(file_name)[0]
            output_file = os.path.join(output_dir, f"{base_name}_{target_fs}Hz.xlsx")

            processed_data = {}
            for column in df.columns:
                col_data = df[column].values
                if 'time' in column.lower():
                    print(f"对 {column} 进行重采样: {original_fs}Hz -> {target_fs}Hz")
                    resampled_col = DataResampler.resample_data_poly(col_data, original_fs, target_fs)
                    processed_data[column] = resampled_col
                else:
                    processed_data[column] = col_data

            max_length = max(len(v) if hasattr(v, '__len__') else 1 for v in processed_data.values())

            for key in processed_data:
                data = processed_data[key]
                if hasattr(data, '__len__') and len(data) < max_length:
                    padded_data = np.full(max_length, np.nan)
                    padded_data[:len(data)] = data
                    processed_data[key] = padded_data
                elif not hasattr(data, '__len__'):
                    processed_data[key] = np.full(max_length, data)

            processed_df = pd.DataFrame(processed_data)
            processed_df.to_excel(output_file, index=False)
            print(f"已处理: {xlsx_file_path} -> {output_file} (重采样: {original_fs}Hz -> {target_fs}Hz)")
            return True

        except Exception as e:
            print(f"处理文件时出错 {xlsx_file_path}: {e}")
```

```
            return False

    @staticmethod
    def process_all_xlsx_files(input_dir, output_base_dir):
        if not os.path.exists(output_base_dir):
            os.makedirs(output_base_dir)

        file_count = 0
        success_count = 0

        for root, dirs, files in os.walk(input_dir):
            for file in files:
                if file.endswith('.xlsx') and not file.endswith('_32000Hz.xlsx'):
                    file_path = os.path.join(root, file)
                    relative_path = os.path.relpath(root, input_dir)
                    output_dir = os.path.join(output_base_dir, relative_path)

                    if not os.path.exists(output_dir):
                        os.makedirs(output_dir)

                    if DataResampler.process_xlsx_file(file_path, output_dir):
                        success_count += 1

                    file_count += 1

        print(f"\n 处理完成！总共处理了 {file_count} 个文件，成功处理了 {success_count} 个文件")

class BearingFeatureAnalyzer:
    def __init__(self, data_path):
        self.data_path = data_path
        self.features_df = None
        self.bearing_params = {
            'SKF6205': {'n': 9, 'd': 0.3126, 'D': 1.537},
            'SKF6203': {'n': 9, 'd': 0.2656, 'D': 1.122}
        }

    def calculate_fault_frequencies(self, rpm, bearing_type='SKF6205'):
        fr = rpm / 60
        params = self.bearing_params[bearing_type]
        n, d, D = params['n'], params['d'], params['D']
        bpfo = fr * n / 2 * (1 - d / D)
        bpfi = fr * n / 2 * (1 + d / D)
        bsf = fr * D / d * (1 - (d / D) ** 2)
        return {'BPFO': bpfo, 'BPFI': bpfi, 'BSF': bsf, 'FR': fr}
```

```
    def extract_time_domain_features(self, signal_data):
        features = {}
        features['mean'] = np.mean(signal_data)
        features['var'] = np.var(signal_data)
        features['rms'] = np.sqrt(np.mean(signal_data ** 2))
        features['max'] = np.max(signal_data)
        features['crest_factor'] = features['max'] / features['rms']
        features['skewness'] = skew(signal_data)
        features['kurtosis'] = kurtosis(signal_data)
        return features

    def extract_frequency_domain_features(self, signal_data, fs):
        fft_data = np.fft.fft(signal_data)
        fft_magnitude = np.abs(fft_data)
        freqs = np.fft.fftfreq(len(signal_data), 1 / fs)
        positive_freqs = freqs[:len(freqs) // 2]
        positive_magnitude = fft_magnitude[:len(fft_magnitude) // 2]
        features = {}
        features['spectral_centroid'] = np.sum(positive_freqs * positive_magnitude) /
np.sum(positive_magnitude)
        return features, positive_freqs, positive_magnitude

    def extract_wavelet_packet_energy(self, signal_data, wavelet='db4', level=3):
        wp = pywt.WaveletPacket(data=signal_data, wavelet=wavelet, mode='symmetric', maxlevel=level)
        nodes = [node.path for node in wp.get_level(level, 'natural')]
        energies = {}
        total_energy = 0
        for node in nodes:
            coeffs = wp[node].data
            energy = np.sum(coeffs**2)
            energies[node] = energy
            total_energy += energy
        features = {}
        for node, energy in energies.items():
            if total_energy > 0:
                features[f'wp_{node}_energy'] = energy / total_energy
            else:
                features[f'wp_{node}_energy'] = 0
        return features

    def load_and_process_data(self, max_files_per_category=1000):
        print("开始加载和处理 XLSX 数据...")
        all_features = []
```

```
        all_labels = []
        all_filenames = []

        for root, dirs, files in os.walk(self.data_path):
            for file in files:
                if file.endswith('.xlsx') and len(all_features) < max_files_per_category * 4:
                    file_path = os.path.join(root, file)

                    try:
                        if 'OR' in file or 'OR' in root:
                            label = 'OR'
                        elif 'IR' in file or 'IR' in root:
                            label = 'IR'
                        elif 'B' in file or ('B' in root and 'BA' not in root):
                            label = 'B'
                        elif 'N' in file or 'Normal' in root:
                            label = 'N'
                        else:
                            continue

                        df = pd.read_excel(file_path)
                        time_cols = [col for col in df.columns if 'time' in col.lower()]
                        if not time_cols:
                            print(f"警告: {file} 中没有找到时间序列数据列")
                            continue
                        de_cols = [col for col in time_cols if 'de' in col.lower()]
                        if de_cols:
                            signal_col = de_cols[0]
                        else:
                            signal_col = time_cols[0]
                        signal_data = df[signal_col].dropna().values
                        if len(signal_data) < 1000:
                            print(f"警告: {file} 中的数据长度不足 ({len(signal_data)} 个点)")
                            continue
                        rpm_map = {0: 1797, 1: 1772, 2: 1750, 3: 1730}
                        try:
                            load = int(file.split('_')[1])
                            rpm = rpm_map.get(load, 1797)
                        except:
                            rpm = 1797
                        signal_length = min(320000, len(signal_data))
                        signal_data = signal_data[:signal_length]
                        fs = 32000
                        features = {}
```

```
                                time_features = self.extract_time_domain_features(signal_data)
                                features.update(time_features)
                                freq_features,                freqs,                magnitude                =
self.extract_frequency_domain_features(signal_data, fs)
                                features.update(freq_features)
                                fault_freqs = self.calculate_fault_frequencies(rpm)
                                for fault_type, freq in fault_freqs.items():
                                    freq_tolerance = 10
                                    freq_mask = (freqs >= freq - freq_tolerance) & (freqs <= freq +
freq_tolerance)
                                    if np.any(freq_mask):
                                        features[f'{fault_type}_energy'] = np.sum(magnitude[freq_mask] **
2)
                                    else:
                                        features[f'{fault_type}_energy'] = 0
                                wavelet_features = self.extract_wavelet_packet_energy(signal_data)
                                features.update(wavelet_features)
                                features['rpm'] = rpm
                                features['filename'] = file
                                features['sampling_rate'] = fs
                                all_features.append(features)
                                all_labels.append(label)
                                all_filenames.append(file)
                                print(f"处理文件: {file}, 标签: {label}, 特征数: {len(features)}")
                            except Exception as e:
                                print(f"处理文件 {file} 时出错: {e}")
                                continue

        self.features_df = pd.DataFrame(all_features)
        self.features_df['label'] = all_labels
        self.features_df['filename'] = all_filenames
        print(f"\n 数据加载完成！ ")
        print(f"总样本数: {len(self.features_df)}")
        print(f"标签分布:\n{pd.Series(all_labels).value_counts()}")
        return self.features_df

    def visualize_signal_analysis(self, sample_files=None):
        if sample_files is None:
            sample_files = {}
            for label in ['N', 'B', 'IR', 'OR']:
                label_files = self.features_df[self.features_df['label'] == label]['filename'].tolist()
                if label_files:
                    sample_files[label] = label_files[0]
```

```
fig, axes = plt.subplots(4, 3, figsize=(15, 12))
fig.suptitle('轴承故障信号分析 (32kHz 采样率)', fontsize=16)

for i, (label, filename) in enumerate(sample_files.items()):
    if i >= 4:
        break
    file_path = None
    for root, dirs, files in os.walk(self.data_path):
        if filename in files:
            file_path = os.path.join(root, filename)
            break
    if file_path is None:
        continue
    try:
        df = pd.read_excel(file_path)
        time_cols = [col for col in df.columns if 'time' in col.lower()]
        if not time_cols:
            continue
        de_cols = [col for col in time_cols if 'de' in col.lower()]
        if de_cols:
            signal_col = de_cols[0]
        else:
            signal_col = time_cols[0]
        signal_data = df[signal_col].dropna().values
        signal_length = min(32000, len(signal_data))
        signal_data = signal_data[:signal_length]
        fs = 32000
        time_axis = np.arange(len(signal_data)) / fs
        axes[i, 0].plot(time_axis, signal_data)
        axes[i, 0].set_title(f'{label} - 时域信号')
        axes[i, 0].set_xlabel('时间 (s)')
        axes[i, 0].set_ylabel('幅值')
        freqs = np.fft.fftfreq(len(signal_data), 1 / fs)
        fft_magnitude = np.abs(np.fft.fft(signal_data))
        positive_freqs = freqs[:len(freqs) // 2]
        positive_magnitude = fft_magnitude[:len(fft_magnitude) // 2]
        axes[i, 1].plot(positive_freqs, positive_magnitude)
        axes[i, 1].set_title(f'{label} - 频域信号')
        axes[i, 1].set_xlabel('频率 (Hz)')
        axes[i, 1].set_ylabel('幅值')
        axes[i, 1].set_xlim(0, 5000)
        wavelet_features = self.extract_wavelet_packet_energy(signal_data)
        wp_energies = [val for key, val in wavelet_features.items() if key.startswith('wp_')]
        wp_nodes = [key for key in wavelet_features.keys() if key.startswith('wp_')]
```

```
                axes[i, 2].bar(range(len(wp_energies)), wp_energies)
                axes[i, 2].set_title(f'{label} - 小波包能量分布')
                axes[i, 2].set_xlabel('小波包节点')
                axes[i, 2].set_ylabel('能量比例')
                axes[i, 2].set_xticks(range(len(wp_nodes)))
                axes[i, 2].set_xticklabels(wp_nodes, rotation=45)
            except Exception as e:
                print(f"可视化文件 {filename} 时出错: {e}")

        plt.tight_layout()
        plt.show()

    def visualize_features(self):
        if self.features_df is None:
            print("请先加载数据！")
            return

        feature_columns = [col for col in self.features_df.columns
                           if col not in ['label', 'filename', 'rpm', 'sampling_rate']]

        fig, axes = plt.subplots(3, 4, figsize=(16, 12))
        fig.suptitle('主要特征分布对比 (32kHz 采样率)', fontsize=16)

        important_features = ['rms', 'kurtosis', 'crest_factor', 'spectral_centroid',
                              'BPFO_energy', 'BPFI_energy', 'BSF_energy',
                              'wp_aaa_energy', 'wp_aad_energy', 'wp_ada_energy', 'wp_add_energy']

        for i, feature in enumerate(important_features):
            if i >= 12:
                break
            row, col = i // 4, i % 4

            if feature in self.features_df.columns:
                data_for_plot = []
                labels_for_plot = []

                for label in ['N', 'B', 'IR', 'OR']:
                    feature_data = self.features_df[self.features_df['label'] == label][feature]
                    if len(feature_data) > 0:
                        data_for_plot.append(feature_data)
                        labels_for_plot.append(label)

                if data_for_plot:
                    axes[row, col].boxplot(data_for_plot, labels=labels_for_plot)
```

```
                axes[row, col].set_title(feature)
                axes[row, col].tick_params(axis='x', rotation=45)

        plt.tight_layout()
        plt.show()

        feature_data = self.features_df[feature_columns].fillna(0)
        scaler = StandardScaler()
        scaled_features = scaler.fit_transform(feature_data)
        pca = PCA(n_components=2)
        pca_result = pca.fit_transform(scaled_features)
        plt.figure(figsize=(10, 8))
        colors = {'N': 'green', 'B': 'blue', 'IR': 'red', 'OR': 'orange'}

        for label in colors.keys():
            mask = self.features_df['label'] == label
            if np.any(mask):
                plt.scatter(pca_result[mask, 0], pca_result[mask, 1],
                            c=colors[label], label=label, alpha=0.7)

        plt.xlabel(f'主成分 1 (解释方差: {pca.explained_variance_ratio_[0]:.2%})')
        plt.ylabel(f'主成分 2 (解释方差: {pca.explained_variance_ratio_[1]:.2%})')
        plt.title('特征空间 PCA 可视化 (32kHz 采样率)')
        plt.legend()
        plt.grid(True, alpha=0.3)
        plt.show()

        print("\nPCA 特征重要性分析:")
        feature_importance = np.abs(pca.components_).mean(axis=0)
        feature_importance_df = pd.DataFrame({
            'feature': feature_columns,
            'importance': feature_importance
        }).sort_values('importance', ascending=False)

        print(feature_importance_df.head(10))
        return feature_importance_df

    def get_feature_statistics(self):
        if self.features_df is None:
            print("请先加载数据！")
            return None

        print("\n=== 特征统计信息 ===")
        print(f"特征总数: {len([col for col in self.features_df.columns if col not in ['label', 'filename']])}")
```

```
        print(f"样本总数: {len(self.features_df)}")
        print("\n 标签分布:")
        print(self.features_df['label'].value_counts())

        feature_columns = [col for col in self.features_df.columns
                           if col not in ['label', 'filename', 'rpm', 'sampling_rate']]

        stats_by_label = self.features_df.groupby('label')[feature_columns].mean()
        print("\n 各类别主要特征均值对比:")
        important_features = ['rms', 'kurtosis', 'crest_factor', 'BPFO_energy', 'BPFI_energy', 'BSF_energy']
        for feature in important_features:
            if feature in stats_by_label.columns:
                print(f"\n{feature}:")
                print(stats_by_label[feature])

        return stats_by_label

def main():
    root_dir = "C:/Users/19858/Desktop/数模/数据集/源域数据集"
    xlsx_output_dir = "C:/Users/19858/Desktop/源域数据集_XLSX"
    resampled_output_dir = "轴承故障数据集_32kHz"

    print("=== 步骤 1: 转换 MAT 文件为 XLSX 格式 ===")
    DataFormatConverter.convert_all_mat_files(root_dir, xlsx_output_dir)

    print("\n=== 步骤 2: 重采样数据到 32kHz ===")
    DataResampler.process_all_xlsx_files(xlsx_output_dir, resampled_output_dir)

    print("\n=== 步骤 3: 特征提取与分析 ===")
    analyzer = BearingFeatureAnalyzer(resampled_output_dir)
    features_df = analyzer.load_and_process_data(max_files_per_category=1000)
    features_df.to_csv('bearing_features_32kHz.csv', index=False, encoding='utf-8-sig')
    print("\n 特征数据已保存到 bearing_features_32kHz.csv")

    print("\n 开始可视化分析...")
    analyzer.visualize_signal_analysis()
    analyzer.visualize_features()
    stats = analyzer.get_feature_statistics()
    return analyzer, features_df

if __name__ == "__main__":
    analyzer, features_df = main()

#问题 2
```

```
import numpy as np
import pandas as pd
import os
import re
import torch
import torch.nn as nn
import torch.nn.functional as F
from torch.utils.data import Dataset, DataLoader
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder, StandardScaler
from sklearn.metrics import classification_report, confusion_matrix
import matplotlib.pyplot as plt
import seaborn as sns
import warnings
warnings.filterwarnings('ignore')

class BearingDataset(Dataset):
    def __init__(self, X, y, features_dict=None, filename_list=None):
        self.X = X
        self.y = y
        self.features_dict = features_dict
        self.filename_list = filename_list

    def __len__(self):
        return len(self.X)

    def __getitem__(self, idx):
        sample = self.X[idx]
        label = self.y[idx]

        fft_signal = np.fft.fft(sample)
        freq_signal = np.abs(fft_signal)[:513]

        sample = torch.FloatTensor(sample).unsqueeze(0)
        label = torch.LongTensor([label])
        freq_signal = torch.FloatTensor(freq_signal).unsqueeze(0)

        if self.features_dict is not None and self.filename_list is not None:
            filename = self.filename_list[idx]
            features = self.features_dict.get(filename, torch.zeros(33))
            features = torch.FloatTensor(features)
        else:
            features = torch.zeros(33)
```

```
        return sample, freq_signal, features, label

class BearingDatasetBuilder:
    def __init__(self, data_path, features_csv_path=None, sample_length=1024):
        self.data_path = data_path
        self.features_csv_path = features_csv_path
        self.sample_length = sample_length
        self.X = None
        self.y = None
        self.filenames = None
        self.labels = None
        self.label_encoder = LabelEncoder()
        self.features_dict = {}

        if features_csv_path and os.path.exists(features_csv_path):
            self.load_features_from_csv(features_csv_path)

    def load_features_from_csv(self, csv_path):
        features_df = pd.read_csv(csv_path)
        feature_columns = [col for col in features_df.columns if col not in ['filename', 'label']]
        for _, row in features_df.iterrows():
            filename = row['filename']
            features = row[feature_columns].values.astype(np.float32)
            self.features_dict[filename] = features
        print(f"已加载 {len(self.features_dict)} 个文件的特征")

    def load_and_preprocess_data(self):
        all_samples = []
        all_labels = []
        all_filenames = []

        for root, dirs, files in os.walk(self.data_path):
            for file in files:
                if file.endswith('.xlsx'):
                    file_path = os.path.join(root, file)
                    label = self._determine_label_from_path(root, file)
                    if label is None: continue

                    try:
                        df = pd.read_excel(file_path)
                        time_cols = [col for col in df.columns if 'time' in col.lower()]
                        if not time_cols: continue

                        de_cols = [col for col in time_cols if 'de' in col.lower()]
```

```
                signal_col = de_cols[0] if de_cols else time_cols[0]

                signal_data = df[signal_col].dropna().values
                if len(signal_data) < self.sample_length: continue

                num_samples = len(signal_data) // self.sample_length
                for i in range(num_samples):
                    start_idx = i * self.sample_length
                    end_idx = start_idx + self.sample_length
                    sample = signal_data[start_idx:end_idx]
                    all_samples.append(sample)
                    all_labels.append(label)
                    all_filenames.append(file)

                print(f"从文件 {file} 创建了 {num_samples} 个样本, 标签: {label}")
            except Exception as e:
                print(f"处理文件 {file} 时出错: {e}")
                continue

    self.X = np.array(all_samples)
    self.y = np.array(all_labels)
    self.filenames = np.array(all_filenames)

    print(f"总样本数: {len(self.X)}")
    print(f"标签分布:\n{pd.Series(all_labels).value_counts()}")
    return self.X, self.y, self.filenames

def _determine_label_from_path(self, root_path, filename):
    path_lower = root_path.lower()
    if 'normal' in path_lower: return 11

    fault_type = None
    if '/b/' in path_lower or '\\b\\' in path_lower: fault_type = 'B'
    elif '/ir/' in path_lower or '\\ir\\' in path_lower: fault_type = 'IR'
    elif '/or/' in path_lower or '\\or\\' in path_lower: fault_type = 'OR'

    diameter_pattern = r'(\d{3,4})'
    diameter_match = re.search(diameter_pattern, root_path)
    if diameter_match:
        try: diameter = int(diameter_match.group(1))
        except: return None
    else:
        filename_pattern = r'([BIROR]{1,2})(\d{2,4})'
        filename_match = re.search(filename_pattern, filename, re.IGNORECASE)
```

```
            if filename_match:
                try: diameter = int(filename_match.group(2))
                except: return None
            else: return None

        if diameter == 7: diameter_code = 0
        elif diameter == 14: diameter_code = 1
        elif diameter == 21: diameter_code = 2
        elif diameter == 28: diameter_code = 3
        else: return None

        if fault_type == 'B': return diameter_code
        elif fault_type == 'IR': return diameter_code + 4
        elif fault_type == 'OR':
            if diameter_code == 3: return None
            return diameter_code + 8
        return None

    def encode_labels(self):
        self.y_encoded = self.label_encoder.fit_transform(self.y)
        self.labels = self.label_encoder.classes_
        print(f"标签映射: {dict(zip(range(len(self.labels)), self.labels))}")
        return self.y_encoded

    def split_dataset(self, test_size=0.2, val_size=0.1, random_state=42):
        X_temp, X_test, y_temp, y_test, filenames_temp, filenames_test = train_test_split(
            self.X, self.y_encoded, self.filenames,
            test_size=test_size, stratify=self.y_encoded, random_state=random_state)

        val_ratio = val_size / (1 - test_size)
        X_train, X_val, y_train, y_val, filenames_train, filenames_val = train_test_split(
            X_temp, y_temp, filenames_temp,
            test_size=val_ratio, stratify=y_temp, random_state=random_state)

        print(f"训练集: {X_train.shape[0]} 个样本")
        print(f"验证集: {X_val.shape[0]} 个样本")
        print(f"测试集: {X_test.shape[0]} 个样本")
        return X_train, X_val, X_test, y_train, y_val, y_test, filenames_train, filenames_val, filenames_test

    def normalize_data(self, X_train, X_val, X_test):
        self.scaler = StandardScaler()
        X_train_reshaped = X_train.reshape(X_train.shape[0], -1)
        X_val_reshaped = X_val.reshape(X_val.shape[0], -1)
        X_test_reshaped = X_test.reshape(X_test.shape[0], -1)
```

```
        X_train_normalized = self.scaler.fit_transform(X_train_reshaped)
        X_val_normalized = self.scaler.transform(X_val_reshaped)
        X_test_normalized = self.scaler.transform(X_test_reshaped)

        X_train_normalized = X_train_normalized.reshape(X_train.shape)
        X_val_normalized = X_val_normalized.reshape(X_val.shape)
        X_test_normalized = X_test_normalized.reshape(X_test.shape)
        return X_train_normalized, X_val_normalized, X_test_normalized

    def create_datasets(self, X_train, X_val, X_test, y_train, y_val, y_test, filenames_train, filenames_val,
filenames_test):
        train_dataset = BearingDataset(X_train, y_train, self.features_dict, filenames_train)
        val_dataset = BearingDataset(X_val, y_val, self.features_dict, filenames_val)
        test_dataset = BearingDataset(X_test, y_test, self.features_dict, filenames_test)
        return train_dataset, val_dataset, test_dataset

    def create_dataloaders(self, train_dataset, val_dataset, test_dataset, batch_size=32):
        train_loader = DataLoader(train_dataset, batch_size=batch_size, shuffle=True)
        val_loader = DataLoader(val_dataset, batch_size=batch_size, shuffle=False)
        test_loader = DataLoader(test_dataset, batch_size=batch_size, shuffle=False)
        return train_loader, val_loader, test_loader

class FeatureExtractor(nn.Module):
    def __init__(self, input_length=1024):
        super(FeatureExtractor, self).__init__()
        self.conv1 = nn.Conv1d(1, 64, kernel_size=7, stride=1, padding=3)
        self.bn1 = nn.BatchNorm1d(64)
        self.relu1 = nn.ReLU()
        self.pool1 = nn.MaxPool1d(kernel_size=2, stride=2)
        self.conv2 = nn.Conv1d(64, 128, kernel_size=5, stride=1, padding=2)
        self.bn2 = nn.BatchNorm1d(128)
        self.relu2 = nn.ReLU()
        self.pool2 = nn.MaxPool1d(kernel_size=2, stride=2)
        self.conv3 = nn.Conv1d(128, 256, kernel_size=3, stride=1, padding=1)
        self.bn3 = nn.BatchNorm1d(256)
        self.relu3 = nn.ReLU()
        self.pool3 = nn.MaxPool1d(kernel_size=2, stride=2)

    def forward(self, x):
        x = self.pool1(self.relu1(self.bn1(self.conv1(x))))
        x = self.pool2(self.relu2(self.bn2(self.conv2(x))))
        x = self.pool3(self.relu3(self.bn3(self.conv3(x))))
        x = x.view(x.size(0), -1)
```

```
        return x

class FrequencyFeatureExtractor(nn.Module):
    def __init__(self, input_length=513):
        super(FrequencyFeatureExtractor, self).__init__()
        self.conv1 = nn.Conv1d(1, 64, kernel_size=5, stride=1, padding=2)
        self.bn1 = nn.BatchNorm1d(64)
        self.relu1 = nn.ReLU()
        self.pool1 = nn.MaxPool1d(kernel_size=2, stride=2)
        self.conv2 = nn.Conv1d(64, 128, kernel_size=3, stride=1, padding=1)
        self.bn2 = nn.BatchNorm1d(128)
        self.relu2 = nn.ReLU()
        self.pool2 = nn.MaxPool1d(kernel_size=2, stride=2)

    def forward(self, x):
        x = self.pool1(self.relu1(self.bn1(self.conv1(x))))
        x = self.pool2(self.relu2(self.bn2(self.conv2(x))))
        x = torch.mean(x, dim=2)
        return x

class FeatureProcessingModule(nn.Module):
    def __init__(self, input_dim=33, hidden_dim=64):
        super(FeatureProcessingModule, self).__init__()
        self.fc1 = nn.Linear(input_dim, hidden_dim)
        self.bn1 = nn.BatchNorm1d(hidden_dim)
        self.relu1 = nn.ReLU()
        self.fc2 = nn.Linear(hidden_dim, hidden_dim)
        self.bn2 = nn.BatchNorm1d(hidden_dim)
        self.relu2 = nn.ReLU()

    def forward(self, x):
        x = self.relu1(self.bn1(self.fc1(x)))
        x = self.relu2(self.bn2(self.fc2(x)))
        return x

class FeatureFusionModule(nn.Module):
    def __init__(self, time_feat_dim=64, freq_feat_dim=128, handcrafted_feat_dim=64):
        super(FeatureFusionModule, self).__init__()
        self.attention_time = nn.Linear(time_feat_dim, 1)
        self.attention_freq = nn.Linear(freq_feat_dim, 1)
        self.attention_handcrafted = nn.Linear(handcrafted_feat_dim, 1)
        self.fusion_fc = nn.Linear(time_feat_dim + freq_feat_dim + handcrafted_feat_dim, 256)
        self.bn_fusion = nn.BatchNorm1d(256)
        self.relu_fusion = nn.ReLU()
```

```
        self.fc1 = nn.Linear(256, 128)
        self.bn1 = nn.BatchNorm1d(128)
        self.relu1 = nn.ReLU()

    def forward(self, time_feat, freq_feat, handcrafted_feat):
        att_time = torch.sigmoid(self.attention_time(time_feat))
        att_freq = torch.sigmoid(self.attention_freq(freq_feat))
        att_handcrafted = torch.sigmoid(self.attention_handcrafted(handcrafted_feat))
        time_feat = time_feat * att_time
        freq_feat = freq_feat * att_freq
        handcrafted_feat = handcrafted_feat * att_handcrafted
        fused = torch.cat([time_feat, freq_feat, handcrafted_feat], dim=1)
        fused = self.relu_fusion(self.bn_fusion(self.fusion_fc(fused)))
        fused = self.relu1(self.bn1(self.fc1(fused)))
        return fused

class Classifier(nn.Module):
    def __init__(self, input_dim=128, num_classes=13):
        super(Classifier, self).__init__()
        self.fc1 = nn.Linear(input_dim, 64)
        self.bn1 = nn.BatchNorm1d(64)
        self.relu1 = nn.ReLU()
        self.dropout = nn.Dropout(0.5)
        self.fc2 = nn.Linear(64, num_classes)

    def forward(self, x):
        x = self.relu1(self.bn1(self.fc1(x)))
        x = self.dropout(x)
        x = self.fc2(x)
        return x

class BearingQTFFC(nn.Module):
    def __init__(self, num_classes=13):
        super(BearingQTFFC, self).__init__()
        self.time_feature_extractor = FeatureExtractor()
        self.freq_feature_extractor = FrequencyFeatureExtractor()
        self.handcrafted_feature_processor = FeatureProcessingModule(input_dim=33, hidden_dim=64)
        self.feature_fusion = FeatureFusionModule(
            time_feat_dim=256, freq_feat_dim=128, handcrafted_feat_dim=64)
        self.classifier = Classifier(input_dim=128, num_classes=num_classes)

    def forward(self, x_time, x_freq, x_features):
        time_base_features = self.time_feature_extractor(x_time)
        freq_base_features = self.freq_feature_extractor(x_freq)
```

```
        handcrafted_features = self.handcrafted_feature_processor(x_features)
        fused_features = self.feature_fusion(time_base_features, freq_base_features,
handcrafted_features)
        output = self.classifier(fused_features)
        return output

class BearingTrainer:
    def __init__(self, model, device, model_name="bearing_model"):
        self.model = model.to(device)
        self.device = device
        self.model_name = model_name
        self.train_losses = []
        self.val_losses = []
        self.train_accs = []
        self.val_accs = []

    def train(self, train_loader, val_loader, criterion, optimizer, scheduler, num_epochs=50):
        best_val_acc = 0.0
        for epoch in range(num_epochs):
            self.model.train()
            running_loss = 0.0
            correct = 0
            total = 0

            for inputs, freq_inputs, features, labels in train_loader:
                inputs = inputs.to(self.device)
                freq_inputs = freq_inputs.to(self.device)
                features = features.to(self.device)
                labels = labels.to(self.device).squeeze()

                outputs = self.model(inputs, freq_inputs, features)
                loss = criterion(outputs, labels)

                optimizer.zero_grad()
                loss.backward()
                optimizer.step()

                running_loss += loss.item() * inputs.size(0)
                _, predicted = torch.max(outputs.data, 1)
                total += labels.size(0)
                correct += (predicted == labels).sum().item()

            train_loss = running_loss / len(train_loader.dataset)
            train_acc = 100 * correct / total
```

```
            val_loss, val_acc = self.evaluate(val_loader, criterion)

            self.train_losses.append(train_loss)
            self.val_losses.append(val_loss)
            self.train_accs.append(train_acc)
            self.val_accs.append(val_acc)

            if scheduler: scheduler.step()

            print(f'Epoch [{epoch + 1}/{num_epochs}], '
                  f'Train Loss: {train_loss:.4f}, Train Acc: {train_acc:.2f}%, '
                  f'Val Loss: {val_loss:.4f}, Val Acc: {val_acc:.2f}%')

            if val_acc > best_val_acc:
                best_val_acc = val_acc
                torch.save(self.model.state_dict(), f"{self.model_name}_best.pth")
                print(f'Best model saved with validation accuracy: {val_acc:.2f}%')

        torch.save(self.model.state_dict(), f"{self.model_name}_final.pth")
        return self.train_losses, self.val_losses, self.train_accs, self.val_accs

    def evaluate(self, data_loader, criterion):
        self.model.eval()
        running_loss = 0.0
        correct = 0
        total = 0

        with torch.no_grad():
            for inputs, freq_inputs, features, labels in data_loader:
                inputs = inputs.to(self.device)
                freq_inputs = freq_inputs.to(self.device)
                features = features.to(self.device)
                labels = labels.to(self.device).squeeze()

                outputs = self.model(inputs, freq_inputs, features)
                loss = criterion(outputs, labels)

                running_loss += loss.item() * inputs.size(0)
                _, predicted = torch.max(outputs.data, 1)
                total += labels.size(0)
                correct += (predicted == labels).sum().item()

        loss = running_loss / len(data_loader.dataset)
        accuracy = 100 * correct / total
```

```
        return loss, accuracy

    def test(self, test_loader, criterion):
        test_loss, test_acc = self.evaluate(test_loader, criterion)
        print(f'Test Loss: {test_loss:.4f}, Test Acc: {test_acc:.2f}%')

        self.model.eval()
        all_preds = []
        all_labels = []

        with torch.no_grad():
            for inputs, freq_inputs, features, labels in test_loader:
                inputs = inputs.to(self.device)
                freq_inputs = freq_inputs.to(self.device)
                features = features.to(self.device)
                labels = labels.to(self.device).squeeze()

                outputs = self.model(inputs, freq_inputs, features)
                _, predicted = torch.max(outputs.data, 1)

                all_preds.extend(predicted.cpu().numpy())
                all_labels.extend(labels.cpu().numpy())

        print("\nClassification Report:")
        class_names = [f'Class {i}' for i in range(len(np.unique(all_labels)))]
        print(classification_report(all_labels, all_preds, target_names=class_names))

        cm = confusion_matrix(all_labels, all_preds)
        plt.figure(figsize=(10, 8))
        sns.heatmap(cm, annot=True, fmt='d', cmap='Blues',
                    xticklabels=class_names,
                    yticklabels=class_names)
        plt.ylabel('True Label')
        plt.xlabel('Predicted Label')
        plt.title('Confusion Matrix')
        plt.savefig(f"{self.model_name}_confusion_matrix.png")
        plt.show()

        return test_loss, test_acc, all_preds, all_labels

    def plot_training_history(self):
        fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(12, 4))
        ax1.plot(self.train_losses, label='Train Loss')
        ax1.plot(self.val_losses, label='Validation Loss')
```

```
        ax1.set_title('Loss vs Epochs')
        ax1.set_xlabel('Epochs')
        ax1.set_ylabel('Loss')
        ax1.legend()
        ax1.grid(True)
        ax2.plot(self.train_accs, label='Train Accuracy')
        ax2.plot(self.val_accs, label='Validation Accuracy')
        ax2.set_title('Accuracy vs Epochs')
        ax2.set_xlabel('Epochs')
        ax2.set_ylabel('Accuracy (%)')
        ax2.legend()
        ax2.grid(True)
        plt.tight_layout()
        plt.savefig(f"{self.model_name}_training_history.png")
        plt.show()

def main():
    device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
    print(f"使用设备: {device}")

    data_path = "轴承故障分类数据集"
    dataset_builder = BearingDatasetBuilder(data_path)
    X, y, filenames = dataset_builder.load_and_preprocess_data()
    y_encoded = dataset_builder.encode_labels()

    X_train, X_val, X_test, y_train, y_val, y_test, filenames_train, filenames_val, filenames_test =
dataset_builder.split_dataset()
    X_train_norm, X_val_norm, X_test_norm = dataset_builder.normalize_data(X_train, X_val, X_test)

    train_dataset, val_dataset, test_dataset = dataset_builder.create_datasets(
        X_train_norm, X_val_norm, X_test_norm,
        y_train, y_val, y_test,
        filenames_train, filenames_val, filenames_test)

    train_loader, val_loader, test_loader = dataset_builder.create_dataloaders(
        train_dataset, val_dataset, test_dataset, batch_size=32)

    model = BearingQTFFC(num_classes=12)
    criterion = nn.CrossEntropyLoss()
    optimizer = optim.Adam(model.parameters(), lr=0.001)
    scheduler = StepLR(optimizer, step_size=10, gamma=0.1)

    trainer = BearingTrainer(model, device)
    trainer.train(train_loader, val_loader, criterion, optimizer, scheduler, num_epochs=50)
```

```
    trainer.test(test_loader, criterion)
    trainer.plot_training_history()

if __name__ == "__main__":
    main()

#问题 3
import numpy as np
import pandas as pd
import os
import re
import torch
import torch.nn as nn
import torch.nn.functional as F
from torch.utils.data import Dataset, DataLoader
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder, StandardScaler
from sklearn.metrics import classification_report, confusion_matrix
import matplotlib.pyplot as plt
import seaborn as sns
import warnings
from collections import defaultdict
warnings.filterwarnings('ignore')

def aggregate_predictions_by_filename(predictions, true_labels, probabilities, filenames, label_encoder):
    filename_groups = defaultdict(list)

    for i, filename in enumerate(filenames):
        base_filename = re.sub(r'_\d+$', '', filename)
        filename_groups[base_filename].append({
            'prediction': predictions[i],
            'true_label': true_labels[i] if len(np.unique(true_labels)) > 1 else -1,
            'probabilities': probabilities[i],
            'sample_filename': filename
        })

    aggregated_results = []

    for filename, samples in filename_groups.items():
        if len(samples) == 0: continue

        all_predictions = [s['prediction'] for s in samples]
        all_probabilities = np.array([s['probabilities'] for s in samples])
```

```
unique, counts = np.unique(all_predictions, return_counts=True)
vote_prediction = unique[np.argmax(counts)]
vote_confidence = np.max(counts) / len(all_predictions)

avg_probabilities = np.mean(all_probabilities, axis=0)
avg_prediction = np.argmax(avg_probabilities)
avg_confidence = np.max(avg_probabilities)

max_sample_idx = np.argmax(np.max(all_probabilities, axis=1))
max_prob_prediction = all_predictions[max_sample_idx]
max_prob_confidence = np.max(all_probabilities[max_sample_idx])

final_prediction = vote_prediction
final_confidence = vote_confidence

true_label = samples[0]['true_label'] if samples[0]['true_label'] != -1 else -1

if label_encoder is not None:
    final_prediction_label = label_encoder.inverse_transform([final_prediction])[0]
    vote_prediction_label = label_encoder.inverse_transform([vote_prediction])[0]
    avg_prediction_label = label_encoder.inverse_transform([avg_prediction])[0]
    max_prob_prediction_label = label_encoder.inverse_transform([max_prob_prediction])[0]

    if true_label != -1:
        true_label_decoded = label_encoder.inverse_transform([true_label])[0]
    else:
        true_label_decoded = "Unknown"
else:
    final_prediction_label = final_prediction
    vote_prediction_label = vote_prediction
    avg_prediction_label = avg_prediction
    max_prob_prediction_label = max_prob_prediction
    true_label_decoded = true_label if true_label != -1 else "Unknown"

aggregated_results.append({
    'Filename': filename,
    'Final_Prediction': final_prediction_label,
    'Final_Prediction_Class': final_prediction,
    'Final_Confidence': final_confidence,
    'Vote_Prediction': vote_prediction_label,
    'Vote_Confidence': vote_confidence,
    'Avg_Probability_Prediction': avg_prediction_label,
    'Avg_Probability_Confidence': avg_confidence,
    'Max_Probability_Prediction': max_prob_prediction_label,
```

```
                'Max_Probability_Confidence': max_prob_confidence,
                'True_Label': true_label_decoded,
                'True_Class': true_label,
                'Num_Samples': len(samples),
                'Sample_Filenames': ';'.join([s['sample_filename'] for s in samples])
            })

    return aggregated_results

def predict_target_domain(model, target_loader, device, label_encoder, save_path="target_domain_predictions.csv"):
    model.eval()
    all_predictions = []
    all_true_labels = []
    all_filenames = []
    all_probabilities = []

    with torch.no_grad():
        for batch_idx, (inputs, freq_inputs, features, labels, domains) in enumerate(target_loader):
            inputs = inputs.to(device)
            freq_inputs = freq_inputs.to(device)
            features = features.to(device)

            class_outputs, _, _, _ = model(inputs, freq_inputs, features)

            probabilities = F.softmax(class_outputs, dim=1)
            max_prob, predictions = torch.max(probabilities, 1)

            all_predictions.extend(predictions.cpu().numpy())
            all_true_labels.extend(labels.cpu().numpy())
            all_probabilities.extend(probabilities.cpu().numpy())

            if hasattr(target_loader.dataset, 'filename_list'):
                start_idx = batch_idx * target_loader.batch_size
                end_idx = start_idx + len(predictions)
                batch_filenames = target_loader.dataset.filename_list[start_idx:end_idx]
                all_filenames.extend(batch_filenames)
            else:
                all_filenames.extend([f"sample_{batch_idx}_{i}" for i in range(len(predictions))])

    all_predictions = np.array(all_predictions)
    all_true_labels = np.array(all_true_labels).flatten()
    all_probabilities = np.array(all_probabilities)
```

```
    aggregated_results = aggregate_predictions_by_filename(
        all_predictions, all_true_labels, all_probabilities, all_filenames, label_encoder
    )

    summary_df = pd.DataFrame(aggregated_results)

    detailed_df = pd.DataFrame({
        'Sample_Filename': all_filenames,
        'Sample_Prediction': all_predictions,
        'Sample_True_Label': all_true_labels,
        'Sample_Confidence': np.max(all_probabilities, axis=1)
    })

    if label_encoder is not None:
        detailed_df['Sample_Prediction_Label']                                                    =
label_encoder.inverse_transform(detailed_df['Sample_Prediction'])
        if len(np.unique(all_true_labels)) > 1:
            detailed_df['Sample_True_Label_Decoded']                                              =
label_encoder.inverse_transform(detailed_df['Sample_True_Label'])
        else:
            detailed_df['Sample_True_Label_Decoded'] = "Unknown"

    for i in range(all_probabilities.shape[1]):
        class_name = f"Prob_Class_{i}"
        if label_encoder is not None and i < len(label_encoder.classes_):
            class_name = f"Prob_{label_encoder.classes_[i]}"
        detailed_df[class_name] = all_probabilities[:, i]

    summary_df.to_csv(save_path.replace('.csv', '_summary.csv'), index=False, encoding='utf-8-sig')
    detailed_df.to_csv(save_path.replace('.csv', '_detailed.csv'), index=False, encoding='utf-8-sig')

    print("\n" + "="*80)
    print("目标域故障分类预测结果（按文件名汇总）")
    print("="*80)

    prediction_counts = summary_df['Final_Prediction'].value_counts().sort_index()
    print("\n 最终预测结果统计:")
    print("-"*40)
    for label, count in prediction_counts.items():
        percentage = (count / len(summary_df)) * 100
        print(f"{label}: {count}个文件 ({percentage:.2f}%)")

    print("\n 预测方法一致性分析:")
    print("-"*40)
```

```
    vote_avg_agreement = np.mean(summary_df['Vote_Prediction'] == summary_df['Avg_Probability_Prediction'])
    vote_max_agreement = np.mean(summary_df['Vote_Prediction'] == summary_df['Max_Probability_Prediction'])
    avg_max_agreement = np.mean(summary_df['Avg_Probability_Prediction'] == summary_df['Max_Probability_Prediction'])

    print(f"投票法 vs 平均概率法: {vote_avg_agreement*100:.2f}% 一致")
    print(f"投票法 vs 最大概率法: {vote_max_agreement*100:.2f}% 一致")
    print(f"平均概率法 vs 最大概率法: {avg_max_agreement*100:.2f}% 一致")

    if len(np.unique(all_true_labels)) > 1:
        file_level_accuracy = np.mean(
            [row['Final_Prediction_Class'] == row['True_Class']
             for _, row in summary_df.iterrows() if row['True_Class'] != -1]
        ) * 100

        print(f"\n 文件级别分类准确率: {file_level_accuracy:.2f}%")

        sample_level_accuracy = np.mean(all_predictions == all_true_labels) * 100
        print(f"样本级别分类准确率: {sample_level_accuracy:.2f}%")

        valid_files = summary_df[summary_df['True_Class'] != -1]
        if len(valid_files) > 0:
            print("\n 文件级别详细分类报告:")
            print("-"*40)
            print(classification_report(
                valid_files['True_Label'],
                valid_files['Final_Prediction'],
                target_names=label_encoder.classes_
            ))

            plt.figure(figsize=(10, 8))
            cm = confusion_matrix(valid_files['True_Class'], valid_files['Final_Prediction_Class'])
            sns.heatmap(cm, annot=True, fmt='d', cmap='Blues',
                        xticklabels=label_encoder.classes_,
                        yticklabels=label_encoder.classes_)
            plt.title('目标域文件级别混淆矩阵')
            plt.xlabel('预测标签')
            plt.ylabel('真实标签')
            plt.tight_layout()
            plt.savefig("target_domain_file_level_confusion_matrix.png")
            plt.show()
```

```
    print(f"\n 文件级别置信度统计:")
    print("-"*40)
    print(f"平均置信度: {summary_df['Final_Confidence'].mean():.4f}")
    print(f"置信度标准差: {summary_df['Final_Confidence'].std():.4f}")
    print(f"最低置信度: {summary_df['Final_Confidence'].min():.4f}")
    print(f"最高置信度: {summary_df['Final_Confidence'].max():.4f}")

    print(f"\n 样本数量统计:")
    print("-"*40)
    print(f"总文件数: {len(summary_df)}")
    print(f"总样本数: {len(detailed_df)}")
    print(f"平均每个文件的样本数: {len(detailed_df) / len(summary_df):.2f}")
    print(f"最多样本的文件: {summary_df['Num_Samples'].max()} 个样本")
    print(f"最少样本的文件: {summary_df['Num_Samples'].min()} 个样本")

    print(f"\n 前 20 个文件的预测结果:")
    print("-"*100)
    display_columns = ['Filename', 'Final_Prediction', 'True_Label', 'Final_Confidence', 'Num_Samples']
    if len(np.unique(all_true_labels)) <= 1:
        display_columns = ['Filename', 'Final_Prediction', 'Final_Confidence', 'Num_Samples']

    pd.set_option('display.width', 1000)
    pd.set_option('display.max_columns', 10)
    print(summary_df[display_columns].head(20).to_string(index=False))

    print(f"\n 汇总预测结果已保存至: {save_path.replace('.csv', '_summary.csv')}")
    print(f"详细样本结果已保存至: {save_path.replace('.csv', '_detailed.csv')}")

    return summary_df, detailed_df

class FeatureExtractor(nn.Module):
    def __init__(self, input_length=1024):
        super(FeatureExtractor, self).__init__()
        self.conv1 = nn.Conv1d(1, 64, kernel_size=7, stride=1, padding=3)
        self.bn1 = nn.BatchNorm1d(64)
        self.relu1 = nn.ReLU()
        self.pool1 = nn.MaxPool1d(kernel_size=2, stride=2)
        self.conv2 = nn.Conv1d(64, 128, kernel_size=5, stride=1, padding=2)
        self.bn2 = nn.BatchNorm1d(128)
        self.relu2 = nn.ReLU()
        self.pool2 = nn.MaxPool1d(kernel_size=2, stride=2)
        self.conv3 = nn.Conv1d(128, 256, kernel_size=3, stride=1, padding=1)
        self.bn3 = nn.BatchNorm1d(256)
        self.relu3 = nn.ReLU()
```

```
        self.pool3 = nn.MaxPool1d(kernel_size=2, stride=2)

    def forward(self, x):
        x = self.pool1(self.relu1(self.bn1(self.conv1(x))))
        x = self.pool2(self.relu2(self.bn2(self.conv2(x))))
        x = self.pool3(self.relu3(self.bn3(self.conv3(x))))
        x = x.view(x.size(0), -1)
        return x

class FrequencyFeatureExtractor(nn.Module):
    def __init__(self, input_length=513):
        super(FrequencyFeatureExtractor, self).__init__()
        self.conv1 = nn.Conv1d(1, 64, kernel_size=5, stride=1, padding=2)
        self.bn1 = nn.BatchNorm1d(64)
        self.relu1 = nn.ReLU()
        self.pool1 = nn.MaxPool1d(kernel_size=2, stride=2)
        self.conv2 = nn.Conv1d(64, 128, kernel_size=3, stride=1, padding=1)
        self.bn2 = nn.BatchNorm1d(128)
        self.relu2 = nn.ReLU()
        self.pool2 = nn.MaxPool1d(kernel_size=2, stride=2)

    def forward(self, x):
        x = self.pool1(self.relu1(self.bn1(self.conv1(x))))
        x = self.pool2(self.relu2(self.bn2(self.conv2(x))))
        x = torch.mean(x, dim=2)
        return x

class FeatureProcessingModule(nn.Module):
    def __init__(self, input_dim=33, hidden_dim=64):
        super(FeatureProcessingModule, self).__init__()
        self.fc1 = nn.Linear(input_dim, hidden_dim)
        self.bn1 = nn.BatchNorm1d(hidden_dim)
        self.relu1 = nn.ReLU()
        self.fc2 = nn.Linear(hidden_dim, hidden_dim)
        self.bn2 = nn.BatchNorm1d(hidden_dim)
        self.relu2 = nn.ReLU()

    def forward(self, x):
        x = self.relu1(self.bn1(self.fc1(x)))
        x = self.relu2(self.bn2(self.fc2(x)))
        return x

class DomainInvariantExtractor(nn.Module):
    def __init__(self, input_dim=32768):
```

```
        super(DomainInvariantExtractor, self).__init__()
        self.fc1 = nn.Linear(input_dim, input_dim//16)
        self.bn1 = nn.BatchNorm1d(input_dim//16)
        self.relu1 = nn.ReLU()
        self.fc2 = nn.Linear(input_dim//16, input_dim//512)
        self.bn2 = nn.BatchNorm1d(input_dim//512)
        self.relu2 = nn.ReLU()

    def forward(self, x):
        x = self.relu1(self.bn1(self.fc1(x)))
        x = self.relu2(self.bn2(self.fc2(x)))
        return x

class DomainSpecificExtractor(nn.Module):
    def __init__(self, input_dim=32768):
        super(DomainSpecificExtractor, self).__init__()
        self.fc1 = nn.Linear(input_dim, input_dim//16)
        self.bn1 = nn.BatchNorm1d(input_dim//16)
        self.relu1 = nn.ReLU()
        self.fc2 = nn.Linear(input_dim//16, input_dim//512)
        self.bn2 = nn.BatchNorm1d(input_dim//512)
        self.relu2 = nn.ReLU()

    def forward(self, x):
        x = self.relu1(self.bn1(self.fc1(x)))
        x = self.relu2(self.bn2(self.fc2(x)))
        return x

class FeatureFusionModule(nn.Module):
    def __init__(self, time_feat_dim=64, freq_feat_dim=128, handcrafted_feat_dim=64):
        super(FeatureFusionModule, self).__init__()
        self.attention_time = nn.Linear(time_feat_dim, 1)
        self.attention_freq = nn.Linear(freq_feat_dim, 1)
        self.attention_handcrafted = nn.Linear(handcrafted_feat_dim, 1)
        self.fusion_fc = nn.Linear(time_feat_dim + freq_feat_dim + handcrafted_feat_dim, 256)
        self.bn_fusion = nn.BatchNorm1d(256)
        self.relu_fusion = nn.ReLU()
        self.fc1 = nn.Linear(256, 128)
        self.bn1 = nn.BatchNorm1d(128)
        self.relu1 = nn.ReLU()

    def forward(self, time_feat, freq_feat, handcrafted_feat):
        att_time = torch.sigmoid(self.attention_time(time_feat))
        att_freq = torch.sigmoid(self.attention_freq(freq_feat))
```

```
        att_handcrafted = torch.sigmoid(self.attention_handcrafted(handcrafted_feat))
        time_feat = time_feat * att_time
        freq_feat = freq_feat * att_freq
        handcrafted_feat = handcrafted_feat * att_handcrafted
        fused = torch.cat([time_feat, freq_feat, handcrafted_feat], dim=1)
        fused = self.relu_fusion(self.bn_fusion(self.fusion_fc(fused)))
        fused = self.relu1(self.bn1(self.fc1(fused)))
        return fused

class Classifier(nn.Module):
    def __init__(self, input_dim=128, num_classes=13):
        super(Classifier, self).__init__()
        self.fc1 = nn.Linear(input_dim, 64)
        self.bn1 = nn.BatchNorm1d(64)
        self.relu1 = nn.ReLU()
        self.dropout = nn.Dropout(0.5)
        self.fc2 = nn.Linear(64, num_classes)

    def forward(self, x):
        x = self.relu1(self.bn1(self.fc1(x)))
        x = self.dropout(x)
        x = self.fc2(x)
        return x

class BearingDataset(Dataset):
    def __init__(self, X, y, features_dict=None, filename_list=None):
        self.X = X
        self.y = y
        self.features_dict = features_dict
        self.filename_list = filename_list

    def __len__(self):
        return len(self.X)

    def __getitem__(self, idx):
        sample = self.X[idx]
        label = self.y[idx]
        fft_signal = np.fft.fft(sample)
        freq_signal = np.abs(fft_signal)[:513]
        sample = torch.FloatTensor(sample).unsqueeze(0)
        label = torch.LongTensor([label])
        freq_signal = torch.FloatTensor(freq_signal).unsqueeze(0)
        if self.features_dict is not None and self.filename_list is not None:
            filename = self.filename_list[idx]
```

```
                features = self.features_dict.get(filename, torch.zeros(33))
                features = torch.FloatTensor(features)
            else:
                features = torch.zeros(33)
            return sample, freq_signal, features, label

class BearingDatasetBuilder:
    def __init__(self, data_path, features_csv_path=None, sample_length=1024):
        self.data_path = data_path
        self.features_csv_path = features_csv_path
        self.sample_length = sample_length
        self.X = None
        self.y = None
        self.filenames = None
        self.labels = None
        self.label_encoder = LabelEncoder()
        self.features_dict = {}
        self.scaler = StandardScaler()

        if features_csv_path and os.path.exists(features_csv_path):
            self.load_features_from_csv(features_csv_path)

    def load_features_from_csv(self, csv_path):
        features_df = pd.read_csv(csv_path)
        feature_columns = [col for col in features_df.columns if col not in ['filename', 'label']]
        for _, row in features_df.iterrows():
            filename = row['filename']
            features = row[feature_columns].values.astype(np.float32)
            self.features_dict[filename] = features
        print(f"已加载 {len(self.features_dict)} 个文件的特征")

    def load_and_preprocess_data(self):
        all_samples = []
        all_labels = []
        all_filenames = []

        for root, dirs, files in os.walk(self.data_path):
            for file in files:
                if file.endswith('.xlsx'):
                    file_path = os.path.join(root, file)
                    label = self._determine_label_from_path(root, file)
                    if label is None: continue

                    try:
```

```
                df = pd.read_excel(file_path)
                time_cols = [col for col in df.columns if 'time' in col.lower()]
                if not time_cols: continue

                de_cols = [col for col in time_cols if 'de' in col.lower()]
                signal_col = de_cols[0] if de_cols else time_cols[0]

                signal_data = df[signal_col].dropna().values
                if len(signal_data) < self.sample_length: continue

                num_samples = len(signal_data) // self.sample_length
                for i in range(num_samples):
                    start_idx = i * self.sample_length
                    end_idx = start_idx + self.sample_length
                    sample = signal_data[start_idx:end_idx]
                    all_samples.append(sample)
                    all_labels.append(label)
                    all_filenames.append(file)

                print(f"从文件 {file} 创建了 {num_samples} 个样本, 标签: {label}")
            except Exception as e:
                print(f"处理文件 {file} 时出错: {e}")
                continue

        self.X = np.array(all_samples)
        self.y = np.array(all_labels)
        self.filenames = np.array(all_filenames)

        print(f"总样本数: {len(self.X)}")
        print(f"标签分布:\n{pd.Series(all_labels).value_counts()}")
        return self.X, self.y, self.filenames

    def _determine_label_from_path(self, root_path, filename):
        path_lower = root_path.lower()
        if 'normal' in path_lower: return 11

        fault_type = None
        if '/b/' in path_lower or '\\b\\' in path_lower: fault_type = 'B'
        elif '/ir/' in path_lower or '\\ir\\' in path_lower: fault_type = 'IR'
        elif '/or/' in path_lower or '\\or\\' in path_lower: fault_type = 'OR'

        diameter_pattern = r'(\d{3,4})'
        diameter_match = re.search(diameter_pattern, root_path)
        if diameter_match:
```

```
            try: diameter = int(diameter_match.group(1))
            except: return None
        else:
            filename_pattern = r'([BIROR]{1,2})(\d{2,4})'
            filename_match = re.search(filename_pattern, filename, re.IGNORECASE)
            if filename_match:
                try: diameter = int(filename_match.group(2))
                except: return None
            else: return None

        if diameter == 7: diameter_code = 0
        elif diameter == 14: diameter_code = 1
        elif diameter == 21: diameter_code = 2
        elif diameter == 28: diameter_code = 3
        else: return None

        if fault_type == 'B': return diameter_code
        elif fault_type == 'IR': return diameter_code + 4
        elif fault_type == 'OR':
            if diameter_code == 3: return None
            return diameter_code + 8
        return None

    def encode_labels(self):
        self.y_encoded = self.label_encoder.fit_transform(self.y)
        self.labels = self.label_encoder.classes_
        print(f"标签映射: {dict(zip(range(len(self.labels)), self.labels))}")
        return self.y_encoded

    def split_dataset(self, test_size=0.2, val_size=0.1, random_state=42):
        X_temp, X_test, y_temp, y_test, filenames_temp, filenames_test = train_test_split(
            self.X, self.y_encoded, self.filenames,
            test_size=test_size, stratify=self.y_encoded, random_state=random_state)

        val_ratio = val_size / (1 - test_size)
        X_train, X_val, y_train, y_val, filenames_train, filenames_val = train_test_split(
            X_temp, y_temp, filenames_temp,
            test_size=val_ratio, stratify=y_temp, random_state=random_state)

        print(f"训练集: {X_train.shape[0]} 个样本")
        print(f"验证集: {X_val.shape[0]} 个样本")
        print(f"测试集: {X_test.shape[0]} 个样本")
        return X_train, X_val, X_test, y_train, y_val, y_test, filenames_train, filenames_val, filenames_test
```

```
    def normalize_data(self, X_train, X_val, X_test):
        self.scaler = StandardScaler()
        X_train_reshaped = X_train.reshape(X_train.shape[0], -1)
        X_val_reshaped = X_val.reshape(X_val.shape[0], -1)
        X_test_reshaped = X_test.reshape(X_test.shape[0], -1)

        X_train_normalized = self.scaler.fit_transform(X_train_reshaped)
        X_val_normalized = self.scaler.transform(X_val_reshaped)
        X_test_normalized = self.scaler.transform(X_test_reshaped)

        X_train_normalized = X_train_normalized.reshape(X_train.shape)
        X_val_normalized = X_val_normalized.reshape(X_val.shape)
        X_test_normalized = X_test_normalized.reshape(X_test.shape)
        return X_train_normalized, X_val_normalized, X_test_normalized

    def create_datasets(self, X_train, X_val, X_test, y_train, y_val, y_test, filenames_train, filenames_val,
filenames_test):
        train_dataset = BearingDataset(X_train, y_train, self.features_dict, filenames_train)
        val_dataset = BearingDataset(X_val, y_val, self.features_dict, filenames_val)
        test_dataset = BearingDataset(X_test, y_test, self.features_dict, filenames_test)
        return train_dataset, val_dataset, test_dataset

    def create_dataloaders(self, train_dataset, val_dataset, test_dataset, batch_size=32):
        train_loader = DataLoader(train_dataset, batch_size=batch_size, shuffle=True)
        val_loader = DataLoader(val_dataset, batch_size=batch_size, shuffle=False)
        test_loader = DataLoader(test_dataset, batch_size=batch_size, shuffle=False)
        return train_loader, val_loader, test_loader

class DomainAdversarialDataset(Dataset):
    def __init__(self, X, y, domain_labels, features_dict=None, filename_list=None):
        self.X = X
        self.y = y
        self.domain_labels = domain_labels
        self.features_dict = features_dict
        self.filename_list = filename_list

    def __len__(self):
        return len(self.X)

    def __getitem__(self, idx):
        sample = self.X[idx]
        label = self.y[idx]
        domain_label = self.domain_labels[idx]
        fft_signal = np.fft.fft(sample)
```

```
        freq_signal = np.abs(fft_signal)[:513]
        sample = torch.FloatTensor(sample).unsqueeze(0)
        label = torch.LongTensor([label])
        domain_label = torch.LongTensor([domain_label])
        freq_signal = torch.FloatTensor(freq_signal).unsqueeze(0)
        if self.features_dict is not None and self.filename_list is not None:
            filename = self.filename_list[idx]
            features = self.features_dict.get(filename, torch.zeros(33))
            features = torch.FloatTensor(features)
        else:
            features = torch.zeros(33)
        return sample, freq_signal, features, label, domain_label

class GradientReversalLayer(torch.autograd.Function):
    @staticmethod
    def forward(ctx, x, alpha):
        ctx.alpha = alpha
        return x.view_as(x)

    @staticmethod
    def backward(ctx, grad_output):
        output = grad_output.neg() * ctx.alpha
        return output, None

class DomainClassifier(nn.Module):
    def __init__(self, input_dim=128, hidden_dim=64):
        super(DomainClassifier, self).__init__()
        self.grl = GradientReversalLayer.apply
        self.fc1 = nn.Linear(input_dim, hidden_dim)
        self.bn1 = nn.BatchNorm1d(hidden_dim)
        self.relu1 = nn.ReLU()
        self.fc2 = nn.Linear(hidden_dim, 2)

    def forward(self, x, alpha=1.0):
        x = self.grl(x, alpha)
        x = self.relu1(self.bn1(self.fc1(x)))
        x = self.fc2(x)
        return x

class DomainAdversarialQTFFC(nn.Module):
    def __init__(self, num_classes=13):
        super(DomainAdversarialQTFFC, self).__init__()
        self.time_feature_extractor = FeatureExtractor()
        self.freq_feature_extractor = FrequencyFeatureExtractor()
```

```
        self.handcrafted_feature_processor = FeatureProcessingModule(input_dim=33, hidden_dim=64)
        self.domain_invariant_extractor = DomainInvariantExtractor()
        self.domain_specific_extractor = DomainSpecificExtractor()
        self.feature_fusion = FeatureFusionModule(
            time_feat_dim=64,
            freq_feat_dim=128,
            handcrafted_feat_dim=64
        )
        self.classifier = Classifier(input_dim=128, num_classes=num_classes)
        self.domain_classifier = DomainClassifier(input_dim=128, hidden_dim=64)

    def forward(self, x_time, x_freq, x_features, alpha=0.1):
        time_base_features = self.time_feature_extractor(x_time)
        domain_invariant = self.domain_invariant_extractor(time_base_features)
        domain_specific = self.domain_specific_extractor(time_base_features)
        freq_features = self.freq_feature_extractor(x_freq)
        handcrafted_features = self.handcrafted_feature_processor(x_features) * 0.2
        fused_features = self.feature_fusion(
            domain_invariant,
            freq_features,
            handcrafted_features
        )
        class_outputs = self.classifier(fused_features)
        domain_outputs = self.domain_classifier(fused_features, alpha)
        return class_outputs, domain_outputs, domain_invariant, domain_specific

class DomainAdversarialTrainer:
    def __init__(self, model, device, model_name="domain_adversarial_model"):
        self.model = model.to(device)
        self.device = device
        self.model_name = model_name
        self.train_losses = []
        self.val_losses = []
        self.train_accs = []
        self.val_accs = []
        self.domain_losses = []
        self.domain_accs = []

    def train(self, source_loader, target_loader, val_loader,
              criterion, domain_criterion, optimizer, scheduler,
              num_epochs=100, alpha=0.1, beta=0.1):
        best_val_acc = 0.0

        for epoch in range(num_epochs):
```

```
        self.model.train()
        running_loss = 0.0
        running_domain_loss = 0.0
        correct = 0
        domain_correct = 0
        total = 0
        domain_total = 0

        source_iter = iter(source_loader)
        target_iter = iter(target_loader)
        min_len = min(len(source_loader), len(target_loader))

        for batch_idx in range(min_len):
            try:
                source_data = next(source_iter)
                source_inputs, source_freq, source_features, source_labels, source_domains =
source_data
                target_data = next(target_iter)
                target_inputs, target_freq, target_features, target_labels, target_domains =
target_data
            except StopIteration:
                break

            inputs = torch.cat([source_inputs, target_inputs], 0)
            freq_inputs = torch.cat([source_freq, target_freq], 0)
            features = torch.cat([source_features, target_features], 0)
            labels = torch.cat([source_labels, target_labels], 0).squeeze()
            domain_labels = torch.cat([
                torch.zeros(source_inputs.size(0)),
                torch.ones(target_inputs.size(0))
            ]).long()

            inputs = inputs.to(self.device)
            freq_inputs = freq_inputs.to(self.device)
            features = features.to(self.device)
            labels = labels.to(self.device)
            domain_labels = domain_labels.to(self.device)

            class_outputs, domain_outputs, _, _ = self.model(inputs, freq_inputs, features, alpha)
            source_class_outputs = class_outputs[:source_inputs.size(0)]
            source_labels = labels[:source_inputs.size(0)]
            class_loss = criterion(source_class_outputs, source_labels)
            domain_loss = domain_criterion(domain_outputs, domain_labels)
            total_loss = class_loss - alpha * domain_loss
```

```
            optimizer.zero_grad()
            total_loss.backward()
            optimizer.step()

            _, predicted = torch.max(source_class_outputs.data, 1)
            total += source_labels.size(0)
            correct += (predicted == source_labels).sum().item()

            _, domain_predicted = torch.max(domain_outputs.data, 1)
            domain_total += domain_labels.size(0)
            domain_correct += (domain_predicted == domain_labels).sum().item()

            running_loss += class_loss.item() * source_inputs.size(0)
            running_domain_loss += domain_loss.item() * inputs.size(0)

        train_loss = running_loss / total if total > 0 else 0
        train_domain_loss = running_domain_loss / domain_total if domain_total > 0 else 0
        train_acc = 100 * correct / total if total > 0 else 0
        domain_acc = 100 * domain_correct / domain_total if domain_total > 0 else 0

        val_loss, val_acc = self.evaluate(val_loader, criterion)

        self.train_losses.append(train_loss)
        self.val_losses.append(val_loss)
        self.train_accs.append(train_acc)
        self.val_accs.append(val_acc)
        self.domain_losses.append(train_domain_loss)
        self.domain_accs.append(domain_acc)

        if scheduler:
            scheduler.step()

        print(f'Epoch [{epoch+1}/{num_epochs}], '
              f'Train Loss: {train_loss:.4f}, Train Acc: {train_acc:.2f}%, '
              f'Domain Loss: {train_domain_loss:.4f}, Domain Acc: {domain_acc:.2f}%, '
              f'Val Loss: {val_loss:.4f}, Val Acc: {val_acc:.2f}%')

        if val_acc > best_val_acc:
            best_val_acc = val_acc
            torch.save(self.model.state_dict(), f"{self.model_name}_best.pth")
            print(f'Best model saved with validation accuracy: {val_acc:.2f}%')

    torch.save(self.model.state_dict(), f"{self.model_name}_final.pth")
```

```
        print('Domain adversarial training completed.')
        return self.train_losses, self.val_losses, self.train_accs, self.val_accs

    def evaluate(self, data_loader, criterion):
        self.model.eval()
        running_loss = 0.0
        correct = 0
        total = 0

        with torch.no_grad():
            for inputs, freq_inputs, features, labels, domains in data_loader:
                inputs = inputs.to(self.device)
                freq_inputs = freq_inputs.to(self.device)
                features = features.to(self.device)
                labels = labels.to(self.device).squeeze()

                class_outputs, _, _, _ = self.model(inputs, freq_inputs, features)
                loss = criterion(class_outputs, labels)

                running_loss += loss.item() * inputs.size(0)
                _, predicted = torch.max(class_outputs.data, 1)
                total += labels.size(0)
                correct += (predicted == labels).sum().item()

        loss = running_loss / len(data_loader.dataset) if len(data_loader.dataset) > 0 else 0
        accuracy = 100 * correct / total if total > 0 else 0
        return loss, accuracy

def main():
    device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
    print(f"使用设备: {device}")

    source_data_path = r"轴承故障分类数据集 2"
    target_save_path = r"目标域数据集_处理后"
    complete_dataset_path = r"完整数据集_源域+目标域"

    dataset_builder = BearingDatasetBuilder(source_data_path)
    X_train, X_val, X_test, y_train, y_val, y_test, filenames_train, filenames_val, filenames_test =
dataset_builder.load_datasets(source_data_path)

    source_domain_labels_train = np.zeros(len(X_train))
    train_dataset = DomainAdversarialDataset(
        X_train, y_train, source_domain_labels_train,
        dataset_builder.features_dict, filenames_train
```

```
)

source_domain_labels_val = np.zeros(len(X_val))
val_dataset = DomainAdversarialDataset(
    X_val, y_val, source_domain_labels_val,
    dataset_builder.features_dict, filenames_val
)

source_domain_labels_test = np.zeros(len(X_test))
test_dataset = DomainAdversarialDataset(
    X_test, y_test, source_domain_labels_test,
    dataset_builder.features_dict, filenames_test
)

print("加载已处理的目标域数据...")
X_target = np.load(os.path.join(complete_dataset_path, 'X_target.npy'))
y_target = np.load(os.path.join(complete_dataset_path, 'y_target.npy'))
filenames_target = np.load(os.path.join(complete_dataset_path, 'filenames_target.npy'))
domain_labels_target = np.ones(len(X_target))

source_domain_dataset = DomainAdversarialDataset(
    X_train, y_train, source_domain_labels_train,
    dataset_builder.features_dict, filenames_train
)

target_domain_dataset = DomainAdversarialDataset(
    X_target, y_target, domain_labels_target,
    dataset_builder.features_dict, filenames_target
)

batch_size = 32
source_loader = DataLoader(source_domain_dataset, batch_size=batch_size, shuffle=True)
target_loader = DataLoader(target_domain_dataset, batch_size=batch_size, shuffle=True)
val_loader = DataLoader(val_dataset, batch_size=batch_size, shuffle=False)
test_loader = DataLoader(test_dataset, batch_size=batch_size, shuffle=False)

model = DomainAdversarialQTFFC(num_classes=len(dataset_builder.labels))
model = model.to(device)

print(f"域对抗模型参数数量: {sum(p.numel() for p in model.parameters()):,}")

criterion = nn.CrossEntropyLoss()
domain_criterion = nn.CrossEntropyLoss()
optimizer = optim.Adam(model.parameters(), lr=0.001, weight_decay=1e-5)
```

```
scheduler = StepLR(optimizer, step_size=30, gamma=0.1)

trainer = DomainAdversarialTrainer(model, device, model_name="domain_adversarial_bearing")

train_losses, val_losses, train_accs, val_accs = trainer.train(
    source_loader, target_loader, val_loader,
    criterion, domain_criterion, optimizer, scheduler,
    num_epochs=50, alpha=0.1, beta=0.1
)

plt.figure(figsize=(15, 5))

plt.subplot(1, 3, 1)
plt.plot(train_losses, label='Train Loss')
plt.plot(val_losses, label='Val Loss')
plt.title('Loss vs Epochs')
plt.xlabel('Epochs')
plt.ylabel('Loss')
plt.legend()
plt.grid(True)

plt.subplot(1, 3, 2)
plt.plot(train_accs, label='Train Accuracy')
plt.plot(val_accs, label='Val Accuracy')
plt.title('Accuracy vs Epochs')
plt.xlabel('Epochs')
plt.ylabel('Accuracy (%)')
plt.legend()
plt.grid(True)

plt.subplot(1, 3, 3)
plt.plot(trainer.domain_losses, label='Domain Loss', color='red')
plt.title('Domain Loss vs Epochs')
plt.xlabel('Epochs')
plt.ylabel('Domain Loss')
plt.legend()
plt.grid(True)

plt.tight_layout()
plt.savefig("domain_adversarial_training_history.png")
plt.show()

print("\n 测试模型在源域测试集上的表现:")
test_loss, test_acc = trainer.evaluate(test_loader, criterion)
```

```
    print(f'测试集损失: {test_loss:.4f}, 测试集准确率: {test_acc:.2f}%')

    print("\n 开始预测目标域故障分类...")
    summary_df, detailed_df = predict_target_domain(
        model, target_loader, device, dataset_builder.label_encoder,
        save_path="target_domain_predictions.csv"
    )

    print("\n" + "="*80)
    print("最终文件分类结果")
    print("="*80)
    for _, row in summary_df.iterrows():
        print(f"文件: {row['Filename']}")
        print(f"  最终预测: {row['Final_Prediction']} (置信度: {row['Final_Confidence']:.4f})")
        print(f"  投票预测: {row['Vote_Prediction']} (置信度: {row['Vote_Confidence']:.4f})")
        print(f"  平均概率预测: {row['Avg_Probability_Prediction']} (置信度: {row['Avg_Probability_Confidence']:.4f})")
        print(f"  最大概率预测: {row['Max_Probability_Prediction']} (置信度: {row['Max_Probability_Confidence']:.4f})")
        if row['True_Label'] != "Unknown":
            print(f"  真实标签: {row['True_Label']}")
        print(f"  样本数量: {row['Num_Samples']}")
        print("-"*50)

    return trainer, model, summary_df, detailed_df

if __name__ == "__main__":
    trainer, model, summary_df, detailed_df = main()

#问题四
import torch
import torch.nn as nn
import torch.nn.functional as F
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.metrics import classification_report, confusion_matrix
import pandas as pd
import os
import torch.optim as optim
from torch.optim.lr_scheduler import StepLR
import re
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder, StandardScaler
```

```
import torch
from torch.utils.data import Dataset, DataLoader
import warnings
warnings.filterwarnings('ignore')

class FeatureExtractor(nn.Module):
    """基础特征提取器"""
    def __init__(self, input_channels=1, num_classes=13):
        super(FeatureExtractor, self).__init__()
        self.conv1 = nn.Conv1d(input_channels, 64, kernel_size=3, padding=1)
        self.bn1 = nn.BatchNorm1d(64)
        self.relu1 = nn.ReLU()
        self.pool1 = nn.MaxPool1d(2)

        self.conv2 = nn.Conv1d(64, 128, kernel_size=3, padding=1)
        self.bn2 = nn.BatchNorm1d(128)
        self.relu2 = nn.ReLU()
        self.pool2 = nn.MaxPool1d(2)

        self.conv3 = nn.Conv1d(128, 256, kernel_size=3, padding=1)
        self.bn3 = nn.BatchNorm1d(256)
        self.relu3 = nn.ReLU()
        self.pool3 = nn.MaxPool1d(2)

        # 自适应池化以适应不同长度的输入
        self.adaptive_pool = nn.AdaptiveAvgPool1d(128)

    def forward(self, x):
        x = self.pool1(self.relu1(self.bn1(self.conv1(x))))
        x = self.pool2(self.relu2(self.bn2(self.conv2(x))))
        x = self.pool3(self.relu3(self.bn3(self.conv3(x))))
        x = self.adaptive_pool(x)
        return x

class FrequencyFeatureExtractor(nn.Module):
    def __init__(self, input_channels=1, output_dim=128):
        super(FrequencyFeatureExtractor, self).__init__()
        self.conv1 = nn.Conv1d(input_channels, 32, kernel_size=3, padding=1)
        self.bn1 = nn.BatchNorm1d(32)
        self.relu1 = nn.ReLU()
        self.pool1 = nn.MaxPool1d(2)

        self.conv2 = nn.Conv1d(32, 64, kernel_size=3, padding=1)
        self.bn2 = nn.BatchNorm1d(64)
```

```
        self.relu2 = nn.ReLU()
        self.pool2 = nn.MaxPool1d(2)

        self.conv3 = nn.Conv1d(64, 128, kernel_size=3, padding=1)
        self.bn3 = nn.BatchNorm1d(128)
        self.relu3 = nn.ReLU()
        self.pool3 = nn.MaxPool1d(2)

        self.adaptive_pool = nn.AdaptiveAvgPool1d(128)

    def forward(self, x):
        x = self.pool1(self.relu1(self.bn1(self.conv1(x))))
        x = self.pool2(self.relu2(self.bn2(self.conv2(x))))
        x = self.pool3(self.relu3(self.bn3(self.conv3(x))))
        x = self.adaptive_pool(x)
        return x

class FeatureFusionModule(nn.Module):
    def __init__(self, time_feat_dim=256 * 128, freq_feat_dim=128 * 128, handcrafted_feat_dim=64):
        super(FeatureFusionModule, self).__init__()
        # 调整维度
        self.time_fc = nn.Linear(time_feat_dim, 128)
        self.freq_fc = nn.Linear(freq_feat_dim, 128)
        self.handcrafted_fc = nn.Linear(handcrafted_feat_dim, 128)

        self.fusion_fc = nn.Linear(128 * 3, 256)
        self.relu = nn.ReLU()

    def forward(self, time_feat, freq_feat, handcrafted_feat):
        time_feat = time_feat.view(time_feat.size(0), -1)
        freq_feat = freq_feat.view(freq_feat.size(0), -1)

        time_feat = self.time_fc(time_feat)
        freq_feat = self.freq_fc(freq_feat)
        handcrafted_feat = self.handcrafted_fc(handcrafted_feat)

        fused = torch.cat((time_feat, freq_feat, handcrafted_feat), dim=1)
        fused = self.relu(self.fusion_fc(fused))
        return fused

class Classifier(nn.Module):
    def __init__(self, input_dim=256, num_classes=13):
        super(Classifier, self).__init__()
        self.fc1 = nn.Linear(input_dim, 128)
```

```
        self.bn1 = nn.BatchNorm1d(128)
        self.relu = nn.ReLU()
        self.dropout = nn.Dropout(0.5)
        self.fc2 = nn.Linear(128, num_classes)

    def forward(self, x):
        x = self.relu(self.bn1(self.fc1(x)))
        x = self.dropout(x)
        x = self.fc2(x)
        return x

class ExplainableFeatureExtractor(FeatureExtractor):
    def __init__(self, input_channels=1, num_classes=13):
        super(ExplainableFeatureExtractor, self).__init__(input_channels, num_classes)

    def get_temporal_features(self, x):
        x = self.pool1(self.relu1(self.bn1(self.conv1(x))))
        x = self.pool2(self.relu2(self.bn2(self.conv2(x))))
        x = self.pool3(self.relu3(self.bn3(self.conv3(x))))
        x = self.adaptive_pool(x)
        x = x.transpose(1, 2)  # 从 (batch, features, seq) 转为 (batch, seq, features)
        return x

class ExplainableBearingQTFFC(nn.Module):

    def __init__(self, num_classes=13, handcrafted_feature_dim=13):
        super(ExplainableBearingQTFFC, self).__init__()
        self.handcrafted_feature_dim = handcrafted_feature_dim

        self.time_feature_extractor = ExplainableFeatureExtractor()

        self.freq_feature_extractor = FrequencyFeatureExtractor()

        self.handcrafted_feature_processor = ExplainableFeatureProcessingModule(
            input_dim=handcrafted_feature_dim, hidden_dim=64)

        self.attention_mechanism = TemporalAttention(input_dim=256)

        self.feature_fusion = FeatureFusionModule(
            time_feat_dim=256 * 128,  # 特征提取器输出维度
            freq_feat_dim=128 * 128,
            handcrafted_feat_dim=64
        )
```

```
        self.classifier = Classifier(input_dim=256, num_classes=num_classes)

        self.attention_weights = None
        self.feature_importance = None

    def forward(self, x_time, x_freq, x_features):
        time_features = self.time_feature_extractor(x_time)

        freq_features = self.freq_feature_extractor(x_freq)

        handcrafted_features, feature_importance = self.handcrafted_feature_processor(x_features)
        self.feature_importance = feature_importance
        temporal_features = self.time_feature_extractor.get_temporal_features(x_time)
        attention_weights = self.attention_mechanism(temporal_features)
        self.attention_weights = attention_weights

        fused_features = self.feature_fusion(time_features, freq_features, handcrafted_features)

        output = self.classifier(fused_features)

        return output, handcrafted_features

    def get_explanations(self):
        explanations = {
            'feature_importance': self.feature_importance,
            'attention_weights': self.attention_weights
        }
        return explanations

class ExplainableFeatureProcessingModule(nn.Module):

    def __init__(self, input_dim=21, hidden_dim=64):
        super(ExplainableFeatureProcessingModule, self).__init__()

        self.fc1 = nn.Linear(input_dim, hidden_dim)
        self.bn1 = nn.BatchNorm1d(hidden_dim)
        self.relu1 = nn.ReLU()

        self.fc2 = nn.Linear(hidden_dim, hidden_dim)
        self.bn2 = nn.BatchNorm1d(hidden_dim)
        self.relu2 = nn.ReLU()

    def forward(self, x):
```

```
        feature_weights = self.fc1.weight.data.clone()

        x = self.relu1(self.bn1(self.fc1(x)))
        x = self.relu2(self.bn2(self.fc2(x)))

        return x, feature_weights

class TemporalAttention(nn.Module):
    def __init__(self, input_dim=256, hidden_dim=64):
        super(TemporalAttention, self).__init__()
        self.attention_fc = nn.Linear(input_dim, hidden_dim)
        self.attention_weights = nn.Linear(hidden_dim, 1)

    def forward(self, x):
        # x shape: (batch_size, seq_len, input_dim)
        batch_size, seq_len, input_dim = x.shape

        attention_scores = torch.tanh(self.attention_fc(x))    # (batch_size, seq_len, hidden_dim)
        attention_scores = self.attention_weights(attention_scores)    # (batch_size, seq_len, 1)
        attention_scores = attention_scores.squeeze(-1)    # (batch_size, seq_len)
        attention_weights = F.softmax(attention_scores, dim=-1)    # (batch_size, seq_len)

        return attention_weights

class ModelExplainer:

    def __init__(self, model, feature_names=None, class_names=None):
        self.model = model
        self.feature_names = feature_names or [f'Feature_{i}' for i in range(13)]
        self.class_names = class_names or [f'Class_{i}' for i in range(13)]

    def visualize_feature_importance(self, feature_weights, save_path=None):

        if feature_weights is None:
            print("没有可用的特征权重数据")
            return

        if len(feature_weights.shape) == 2:
            avg_importance = torch.mean(torch.abs(feature_weights), dim=0)
        else:
            avg_importance = torch.abs(feature_weights)

        importance_scores = avg_importance.cpu().numpy()
```

```
        plt.figure(figsize=(12, 6))

        if len(self.feature_names) != len(importance_scores):
            self.feature_names = [f'Feature_{i}' for i in range(len(importance_scores))]

        indices = np.arange(len(importance_scores))
        plt.bar(indices, importance_scores, color='skyblue', alpha=0.7)

        plt.xlabel('手工特征')
        plt.ylabel('重要性得分')
        plt.title('手工特征重要性分析')
        plt.xticks(indices, self.feature_names, rotation=45, ha='right')
        plt.grid(True, alpha=0.3)
        plt.tight_layout()

        if save_path:
            plt.savefig(save_path, dpi=300, bbox_inches='tight')
        plt.show()

        return importance_scores

    def visualize_attention_weights(self, attention_weights, original_signal=None, save_path=None):

        if attention_weights is None:
            print("没有可用的注意力权重数据")
            return

        if len(attention_weights.shape) > 1:
            sample_attention = attention_weights[0].cpu().detach().numpy()
        else:
            sample_attention = attention_weights.cpu().detach().numpy()

        fig, (ax1, ax2) = plt.subplots(2, 1, figsize=(15, 8))
        if original_signal is not None:
            if len(original_signal.shape) > 1:
                sample_signal = original_signal[0].cpu().numpy()
            else:
                sample_signal = original_signal.cpu().numpy()

            time_axis = np.arange(len(sample_signal))
            ax1.plot(time_axis, sample_signal, 'b-', alpha=0.7, linewidth=1)
            ax1.set_title('原始振动信号')
            ax1.set_ylabel('振幅')
            ax1.grid(True, alpha=0.3)
```

```
        time_axis_attention = np.arange(len(sample_attention))
        ax2.bar(time_axis_attention, sample_attention, alpha=0.7, color='red')
        ax2.set_title('模型注意力分布')
        ax2.set_xlabel('时间点')
        ax2.set_ylabel('注意力权重')
        ax2.grid(True, alpha=0.3)

        plt.tight_layout()

        if save_path:
            plt.savefig(save_path, dpi=300, bbox_inches='tight')
        plt.show()

        return sample_attention

    def analyze_prediction(self, input_data, prediction, true_label=None):
        explanations = self.model.get_explanations()

        fig, axes = plt.subplots(1, 2, figsize=(15, 5))
        if explanations['feature_importance'] is not None:
            feature_importance = explanations['feature_importance']
            if len(feature_importance.shape) == 2:
                avg_importance = torch.mean(torch.abs(feature_importance), dim=0)
            else:
                avg_importance = torch.abs(feature_importance)

            importance_scores = avg_importance.cpu().numpy()

            axes[0].bar(range(len(importance_scores)), importance_scores, color='lightgreen')
            axes[0].set_title('特征重要性分析')
            axes[0].set_xlabel('特征索引')
            axes[0].set_ylabel('重要性得分')
            axes[0].grid(True, alpha=0.3)
                if prediction is not None:
            if isinstance(prediction, torch.Tensor):
                probabilities = F.softmax(prediction, dim=0).cpu().detach().numpy()
            else:
                probabilities = prediction

            axes[1].bar(range(len(probabilities)), probabilities, color='lightcoral')
            axes[1].set_title('预测概率分布')
            axes[1].set_xlabel('类别')
            axes[1].set_ylabel('概率')
```

```
                axes[1].set_xticks(range(len(probabilities)))
                axes[1].set_xticklabels(self.class_names, rotation=45)
                axes[1].grid(True, alpha=0.3)

                if true_label is not None:
                    axes[1].axvline(x=true_label, color='red', linestyle='--',
                                    linewidth=2, label=f'真实标签: {true_label}')
                    axes[1].legend()

        plt.tight_layout()
        plt.show()

        return explanations

class BearingDataset(Dataset):
    def __init__(self, signals, labels, features_dict, filenames=None):
        self.signals = signals
        self.labels = labels
        self.features_dict = features_dict
        self.filenames = filenames or [f"sample_{i}" for i in range(len(signals))]

    def __len__(self):
        return len(self.signals)

    def __getitem__(self, idx):
        signal = self.signals[idx]
        label = self.labels[idx]
        filename = self.filenames[idx]

        features = self.features_dict.get(filename, np.zeros(13))

        signal_tensor = torch.tensor(signal, dtype=torch.float32).unsqueeze(0)
        features_tensor = torch.tensor(features, dtype=torch.float32)
        label_tensor = torch.tensor(label, dtype=torch.long)

        fft_signal = np.abs(np.fft.rfft(signal))
        fft_tensor = torch.tensor(fft_signal, dtype=torch.float32).unsqueeze(0)

        return signal_tensor, fft_tensor, features_tensor, label_tensor

class BearingDatasetBuilder:
    def __init__(self):
        self.features_dict = {}
        self.class_mapping = {
```

```
                'B007': 0, 'B014': 1, 'B021': 2, 'B028': 3,
                'IR007': 4, 'IR014': 5, 'IR021': 6, 'IR028': 7,
                'OR007': 8, 'OR014': 9, 'OR021': 10,
                'Normal': 11
            }

    def extract_features(self, signal):
        """提取手工特征（简化版）"""
        # 实际应用中应实现更复杂的特征提取
        features = np.zeros(13)
        features[0] = np.mean(signal)   # 均值
        features[1] = np.std(signal)    # 标准差
        features[2] = np.max(signal)    # 最大值
        features[3] = np.min(signal)    # 最小值
        features[4] = np.median(signal)   # 中位数
        features[5] = np.percentile(signal, 25)   # 25%分位数
        features[6] = np.percentile(signal, 75)   # 75%分位数
        features[7] = np.sqrt(np.mean(signal**2))   # RMS
        features[8] = np.max(np.abs(signal))   # 峰值
        features[9] = features[8] / features[7]   # 峰值因子
        features[10] = features[7] / np.mean(np.abs(signal))   # 波形因子
        features[11] = np.sum(signal**2)   # 能量
        features[12] = np.mean(np.abs(np.diff(signal)))   # 平均差分
        return features


class ExplainableBearingTrainer(BearingTrainer):
    def __init__(self, model, device, model_name="explainable_bearing_model",
                 feature_names=None, class_names=None):
        super(ExplainableBearingTrainer, self).__init__(model, device, model_name)
        self.explainer = ModelExplainer(model, feature_names, class_names)
        self.attention_history = []
        self.feature_importance_history = []

    def train(self, train_loader, val_loader, criterion, optimizer, scheduler, num_epochs=50):
        best_val_acc = 0.0

        for epoch in range(num_epochs):
            self.model.train()
            running_loss = 0.0
            correct = 0
            total = 0

            epoch_attentions = []
```

```
epoch_feature_importances = []

for batch_idx, (inputs, freq_inputs, features, labels) in enumerate(train_loader):
    inputs = inputs.to(self.device)
    freq_inputs = freq_inputs.to(self.device)
    features = features.to(self.device)
    labels = labels.to(self.device).squeeze()
    outputs, _ = self.model(inputs, freq_inputs, features)
    explanations = self.model.get_explanations()
    if explanations['attention_weights'] is not None:
        try:
            attention_mean = explanations['attention_weights'].mean().item()
            if not np.isnan(attention_mean):
                epoch_attentions.append(attention_mean)
        except Exception as e:
            print(f"记录注意力权重时出错: {e}")

    # 记录特征重要性
    if explanations['feature_importance'] is not None:
        try:
            feature_imp = explanations['feature_importance']
            if len(feature_imp.shape) == 2:
                importance_mean = feature_imp.abs().mean().item()
            else:
                importance_mean = feature_imp.abs().mean().item()

            if not np.isnan(importance_mean):
                epoch_feature_importances.append(importance_mean)
        except Exception as e:
            print(f"记录特征重要性时出错: {e}")
    loss = criterion(outputs, labels)
    optimizer.zero_grad()
    loss.backward()
    optimizer.step()

    # 统计
    running_loss += loss.item() * inputs.size(0)
    _, predicted = torch.max(outputs.data, 1)
    total += labels.size(0)
    correct += (predicted == labels).sum().item()
if epoch_attentions:
    self.attention_history.append(np.mean(epoch_attentions))
else:
    self.attention_history.append(0.1)  # 默认值
```

```
            if epoch_feature_importances:
                self.feature_importance_history.append(np.mean(epoch_feature_importances))
            else:
                self.feature_importance_history.append(0.1)   # 默认值

            train_loss = running_loss / len(train_loader.dataset)
            train_acc = 100 * correct / total

            val_loss, val_acc = self.evaluate(val_loader, criterion)

            self.train_losses.append(train_loss)
            self.val_losses.append(val_loss)
            self.train_accs.append(train_acc)
            self.val_accs.append(val_acc)

            if scheduler:
                scheduler.step()

            print(f'Epoch [{epoch + 1}/{num_epochs}], '
                  f'Train Loss: {train_loss:.4f}, Train Acc: {train_acc:.2f}%, '
                  f'Val Loss: {val_loss:.4f}, Val Acc: {val_acc:.2f}%')

            if val_acc > best_val_acc:
                best_val_acc = val_acc
                torch.save(self.model.state_dict(), f"{self.model_name}_best.pth")
                print(f'Best model saved with validation accuracy: {val_acc:.2f}%')
        torch.save(self.model.state_dict(), f"{self.model_name}_final.pth")
        print('Training completed.')

        self.visualize_explanations('final')

        return self.train_losses, self.val_losses, self.train_accs, self.val_accs

    def visualize_explanations(self, epoch):
        explanations = self.model.get_explanations()
        fig, axes = plt.subplots(2, 2, figsize=(15, 10))
        if explanations['feature_importance'] is not None:
            try:
                feature_importance = explanations['feature_importance']
                if isinstance(feature_importance, torch.Tensor):
                    if len(feature_importance.shape) == 2:
                        avg_importance = torch.mean(torch.abs(feature_importance), dim=0)
                    else:
```

```
                    avg_importance = torch.abs(feature_importance)

                importance_scores = avg_importance.cpu().detach().numpy()

                axes[0,  0].bar(range(len(importance_scores)),  importance_scores,  color='skyblue',
alpha=0.7)
                axes[0, 0].set_title('Feature Importance')
                axes[0, 0].set_xlabel('Feature Index')
                axes[0, 0].set_ylabel('Importance Score')
                axes[0, 0].grid(True, alpha=0.3)
                if hasattr(self.explainer, 'feature_names') and self.explainer.feature_names:
                    if len(self.explainer.feature_names) == len(importance_scores):
                        axes[0, 0].set_xticks(range(len(importance_scores)))
                        axes[0,      0].set_xticklabels(self.explainer.feature_names,      rotation=45,
ha='right')
            except Exception as e:
                print(f"绘制特征重要性时出错: {e}")
                axes[0, 0].text(0.5, 0.5, 'No Feature Importance Data',
                            ha='center', va='center', transform=axes[0, 0].transAxes)
                axes[0, 0].set_title('Feature Importance')
        else:
            axes[0, 0].text(0.5, 0.5, 'No Feature Importance Data',
                        ha='center', va='center', transform=axes[0, 0].transAxes)
            axes[0, 0].set_title('Feature Importance')

        if self.attention_history and len(self.attention_history) > 0:
            try:
                axes[0, 1].plot(self.attention_history, marker='o', linewidth=2, markersize=4)
                axes[0, 1].set_title('Attention Weight History')
                axes[0, 1].set_xlabel('Epoch')
                axes[0, 1].set_ylabel('Average Attention Weight')
                axes[0, 1].grid(True, alpha=0.3)
            except Exception as e:
                print(f"绘制注意力历史时出错: {e}")
                axes[0, 1].text(0.5, 0.5, 'No Attention History Data',
                            ha='center', va='center', transform=axes[0, 1].transAxes)
                axes[0, 1].set_title('Attention History')
        else:
            axes[0, 1].text(0.5, 0.5, 'No Attention History Data',
                        ha='center', va='center', transform=axes[0, 1].transAxes)
            axes[0, 1].set_title('Attention History')

        if self.feature_importance_history and len(self.feature_importance_history) > 0:
            try:
```

```
            axes[1, 0].plot(self.feature_importance_history, marker='s', color='green',
                            linewidth=2, markersize=4)
            axes[1, 0].set_title('Feature Importance History')
            axes[1, 0].set_xlabel('Epoch')
            axes[1, 0].set_ylabel('Average Feature Importance')
            axes[1, 0].grid(True, alpha=0.3)
        except Exception as e:
            print(f"绘制特征重要性历史时出错: {e}")
            axes[1, 0].text(0.5, 0.5, 'No Feature Importance History',
                            ha='center', va='center', transform=axes[1, 0].transAxes)
            axes[1, 0].set_title('Feature Importance History')
    else:
        axes[1, 0].text(0.5, 0.5, 'No Feature Importance History',
                        ha='center', va='center', transform=axes[1, 0].transAxes)
        axes[1, 0].set_title('Feature Importance History')
    try:
        if self.train_losses and self.val_losses:
            epochs = range(len(self.train_losses))
            color = 'tab:red'
            axes[1, 1].set_xlabel('Epoch')
            axes[1, 1].set_ylabel('Loss', color=color)
            line1 = axes[1, 1].plot(epochs, self.train_losses, label='Train Loss',
                                    color=color, linewidth=2)[0]
            line2 = axes[1, 1].plot(epochs, self.val_losses, label='Val Loss',
                                    color='tab:orange', linewidth=2)[0]
            axes[1, 1].tick_params(axis='y', labelcolor=color)
            axes[1, 1].grid(True, alpha=0.3)

            if self.train_accs and self.val_accs:
                ax2 = axes[1, 1].twinx()
                color = 'tab:blue'
                ax2.set_ylabel('Accuracy (%)', color=color)
                line3 = ax2.plot(epochs, self.train_accs, label='Train Acc',
                                 color=color, linestyle='--', linewidth=2)[0]
                line4 = ax2.plot(epochs, self.val_accs, label='Val Acc',
                                 color='tab:green', linestyle='--', linewidth=2)[0]
                ax2.tick_params(axis='y', labelcolor=color)

                lines = [line1, line2, line3, line4]
                labels = [l.get_label() for l in lines]
                axes[1, 1].legend(lines, labels, loc='upper center',
                                  bbox_to_anchor=(0.5, -0.15), ncol=4)

            axes[1, 1].set_title('Training History')
```

```
        except Exception as e:
            print(f"绘制训练历史时出错: {e}")
            axes[1, 1].text(0.5, 0.5, 'No Training History Data',
                            ha='center', va='center', transform=axes[1, 1].transAxes)
            axes[1, 1].set_title('Training History')

        plt.tight_layout()

        save_path = f"{self.model_name}_explanations_epoch_{epoch}.png"
        plt.savefig(save_path, dpi=300, bbox_inches='tight')
        print(f"可解释性图表已保存到: {save_path}")
        plt.show()

    def explain_predictions(self, test_loader, num_samples=5):
        self.model.eval()
        explained_samples = 0

        with torch.no_grad():
            for inputs, freq_inputs, features, labels in test_loader:
                if explained_samples >= num_samples:
                    break

                inputs = inputs.to(self.device)
                freq_inputs = freq_inputs.to(self.device)
                features = features.to(self.device)
                labels = labels.to(self.device).squeeze()

                outputs, _ = self.model(inputs, freq_inputs, features)
                predictions = torch.argmax(outputs, dim=1)

                for i in range(min(inputs.size(0), num_samples - explained_samples)):
                    print(f"\n=== 样本 {explained_samples + 1} 解释 ===")
                    print(f"真实标签: {labels[i].item()}, 预测标签: {predictions[i].item()}")

                    self.explainer.analyze_prediction(
                        inputs[i], outputs[i], labels[i].item())

                    explained_samples += 1

# 更新主函数以使用可解释模型
def main_explainable():
    device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
    print(f"使用设备: {device}")
```

```
    data_path = "轴承故障分类数据集"   # 实际路径

    feature_names = [
        '均值', '标准差', '最大值', '最小值', '中位数',
        '25%分位数', '75%分位数', 'RMS', '峰值', '峰值因子',
        '波形因子', '能量', '平均差分'
    ]

    class_names = [
        'B007', 'B014', 'B021', 'B028',   # 钢球故障
        'IR007', 'IR014', 'IR021', 'IR028',   # 内圈故障
        'OR007', 'OR014', 'OR021',   # 外圈故障（无 028）
        'Normal'   # 正常
    ]

    dataset_builder = BearingDatasetBuilder()
    X_train, X_val, X_test, y_train, y_val, y_test, filenames_train, filenames_val, filenames_test =
dataset_builder.load_datasets(data_path)
    train_dataset = BearingDataset(X_train, y_train, dataset_builder.features_dict, filenames_train)
    val_dataset = BearingDataset(X_val, y_val, dataset_builder.features_dict, filenames_val)
    test_dataset = BearingDataset(X_test, y_test, dataset_builder.features_dict, filenames_test)

    batch_size = 32
    train_loader = DataLoader(train_dataset, batch_size=batch_size, shuffle=True)
    val_loader = DataLoader(val_dataset, batch_size=batch_size, shuffle=False)
    test_loader = DataLoader(test_dataset, batch_size=batch_size, shuffle=False)
    model = ExplainableBearingQTFFC(num_classes=12, handcrafted_feature_dim=13)

    print(f"可解释模型参数数量: {sum(p.numel() for p in model.parameters()):,}")
    criterion = nn.CrossEntropyLoss()
    optimizer = optim.Adam(model.parameters(), lr=0.001, weight_decay=1e-5)
    scheduler = StepLR(optimizer, step_size=20, gamma=0.1)
    trainer = ExplainableBearingTrainer(
        model, device,
        model_name="explainable_bearing_qtffc",
        feature_names=feature_names,
        class_names=class_names
    )

    train_losses, val_losses, train_accs, val_accs = trainer.train(
        train_loader, val_loader, criterion, optimizer, scheduler, num_epochs=50
    )
    model.load_state_dict(torch.load("explainable_bearing_qtffc_best.pth"))
    test_loss, test_acc, all_preds, all_labels = trainer.test(test_loader, criterion)
```

```
    print("\n===  开始解释预测结果  ===")
    trainer.explain_predictions(test_loader, num_samples=3)

    return trainer, test_loss, test_acc

if __name__ == "__main__":
    #  运行可解释版本
    explainable_trainer, test_loss, test_acc = main_explainable()
```